

Neurapy

Release v5.1.88

Neura Robotics GmbH

February 12, 2026



INDEX

1	General safety notice	1
2	Neurapy	2
3	Offline Simulation Setup	23
4	API Installation	34
5	API Reference	40
6	Examples	227
7	Sample Applications	255
8	TCP-flange connection pin outs for LARA	279
9	TCP-flange connection pin outs for Maira	280
	Python Module Index	282
	Index	283

GENERAL SAFETY NOTICE



CAUTION

This document covers **advanced robot functions** that require a high level of technical knowledge and operational experience. Improper handling of these functions can result in **serious injury, equipment damage, or operational failure**.

- **Trained personnel only:** only individuals with verified expertise in robotics should attempt to use these advanced functions. If you lack the necessary experience, do not proceed. Fully understand the robot's user, software and safety manual.
- **Workspace safety:** maintain a clear workspace and ensure all operators and bystanders are aware of the high-risk nature of advanced operations.
- **Equipment check:** perform a comprehensive check of the robot and all peripheral equipment before initiating advanced functions to confirm they are in correct working order.

NEURAPY

NeuraPy is an easy-to-use, extendable, robot-agnostic Python API for Neura's control software, designed for application developers.

2.1 Robot Class

NeuraPy provides a *Robot* class that allows access to all the functionalities and static parameters of the robot. These can be accessed as methods and attributes of an instantiation of the *Robot* class.

2.2 Parameters

- *robot_name*
- *dof*
- *payload*
- *controller_ip*
- *version*

2.3 Functionalities

 Warning:

The function *init_program* is deprecated and will be removed in future versions. It is no longer required to call this function before motion commands. However, it is still required to call *stop* after motion commands at the end of the script or before changing the mode (e.g., from automatic mode to teach mode).

 Warning:

The following functions have been removed from the Robot class. These functions were deprecated in previous versions and are no longer available in the current release of Neurapy. Use the corresponding functions below for similar functionality:

- *get_current_cartesian_pose* → *get_tcp_pose_quaternion*
- *get_current_cartesian_pose_with_timestamp* → *get_tcp_pose_quaternion_with_timestamp*
- *get_mode* →
 - *is_robot_in_teach_mode*
 - *is_robot_in_automatic_mode*
 - *is_robot_in_semi_automatic_mode*

- `get_sim_or_real` → `is_robot_in_simulation`
- `get_tool_flange_pose` → `get_flange_pose`
- `get_zerog_status` → `is_free_drive_mode_enabled`
- `ik_fk` →
 - `compute_forward_kinematics`
 - `compute_inverse_kinematics`
- `override` →
 - `get_override`
 - `set_override`
- `power` →
 - `power_on`
 - `power_off`
- `record_path` → `move_recorded_path`
- `robot_status` →
 - `get_current_joint_angles`
 - `get_current_joint_angles_with_timestamp`
 - `get_tcp_pose_quaternion`
 - `get_tcp_pose_quaternion_with_timestamp`
 - `get_current_joint_torques`
 - `get_current_joint_torques_with_timestamp`
 - `get_current_joint_velocities_with_timestamp`
 - `get_current_load_side_encoder_values`
 - `get_load_side_encoder_values_with_timestamp`
 - `get_current_motor_side_encoder_values`
 - `get_motor_side_encoder_values_with_timestamp`
- `set_mode` →
 - `switch_to_teach_mode`
 - `switch_to_automatic_mode`
 - `switch_to_semi_automatic_mode`
- `set_sim_real` →
 - `switch_to_real`
 - `switch_to_simulation`
- `start_external_interface` →
 - `activate_servo_interface`
 - `activate_ros_interface`
 - `activate_ethernetIP_interface`
 - `activate_opcua_interface`
 - `activate_ethernetcat_interface`
 - `activate_profinet_interface`
- `stop_external_interface` →
 - `deactivate_servo_interface`
 - `deactivate_ros_interface`
 - `deactivate_ethernetIP_interface`
 - `deactivate_opcua_interface`
 - `deactivate_ethernetcat_interface`
 - `deactivate_profinet_interface`
- `zero_g` →
 - `turn_on_free_drive_mode`
 - `turn_off_free_drive_mode`
- `create_external_source_app` →
 - `create_external_device`

Refer to the updated API documentation for details on these functions.

S.No.	Functionality	Action
1.	<i>activate_ethercat_interface</i>	Activate the Ethercat/el6695 interface.
2.	<i>activate_ethernetIP_interface</i>	Activate the Ethernet/IP interface.
3.	<i>activate_modbus_interface</i>	Activate the Modbus interface.
4.	<i>activate_opcua_interface</i>	Activate the OPC UA interface.
5.	<i>activate_profinet_interface</i>	Activate the Profinet interface.
6.	<i>activate_ros_interface</i>	Activate the ROS interface.
7.	<i>activate_servo_interface</i>	Activate the Servo interface.
8.	<i>check_circular_feasibility</i>	Checks if circular motion over all poses is feasible.
9.	<i>check_composite_feasibility</i>	Checks if a combination of linear and circular motions is feasible.
10.	<i>check_joint_feasibility</i>	This method takes the following arguments/keyword arguments:
11.	<i>check_joint_relative_feasibility</i>	To move the robot relative to the current position by the given cartesian offset. The motion is executed as joint motion.
12.	<i>check_linear_feasibility</i>	To plan a move linear path across the given poses.
13.	<i>check_linear_relative_feasibility</i>	To move the robot relative to the current position by the given cartesian offset. The motion is executed as Cartesian motion.
14.	<i>check_move_spline_feasibility</i>	To check whether the motion is feasible with the inputs provided.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
15.	<i>check_recorded_path_feasibility</i>	To check if given motion is feasible.
16.	<i>compute_forward_kinematics</i>	Compute forward kinematics (end-effector's Cartesian pose) for the given joint configuration.
17.	<i>compute_inverse_kinematics</i>	Compute inverse kinematics(joint configuration) for the given end-effector's cartesian pose.
18.	<i>connect_external_device</i>	Connects to an external device.
19.	<i>convert_euler_to_quaternion_pose</i>	Convert a pose with Euler angles to quaternions.
20.	<i>convert_quaternion_to_euler_pose</i>	Convert a pose with quaternion representation to a pose with Euler angles.
21.	<i>copy_pose</i>	Replicate a pose(Point/Frame) in a new parent frame.
22.	<i>create_external_device</i>	Factory method to create an ExternalDeviceApp object, which can be used to interact with external devices.
23.	<i>create_point</i>	Store a point in the database.
24.	<i>create_reference_frame</i>	Store a user-defined reference frame in the database.
25.	<i>create_tool</i>	Create a new tool in the robot's database using the provided tool data.
26.	<i>deactivate_ethercat_interface</i>	Deactivate the Ethercat interface.
27.	<i>deactivate_ethernetIP_interface</i>	Deactivate the Ethernet/IP interface.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
28.	<i>deactivate_modbus_interface</i>	Deactivate the Modbus interface.
29.	<i>deactivate_opcua_interface</i>	Deactivate the OPCUA interface.
30.	<i>deactivate_profinet_interface</i>	Deactivate the Profinet interface.
31.	<i>deactivate_ros_interface</i>	Deactivate the ROS interface.
32.	<i>deactivate_servo_interface</i>	Deactivate the Servo interface.
33.	<i>delete_point</i>	Delete a point from the database.
34.	<i>delete_points_in_frame</i>	Delete all points in a user-defined reference frame in the database.
35.	<i>delete_pre_planned_motion</i>	Deletes pre planned motion.
36.	<i>delete_reference_frame</i>	Delete a user-defined reference frame in the database.
37.	<i>disable_collision_detection</i>	Disable collision detection on the robot.
38.	<i>disable_friction_mode</i>	To turn off friction mode (smooth free drive mode).
39.	<i>disable_reflex</i>	Disable reflex after collision.
40.	<i>disconnect_external_device</i>	Disconnects from an external device.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
41.	<i>enable_collision_detection</i>	Enable collision detection on the robot.
42.	<i>enable_friction_mode</i>	To turn on friction mode (smooth free drive mode). Free drive mode needs to be activated beforehand.
43.	<i>enable_reflex</i>	Enable reflex after collision.
44.	<i>encoder2rad</i>	Convert encoder impulses to joint angles in radians.
45.	<i>execute_closest_orthogonal_tcp_align</i>	Execute the closest orthogonal axes-aligned TCP alignment motion.
46.	<i>execute_exact_tcp_align</i>	Execute the exact axes-aligned TCP alignment motion.
47.	<i>execute_external_device_function</i>	Executes a function of an external device.
48.	<i>execute_external_device_safety_stop</i>	Executes the safety stop function on an external device.
49.	<i>execute_pre_plan_motion</i>	Executes pre planned motion from the motion storage.
50.	<i>execute_pre_plan_motion_from_json</i>	Executes pre planned motion from json file.
51.	<i>execute_program</i>	To run a program created from teach pendant's tree program creator
52.	<i>execute_z_axis_tcp_align</i>	Execute the z-axis-aligned TCP alignment motion.
53.	<i>executor</i>	Execute already planned motion.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
54.	<i>get_analog_input</i>	Retrieve the value of an analog input.
55.	<i>get_analog_output</i>	Retrieve the value of an analog output.
56.	<i>get_commanded_joint_angles</i>	Get commanded joint angles of the robot.
57.	<i>get_commanded_joint_angles_with_timestamp</i>	Get the comamnded joint angles of the robot along with UTC timestamp.
58.	<i>get_current_joint_angles</i>	Get current joint angles of the robot.
59.	<i>get_current_joint_angles_with_timestamp</i>	Get the current joint angles of the robot along with UTC timestamp.
60.	<i>get_current_joint_torques</i>	Get the current joint torques of the robot.
61.	<i>get_current_joint_torques_with_timestamp</i>	Get the current joint torques of the robot along with UTC timestamp.
62.	<i>get_current_joint_velocities</i>	Get current joint velocities of the robot.
63.	<i>get_current_joint_velocities_with_timestamp</i>	Get the current joint velocities of the robot along with UTC timestamp.
64.	<i>get_current_load_side_encoder_values</i>	Get the current load side encoder ticks of the robot.
65.	<i>get_current_motor_side_encoder_values</i>	Get the current motor side encoder ticks of the robot .
66.	<i>get_current_tool_cogs</i>	Get the current tool center of gravity (COG) in XYZ directions(measured from robot's flange frame/tool mounting point).

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
67.	<i>get_current_tool_inertias</i>	Get the current tool inertias Ixx, Iyy, Izz, Ixy, Ixz, and Iyz.
68.	<i>get_current_tool_mass</i>	Get the current tool mass.
69.	<i>get_current_tool_properties</i>	Get the current tool properties.
70.	<i>get_current_tool_rpy_offsets</i>	Get the current tool roll, pitch, and yaw (RPY) offsets in radians.
71.	<i>get_current_tool_translation_offsets</i>	Get the current tool translation offsets in XYZ directions.
72.	<i>get_diagnostics</i>	Method to query current robot diagnostics
73.	<i>get_digital_input</i>	Get the value of a digital input.
74.	<i>get_digital_output</i>	Get the value of a digital output.
75.	<i>get_doc</i>	Get the description and sample usage of the specified function.
76.	<i>get_encoder_offsets</i>	To retrieve the encoder offsets of a robot.
77.	<i>get_errors</i>	Query the list of errors present on the robot.
78.	<i>get_ext_digital_input</i>	Get the value of an external digital input.
79.	<i>get_ext_digital_output</i>	Get the value of an external digital output.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
80.	<i>get_flange_pose</i>	Get the current flange pose in XYZRPY format.
81.	<i>get_flange_pose_quaternion</i>	Get the current flange pose in XYZwxyz format.
82.	<i>get_flange_pose_quaternion_with_timestamp</i>	Get the current flange pose in XYZwxyz format with UTC timestamp.
83.	<i>get_flange_pose_with_timestamp</i>	Get the current tool flange pose in XYZRPY format with UTC timestamp.
84.	<i>get_io_configuration</i>	Get the number of available IOs.
85.	<i>get_io_status</i>	Get all the available IOs and it's values
86.	<i>get_joint_acceleration</i>	Get the current acceleration value for joint motion.
87.	<i>get_joint_speed</i>	Get the current speed value for joint motion.
88.	<i>get_linear_acceleration</i>	Get the current acceleration value for linear motion.
89.	<i>get_linear_speed</i>	Get the current speed value for linear motion.
90.	<i>get_load_side_encoder_values_with_timestamp</i>	Get the current load side encoder ticks of the robot along with UTC timestamp.
91.	<i>get_motor_side_encoder_values_with_timestamp</i>	Get the current motor side encoder ticks of the robot along with UTC timestamp.
92.	<i>get_override</i>	Get the current override value.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
93.	<i>get_point</i>	Retrieves a point from the database based on the given name.
94.	<i>get_point_names</i>	Retrieves a list of names of all saved points in the database.
95.	<i>get_power_status</i>	Query the power status of the robot.
96.	<i>get_program_names</i>	Retrieves list of program names from the robot's database
97.	<i>get_reference_frame</i>	Retrieve the coordinates and orientations of a user-defined reference frame.
98.	<i>get_reference_frame_with_offset</i>	Retrieve a reference frame with an applied offset to its position.
99.	<i>get_register_bytes</i>	Returns a python "bytes" object from a T2O or O2T register specified by its tag name and index range
100.	<i>get_register_values</i>	Returns values from a T2O or O2T register specified by its tag name and index range from first to last element
101.	<i>get_safety</i>	Get the current status of Safety Mode.
102.	<i>get_selected_fts_data</i>	Retrieves the selected external fts from the robot's database.
103.	<i>get_selected_tool_data</i>	Retrieves the selected tool data from the robot's database.
104.	<i>get_selected_tool_name</i>	Retrieves the selected tool name from the robot's database.
105.	<i>get_servo_trajectory_scaling_factor</i>	Method to get the trajectory scaling factor from ServoJ

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
106.	<i>get_tcp_force</i>	Get the current TCP (Tool Center Point) force values of the robot.
107.	<i>get_tcp_pose</i>	to get the current tcp pose in XYZRPY format
108.	<i>get_tcp_pose_quaternion</i>	to get the current tcp pose in XYZwxyz format
109.	<i>get_tcp_pose_quaternion_with_timestamp</i>	Get the current tcp pose in XYZwxyz format along with UTC timestamp
110.	<i>get_tcp_pose_with_timestamp</i>	Get the current robot's TCP pose in XYZRPY format along with UTC timestamp.
111.	<i>get_tcp_speed</i>	Get the current speed of the robot's TCP (Tool Center Point).
112.	<i>get_tcp_torque</i>	Get the current TCP (Tool Center Point) torque values of the robot.
113.	<i>get_tcp_wrench</i>	Get the current TCP (Tool Center Point) wrench values,
114.	<i>get_tool_analog_input</i>	Get the value of a tool analog input.
115.	<i>get_tool_digital_input</i>	Get the value of a tool digital input.
116.	<i>get_tool_digital_output</i>	Get the value of a tool digital output.
117.	<i>get_tools</i>	Retrieves list of tools from the robot's database.
118.	<i>get_warnings</i>	Query the list of warnings present on the robot.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
119.	<i>grasp</i>	To grasp the object with the attached gripper
120.	<i>gripper</i>	Method to control the gripper attached to the robot
121.	<i>io</i>	Access and manipulate (output) I/O values on the robot.
122.	<i>is_collision_enabled</i>	Check if reflex after collision is enabled for the robot.
123.	<i>is_external_device_connected</i>	Checks if an external device is connected.
124.	<i>is_external_device_in_error</i>	Checks if an external device is in error state.
125.	<i>is_free_drive_mode_enabled</i>	Get the status of the robot free drive mode.
126.	<i>is_reflex_enabled</i>	Check if reflex after collision is enabled for the robot.
127.	<i>is_robot_in_automatic_mode</i>	Method to check whether the robot is in automatic mode or not
128.	<i>is_robot_in_collision</i>	Check if the robot is currently in collision.
129.	<i>is_robot_in_semi_automatic_mode</i>	Method to check whether the robot is in semi automatic mode or not
130.	<i>is_robot_in_simulation</i>	Method to check whether the robot is in simulation or not
131.	<i>is_robot_in_teach_mode</i>	Method to check whether the robot is in teach mode or not

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
132.	<i>is_robot_powered_on</i>	Check if the robot is powered on.
133.	<i>jog</i>	Method to jog programatically. This needs to be used in conjunction with <i>turn_on_jog</i> , <i>turn_off_jog</i> methods.
134.	<i>list_methods</i>	To list the available functions in the API
135.	<i>list_recorded_path</i>	To list the recorded path using ZeroG.
136.	<i>load_pre_planned_motion_from_json</i>	Loads pre planned motion from json file and stores it.
137.	<i>lock_cartesian_axes</i>	Lock specific Cartesian axes while in free drive mode.
138.	<i>lock_joint_axes</i>	Lock specific joint axes while in free drive mode.
139.	<i>motion_status</i>	Query the motion status of the robot.
140.	<i>move_circular</i>	To move the robot in a circular path across the given poses.
141.	<i>move_composite</i>	To move the robot in the specified linear and circular motion combinations.
142.	<i>move_joint</i>	To move the robot to specified joint configuration in joint space.
143.	<i>move_joint_relative</i>	To move the robot relative to the current position by the given cartesian offset. The motion is executed as joint motion.
144.	<i>move_linear</i>	To move the robot to specified poses in Cartesian/Task space.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
145.	<i>move_linear_relative</i>	To move the robot relative to the current position by the given cartesian offset. The motion is executed as Cartesian motion.
146.	<i>move_pose</i>	Method to move the input pose(point/frame), which is defined with respect to a parent frame, to a new parent frame.
147.	<i>move_recorded_path</i>	To move the robot in a pre-recorded path.
148.	<i>move_spline</i>	To move the robot along a pre-defined set of joint angles with a Spline trajectory.
149.	<i>movelinear_online</i>	Method to do Servo in Cartesian Space
150.	<i>notify_error</i>	To notify an error message to teach pendant .
151.	<i>notify_info</i>	To notify a info message to teach pendant.
152.	<i>notify_warning</i>	To notify a warning message to teach pendant.
153.	<i>pause</i>	To pause the robot's motion.
154.	<i>plan_move_circular</i>	Plans a motion in a circular path across the given poses.
155.	<i>plan_move_composite</i>	Plans a motion in which linear and circular motions are combined.
156.	<i>plan_move_joint</i>	This method takes the following arguments/keyword arguments:
157.	<i>plan_move_linear</i>	To plan a move linear path across the given poses.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
158.	<i>plan_move_recorded_path</i>	To plan the given pre-recorded path.
159.	<i>plan_move_spline</i>	To plan the given pre-defined path.
160.	<i>plan_realtime_wait</i>	Adds a wait command which is added to the execution pipeline. With this delays are minimized and should be closer than python waits.
161.	<i>power_off</i>	To power off the robot.
162.	<i>power_on</i>	To power on the robot.
163.	<i>pre_plan_move_circular</i>	Pre planning of motion, this allows to plan the motion without adding it for execution!
164.	<i>pre_plan_move_composite</i>	This function allows to pre plan a motion, without adding it to the list for execution.
165.	<i>pre_plan_move_joint</i>	Allows to pre plan the motion and execute it at a later time.
166.	<i>pre_plan_move_joint_relative</i>	Allows to pre plan the motion and execute it at a later time.
167.	<i>pre_plan_move_linear</i>	Allows to pre plan the motion and execute it at a later time.
168.	<i>pre_plan_move_linear_relative</i>	Allows to pre plan the motion and execute it at a later time.
169.	<i>pre_plan_move_recorded_path</i>	Allows to pre plan the motion and adds it at a later time.
170.	<i>pre_plan_move_spline</i>	To pre plan the given pre-defined path.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
171.	<i>pre_plan_realtime_wait</i>	Allows to pre plan the wait and execute it at a later time.
172.	<i>program_status</i>	Query the program status of the robot.
173.	<i>quaternion_to_rpy</i>	Convert a quaternion representation to roll-pitch-yaw (RPY) angles.
174.	<i>rad2encoder</i>	Convert joint angles in radians to encoder impulses.
175.	<i>read_general_purpose_registers</i>	Method to read general purpose registers from a Modbus server.
176.	<i>read_safeio</i>	Read the values of configurable/safe Input/Outputs.
177.	<i>realtime_wait</i>	This function can be used, to add a wait time to the motion buffer.
178.	<i>release</i>	To release the grasped object
179.	<i>remove_load</i>	Remove the load which was set in set_load by setting the load to NoLoad.
180.	<i>reparent_pose</i>	Method which shall change the association of the input pose(point/frame) to a new frame.
181.	<i>reset_axes_lock</i>	Reset all joint and Cartesian axis locks in free drive mode.
182.	<i>reset_collision</i>	Reset the collision state of the robot.
183.	<i>reset_control</i>	To restart the control software running on the robot. Equivalent to reset control option from Teach pendant

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
184.	<i>reset_errors</i>	Method to reset the errors on the robot
185.	<i>reset_external_device_actions</i>	Resets action for all external devices
186.	<i>reset_external_device_error</i>	Resets the error state of an external device.
187.	<i>reset_led_mode</i>	Reset the LED mode to its default state.
188.	<i>reset_warnings</i>	Method to reset/clear the warnings on the robot
189.	<i>rpy_to_quaternion</i>	Convert roll-pitch-yaw (RPY) angles to a quaternion representation.
190.	<i>save_pre_planned_motion_to_json</i>	Saves pre planned motion as json file.
191.	<i>servo_j</i>	Method to do Servo in Joint Space
192.	<i>servo_x</i>	Method to do Servo in Cartesian Space
193.	<i>set_analog_output</i>	Set the analog outputs of the control box.
194.	<i>set_current_position_as_zero</i>	Sets the current joint position of the given axis to zero.
195.	<i>set_dbout_register</i>	Writes values into an element range of the DBOUT register
196.	<i>set_digital_output</i>	Set the digital outputs of the control box.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
197.	<i>set_encoder_offsets</i>	Sets the encoder offsets for the robot's joints relative or absolute.
198.	<i>set_ext_digital_output</i>	Set the digital outputs of the external I/O module.
199.	<i>set_external_device_actions</i>	Sets actions for an external device.
200.	<i>set_intout_register</i>	Writes values or octets into an element range of the INTOUT register
201.	<i>set_joint_acceleration</i>	Set the acceleration value for joint motion.
202.	<i>set_joint_speed</i>	Set the speed value for joint motion.
203.	<i>set_led_mode</i>	Turn on the LED of the robot using the specified mode, a list of colors, and frequency.
204.	<i>set_linear_acceleration</i>	Set the acceleration value for linear motion.
205.	<i>set_linear_speed</i>	Set the speed value for linear motion.
206.	<i>set_load</i>	Sets a load, e.g., an object that is grasped. If the load is present in the database, it is updated. If it is not present, it is first added to the database and then updated.
207.	<i>set_opcua_msg</i>	Send a message to the OPC UA server running on the control box.
208.	<i>set_override</i>	Set the override value.
209.	<i>set_safety</i>	Method to enable or disable the safety mode. If set to True, max speed is slashed to 25%.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
210.	<i>set_tool</i>	Notify the gripper/tool change to software components after the physical change.
211.	<i>set_tool_digital_output</i>	Set the target value on the specified tool digital output.
212.	<i>set_tool_digital_outputs</i>	Set digital outputs of the tool.
213.	<i>speed_j</i>	Method to do Joint Velocity Control
214.	<i>speed_x</i>	Method to do Cartesian Velocity Control in Cartesian Space
215.	<i>stop</i>	To stop the robot's motion and to terminate the script execution. This needs to be added at the end of the script(only once) for a proper script termination
216.	<i>stop_movelinear_online</i>	Method to stop MoveLinearOnline motion if running
217.	<i>switch_to_automatic_mode</i>	Switch the robot to Automatic mode.
218.	<i>switch_to_real</i>	Switches the robot's running context to real mode.
219.	<i>switch_to_semi_automatic_mode</i>	Switch the robot to Semi Automatic mode.
220.	<i>switch_to_simulation</i>	Switches the robot's running context to simulation mode.
221.	<i>switch_to_teach_mode</i>	Switch the robot to Teach mode.
222.	<i>tare_fts</i>	Set the property for taring force torque sensor to true when called and clears the

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
223.	<i>terminate_current_motion</i>	This function terminates the execution of current motion.
224.	<i>turn_off_free_drive_mode</i>	To turn off free drive mode
225.	<i>turn_off_jog</i>	Method to disable jogging programatically. This needs to be used in conjunction with jog, turn_on_jog methods.
226.	<i>turn_on_free_drive_mode</i>	To turn on free drive mode
227.	<i>turn_on_jog</i>	Method to enable jogging programatically. This needs to be used in conjunction with jog, turn_off_jog methods.
228.	<i>unpause</i>	To unpause the robot's motion.
229.	<i>update_current_tool_parameters</i>	Update the current gripper/tool parameters in the tool list and the selected tool properties.
230.	<i>update_reference_frame</i>	Update a user-defined reference frame in the database.
231.	<i>update_tool_parameters</i>	Update the gripper/tool parameters in the tool list with a given tool_name.
232.	<i>wait</i>	Wait for a specified signal.
233.	<i>wait_for_analog_input</i>	Wait for an analog input signal to match the expected value.
234.	<i>wait_for_digital_input</i>	Wait for a digital input signal to match the expected value.
235.	<i>wait_for_digital_input_timer_off_delay</i>	Wait for a given delay and returns after the specified digital input signal reaches low.

continues on next page

Table 1 – continued from previous page

S.No.	Functionality	Action
236.	<i>wait_for_digital_input_timer_on_delay</i>	Wait for a given delay and returns after the specified digital input signal reaches high.
237.	<i>wait_for_tool_analog_input</i>	Wait for a tool-specific analog input signal to match the expected value.
238.	<i>wait_for_tool_digital_input</i>	Wait for a tool-specific digital input signal to match the expected value.
239.	<i>wait_for_tool_digital_input_timer_off_delay</i>	Wait for a given delay and return after the specified tool digital input signal reaches low.
240.	<i>wait_for_tool_digital_input_timer_on_delay</i>	Wait for a given delay and return after the specified tool digital input signal reaches high.

OFFLINE SIMULATION SETUP

The offline simulation setup enables offline programming and simulation of robot programs.

The offline simulation software is provided as a virtual machine (VM) image, tested exclusively on VirtualBox 7.0 and above.

Using this VM image on other virtualization software may cause unexpected issues.

3.1 System Requirements

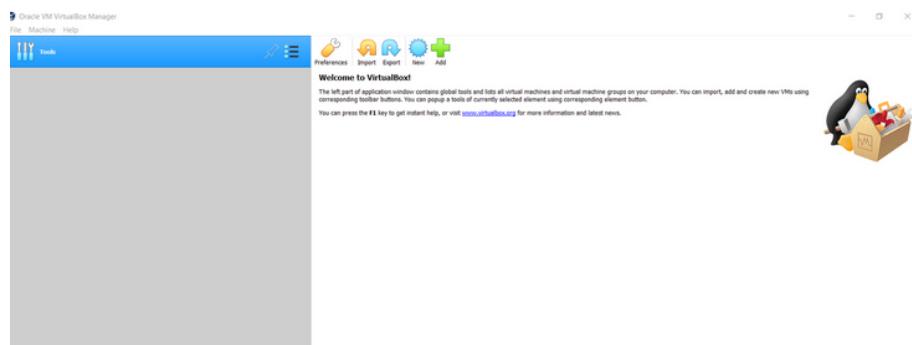
To ensure optimal performance, the following system requirements are recommended:

- **Hard Drive Space:** A minimum of 30GB of free space on your hard drive is required.
- **RAM:** You should have at least 10GB of RAM to ensure smooth performance.
- **Processor:** An 8-12 core processor is recommended to handle the virtual machine efficiently.

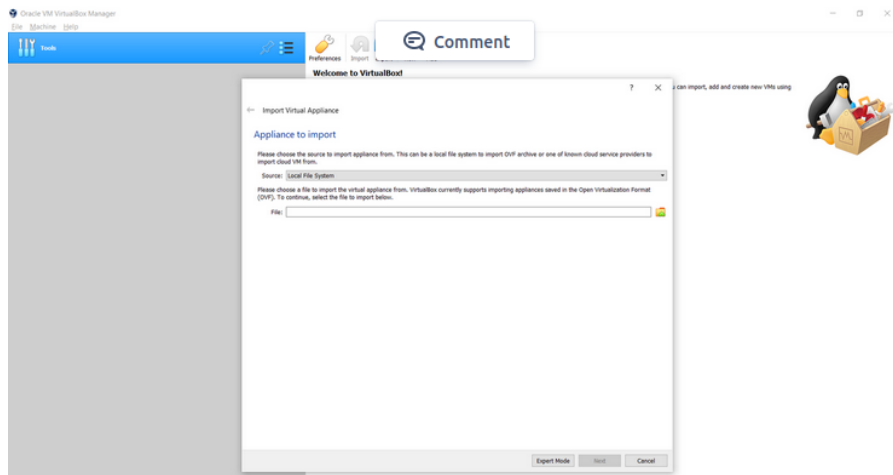
Meeting these system requirements will help ensure that your virtual machine runs smoothly without performance issues.

3.2 Installation Guidelines for Windows

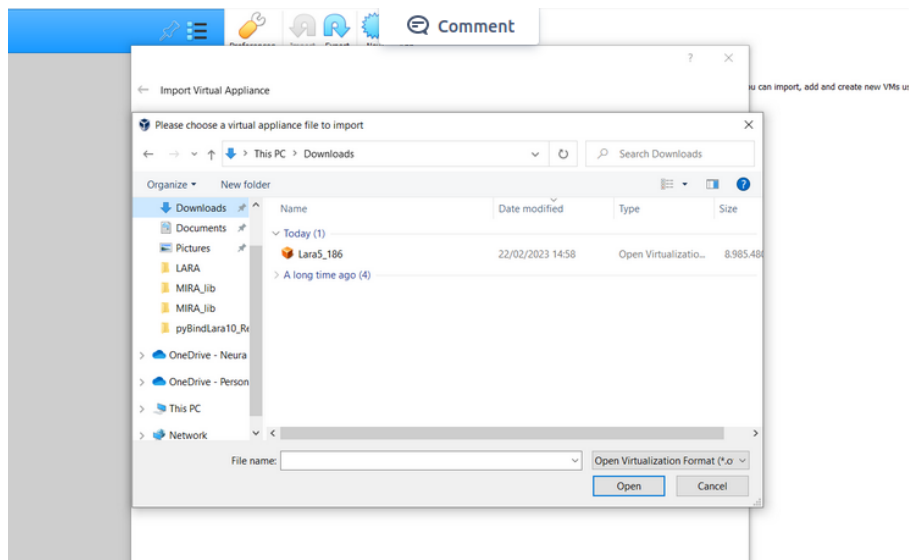
1. Download the Windows-compatible version of VirtualBox from [virtualbox.org](https://www.virtualbox.org) and install it.
2. Download the offline simulation software VM image (LARA5).
3. Open the VirtualBox GUI by navigating to the start menu. You should see a window similar to the one below:



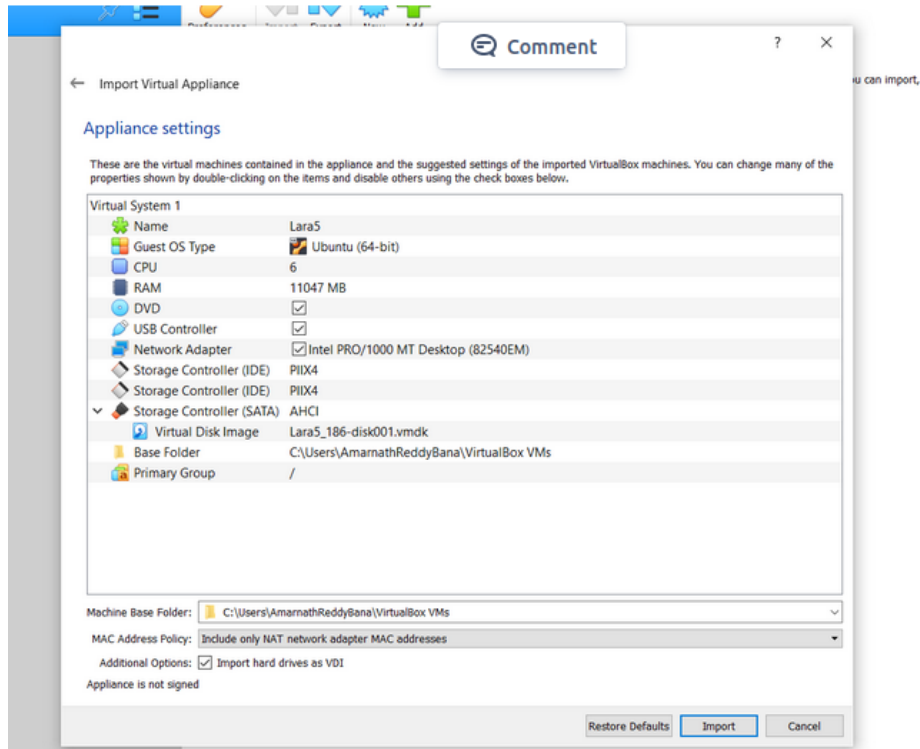
4. Click on the Import Icon. This action will open a second window for importing the Virtual Appliance.



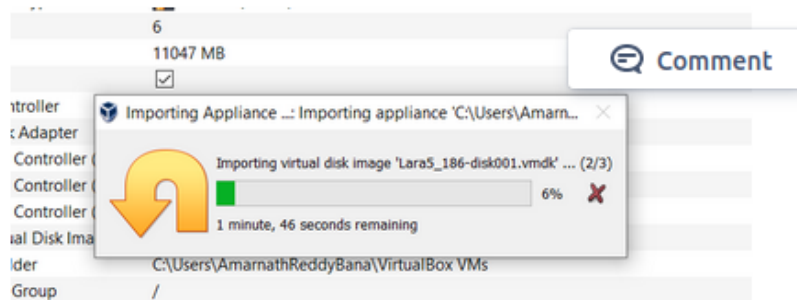
5. Click the folder icon beside **File** to open a browser window. Navigate to the location of the downloaded .ova file (e.g., *Lara5_186.ova*), select the file, and click **Open**.



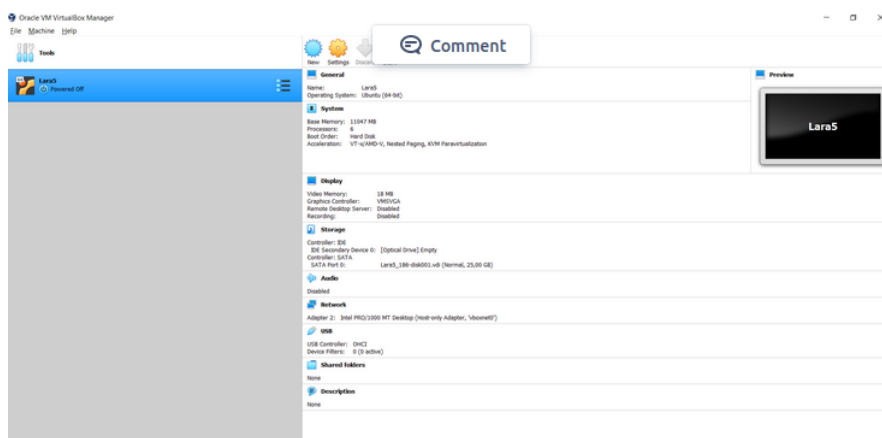
6. Click **Next** to open the configuration settings. It is recommended to retain the default configuration settings. Press **Restore Defaults** before proceeding with the import.



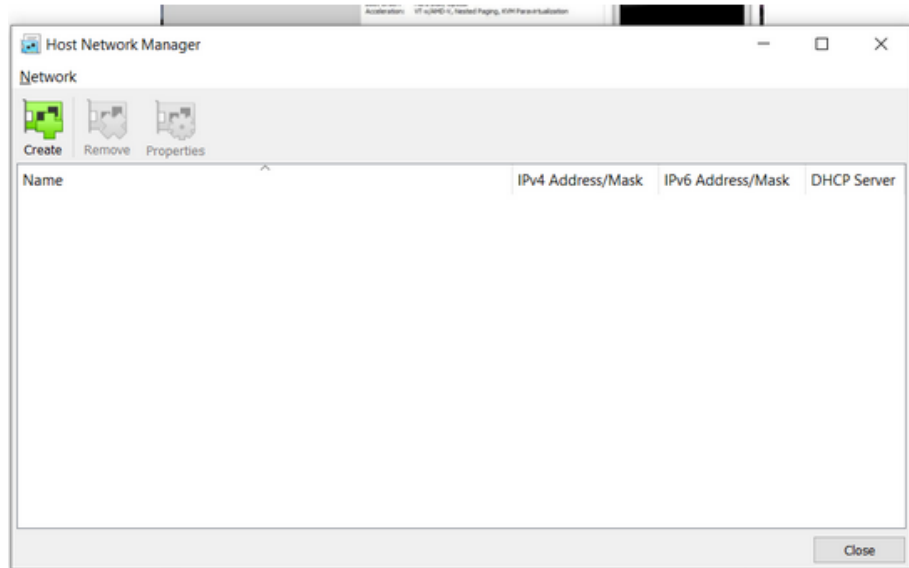
7. Click **Import**, and an acknowledgment window displaying the elapsed time will appear. Allow the process to complete.



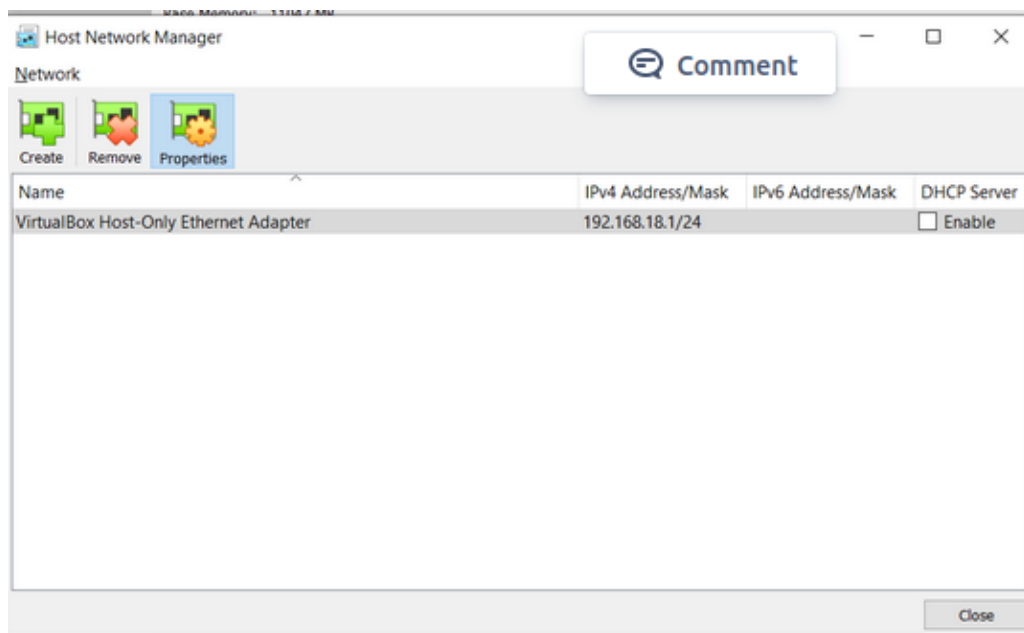
8. After this process is complete, the VirtualBox GUI should resemble the image below:



9. To configure the network adapter, navigate to **File > Host Network Manager**. A window will appear; click **Create**.



10. Once the adapter is created, it should display **VirtualBox Host-Only Ethernet Adapter**. Enable the DHCP Server by clicking the **Enable** checkbox under **DHCP Server** in the top right corner. Next, click on **Properties**.

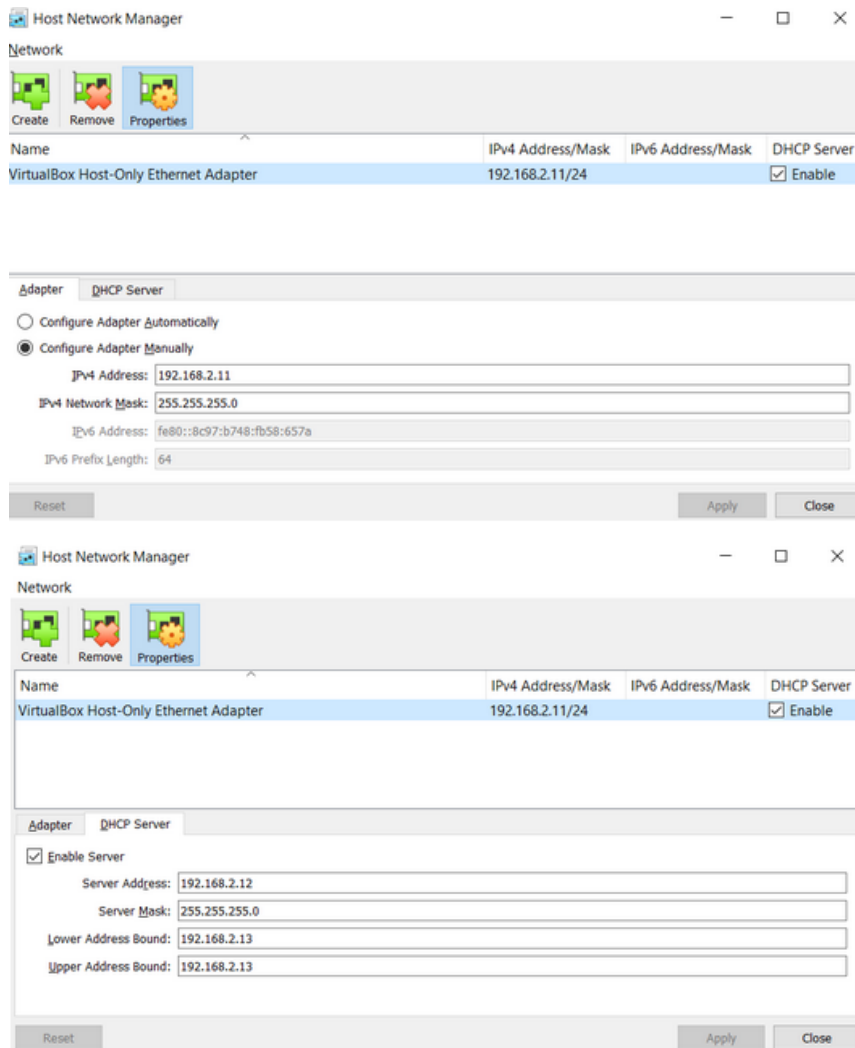


11. Choose the **Configure Adapter Manually** option and adjust the fields as follows:

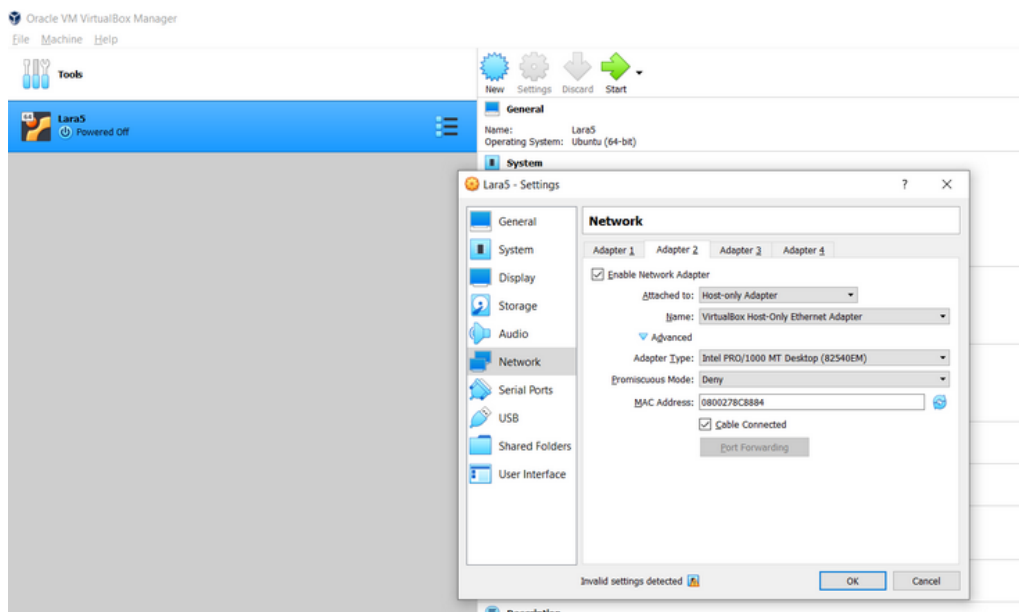
- **IPv4 Address:** 192.168.2.11
- **IPv4 Network Mask:** 255.255.255.0

In the DHCP Server settings, configure the following:

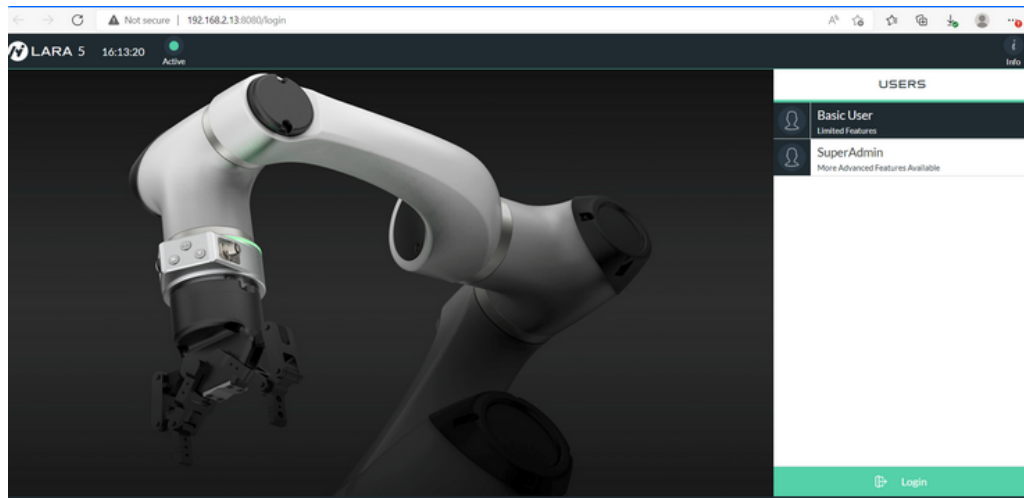
- Click on **Enable Server**.
- **Server Address:** 192.168.2.12
- **Server Mask:** 255.255.255.0
- **Lower Address Bound:** 192.168.2.13
- **Upper Address Bound:** 192.168.2.13



12. Click on **Settings -> Network -> Adapter 1**. If **Enable Network Adapter** is checked, uncheck it first. Then, click on **Adapter 2** and check **Enable Network Adapter**. From the drop-down menu of **Attached to**, select **Host-only Adapter**. In the **Name** field, choose **VirtualBox Host-Only Ethernet Adapter**. The settings should appear as follows:

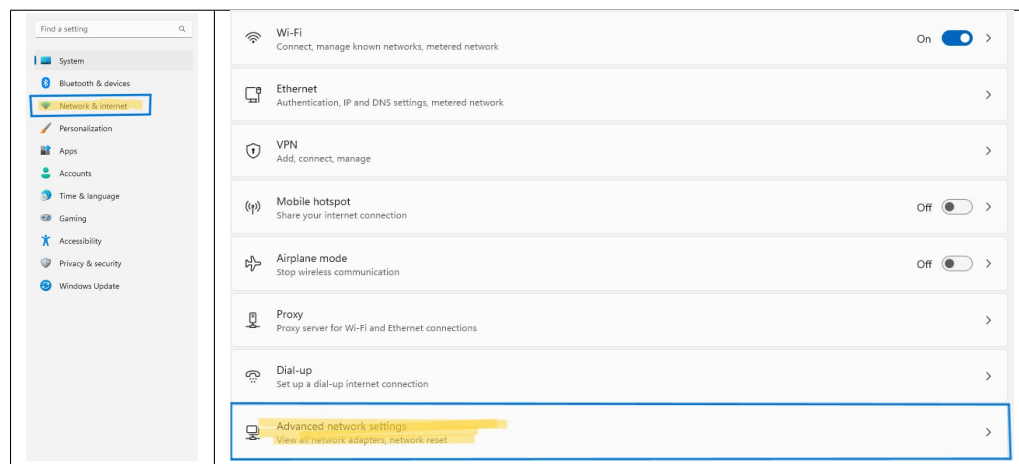


- The configuration of the offline simulation VM image is complete. Start **Lara5** and run it for 30 seconds. To access the GUI, open Chrome and enter `http://192.168.2.13:8080/login`. If all processes were successful, this address should load the GUI interface.

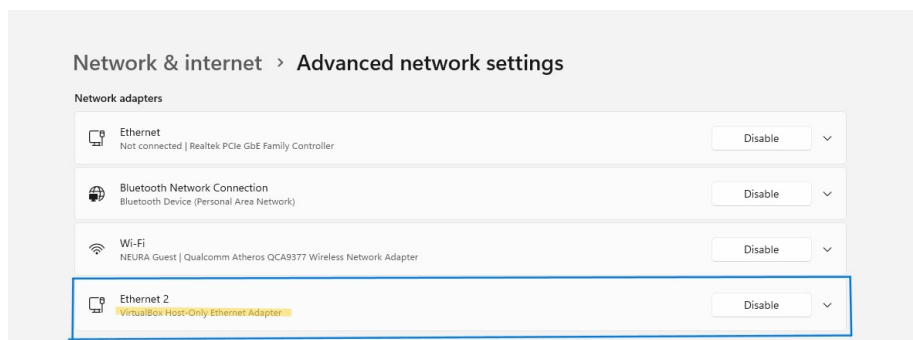


3.3 Troubleshooting Network

- On your Windows PC, click on **Settings** → **Network & Internet** → **Advanced network settings**.



- Click on **Advanced network settings**.



- Verify the status of the Ethernet Adapter for VirtualBox in the **Advanced network settings**. If it is currently enabled, click the **Enable** button to confirm. If it is already enabled by default, click the **Enable** button, wait for it to disable, pause for 3 seconds, and then re-enable it.

3.4 Installation Guidelines for Ubuntu

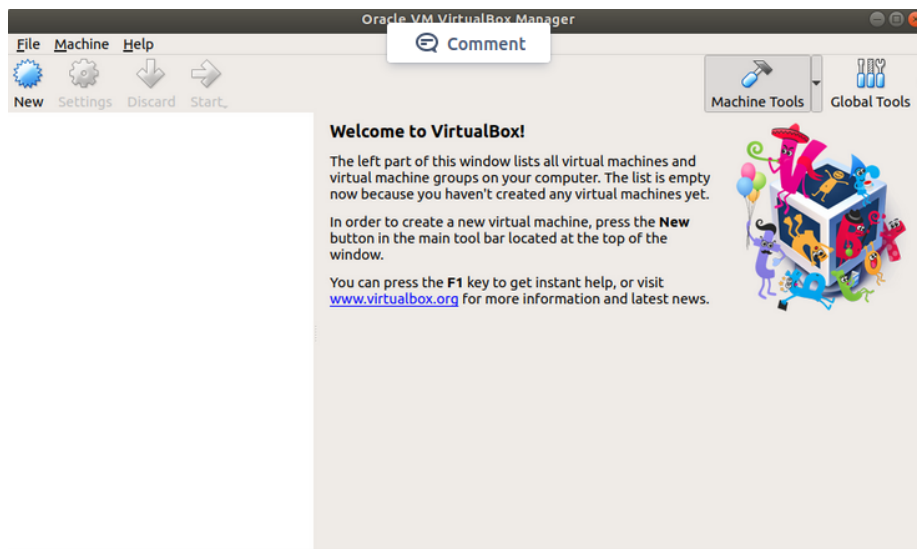
1. Install VirtualBox on Ubuntu using the following commands from the apt repositories:

```
sudo dpkg -i ~/path/virtualbox*deb
```

2. Download the offline simulation software VM image (LARA5).
3. To enable static IP allotment for VirtualBox in a custom subnet, execute the following commands:

```
sudo mkdir /etc/vbox
sudo touch /etc/vbox/networks.conf
echo "* 10.0.0.0/8 192.168.0.0/16" | sudo tee -a /etc/vbox/networks.conf
```

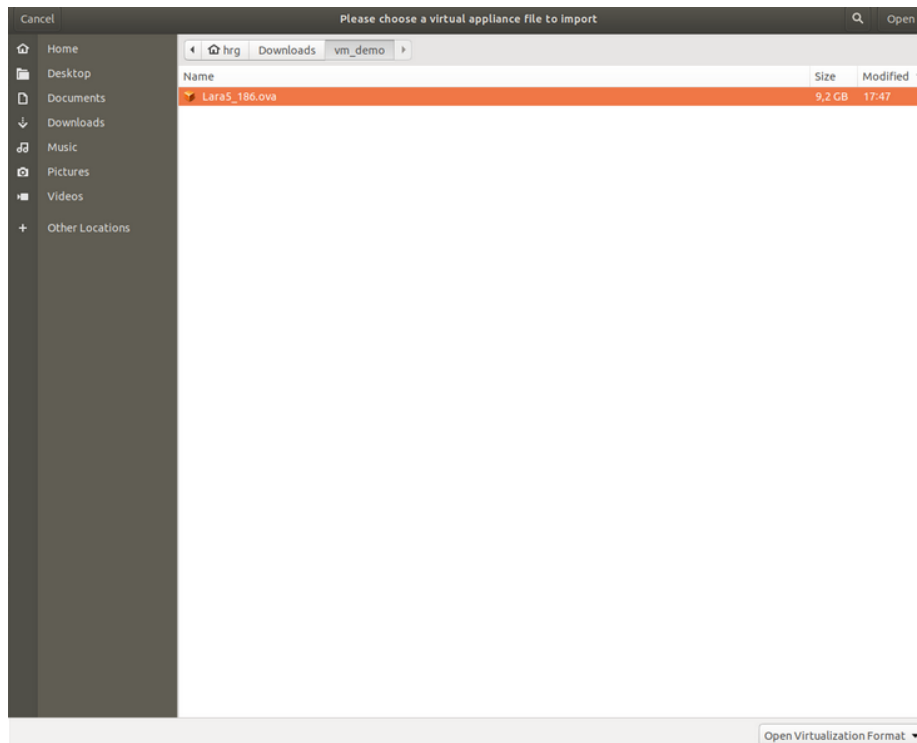
4. Open VirtualBox from the terminal. A window similar to the one below should appear:



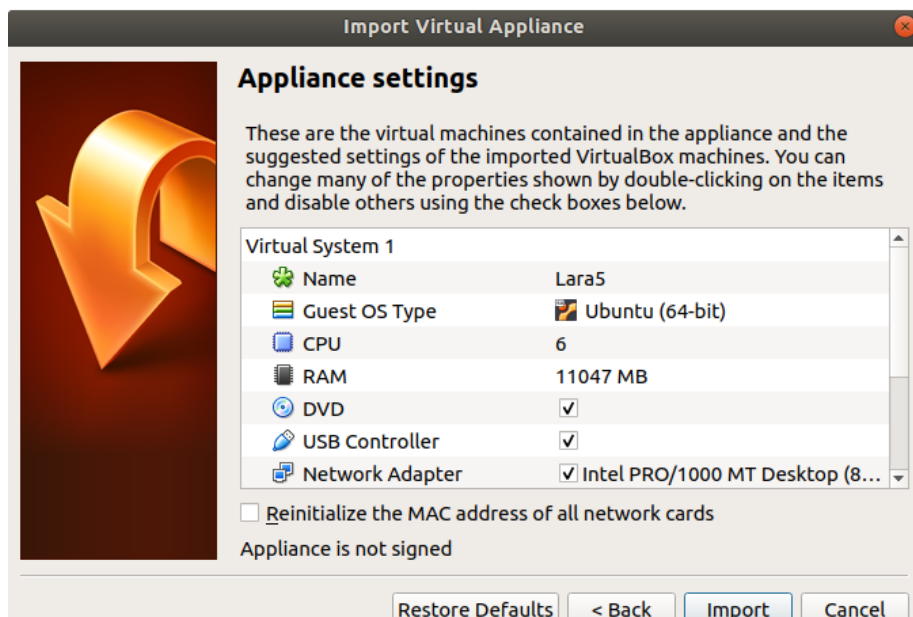
5. Click on the **Import Virtual Appliance from File** tab, and a second window for importing the Virtual Appliance will appear.



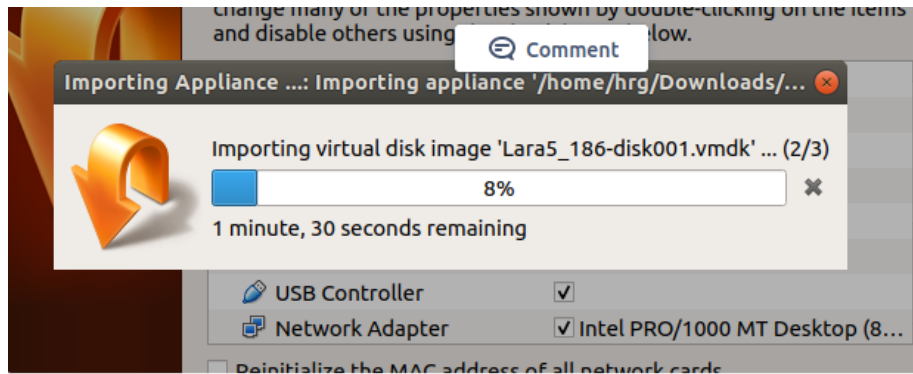
6. Click the folder icon beside **File**. This will open a browser window. Specify the location of the downloaded .ova file (e.g., *Lara5_186.ova*), select the file, and click **Open**.



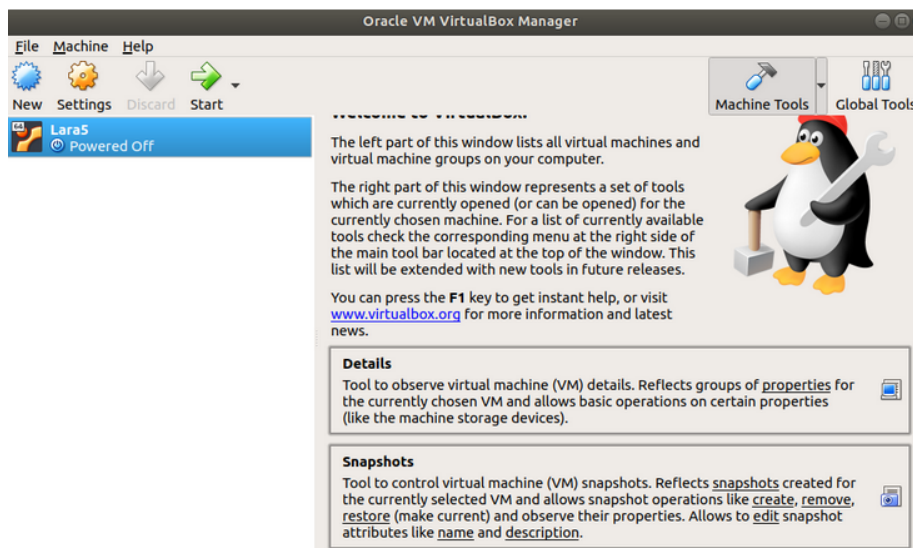
7. Click **Next**, and the configuration settings will appear. It is recommended to retain the default configuration settings. Press **Restore Defaults** before proceeding with the import



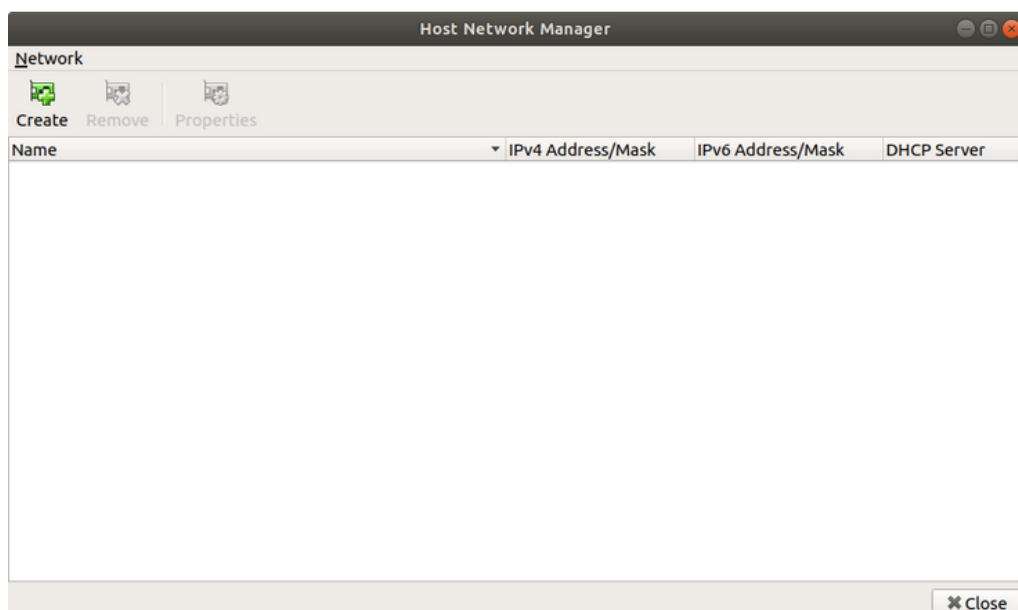
8. Click **Import**, and an acknowledgment window displaying the elapsed time will appear. Allow the process to complete.



9. After this process is complete, the VirtualBox GUI should resemble the image below:



10. To configure the network adapter, navigate to **File > Host Network Manager**. A window will appear; click **Create**.

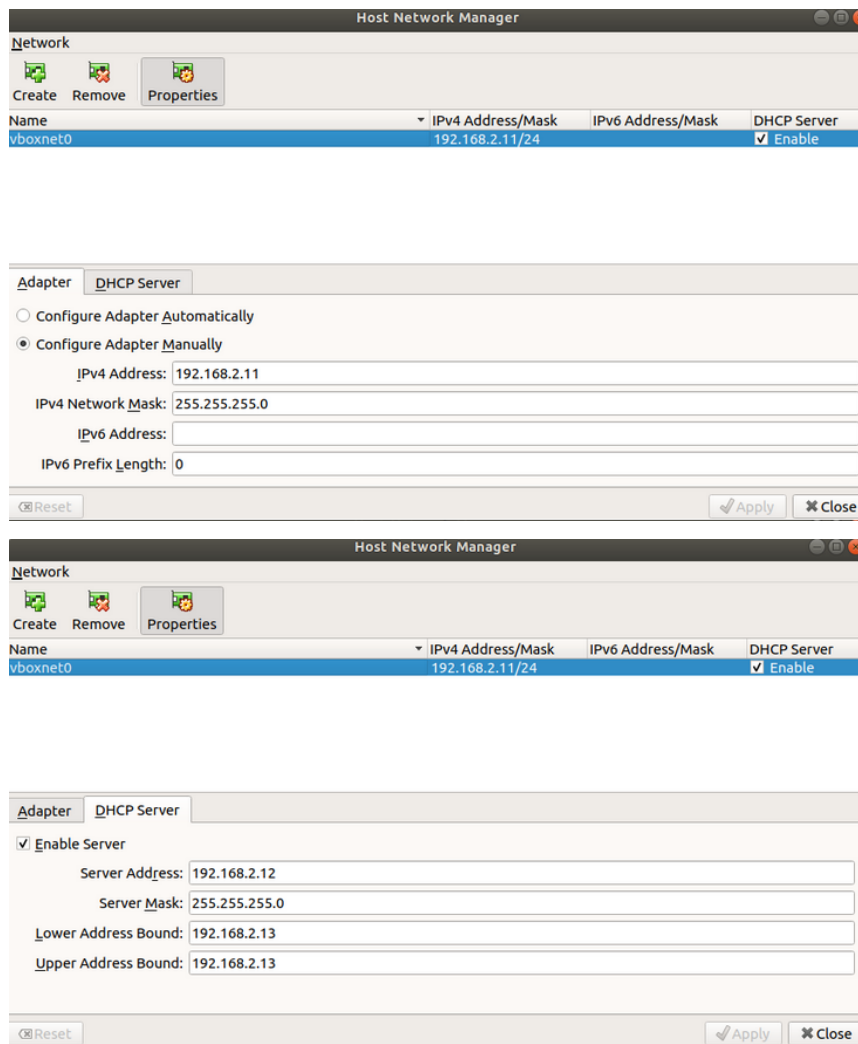


11. Once the adapter is created, it should display **vboxnet0**. Enable the DHCP Server by clicking the **Enable** checkbox under **DHCP Server** in the top right corner. Next, click on **Properties**.
12. Choose the **Configure Adapter Manually** option and adjust the fields as follows:

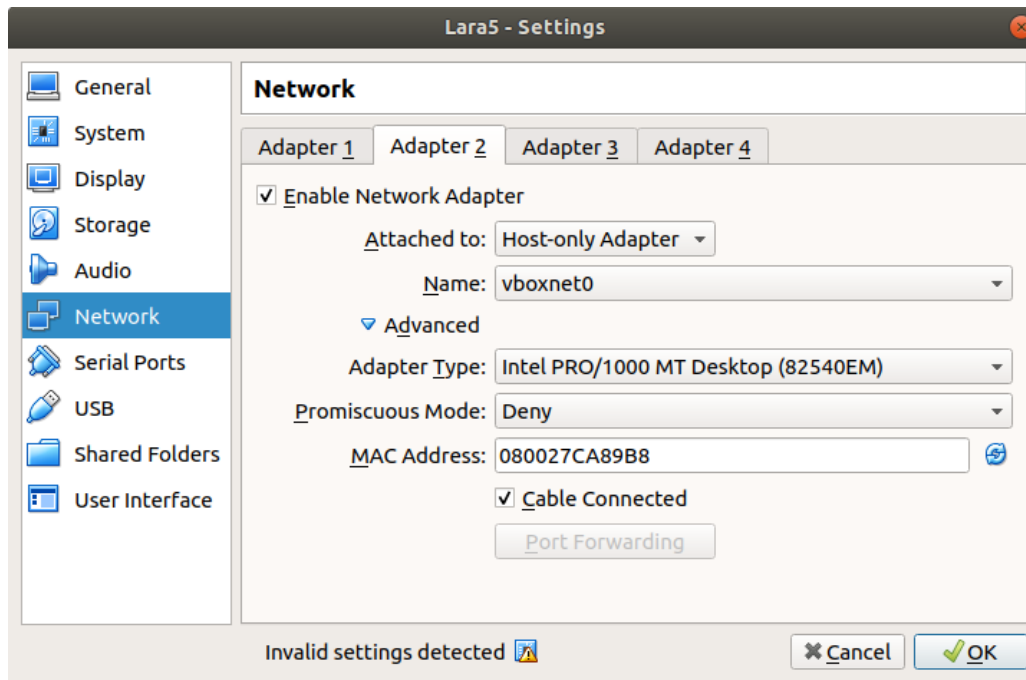
- **IPv4 Address:** 192.168.2.11
- **IPv4 Network Mask:** 255.255.255.0

In the DHCP Server settings, configure the following:

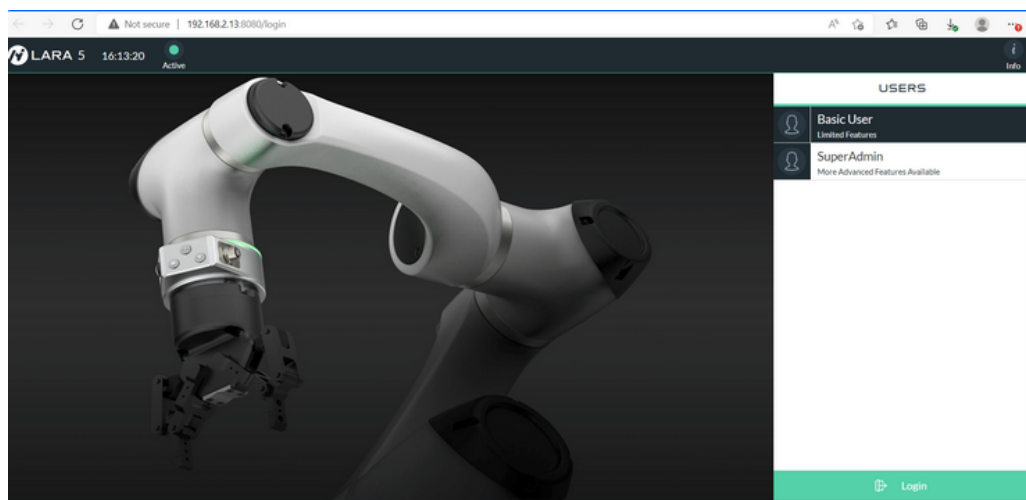
- **Server Address:** 192.168.2.12
- **Server Mask:** 255.255.255.0
- **Lower Address Bound:** 192.168.2.13
- **Upper Address Bound:** 192.168.2.13



- Click on **Settings** → **Network** → **Adapter 1**. If **Enable Network Adapter** is checked, uncheck it first. Then, click on **Adapter 2** and check **Enable Network Adapter**. From the drop-down menu of **Attached to**, select **Host-only Adapter**. In the **Name** field, select **VirtualBox Host-Only Ethernet Adapter**. The configuration should appear as follows:



14. The configuration of **Lara5** is complete. Start **Ubuntu Lara5** and run it for 30 seconds. To access the GUI, open Chrome and enter <http://192.168.2.13:8080/login>. If all processes were successful, this address should load the GUI interface.



API INSTALLATION

This section provides installation instructions for both Debian-based and client-only setups. Choose the appropriate method based on your operating system and deployment environment.

4.1 Quick Installation (Recommended)

The **NeuraPy interface** is available as a Python wheel package for quick and easy client-side installation.

This method is recommended for users for controlling the robot from an external machine.

4.1.1 Prerequisites

- Python 3.8 or higher
- pip (Python package installer)
- Internet connection (for downloading dependencies)

4.1.2 Python Installation

 Note:

You must have **Python 3.8 or higher** installed before installing the NeuraPy wheel package.

For Windows:

1. Download the Python 3.8 executable (.exe) from the [official Python website](#).
2. Run the installer and check “**Add Python to PATH**”.
3. Click **Install Now** and complete the setup.



4. Close the setup window when done.

For Ubuntu:

1. Use your package manager to install Python 3.8+ (if not already installed):

```
sudo apt update
sudo apt install python3.8 python3-pip
```

4.1.3 Installation Steps

1. **Download the NeuraPy wheel package** from the releases section or the link provided.
2. **Install the package** using pip:

```
pip3 install neurapy-<version>-py3-none-any.whl
```

For example:

```
pip3 install neurapy-5.0.1-py3-none-any.whl
```

3. **Verify the installation** by importing the API:

```
from neurapy.robot import Robot
print("NeuraPy installation successful!")
```

4.1.4 Dependencies

The wheel package will automatically install the following dependencies:

- **prettytable** — For displaying available API methods
- **pywin32** — Required only on Windows systems (for signal handling)

Note:

This installation provides the **client-side neurapy interface**, which connects to a robot running the NeuraPy server. The API is identical to the Debian-based installation, and no robot-side deployment is required on the client machine.

4.2 Installation Instructions for Ubuntu (Debian-Based)

Note:

The Debian-based installation of **NeuraPy** is officially supported only on **Ubuntu 18.04** and **Ubuntu 20.04**. These installation files are **provided only upon request** and are not publicly available by default.

1. Check your Ubuntu version:

```
lsb_release -a
```

2. Run the NeuraPy Deployment Script:

After receiving the appropriate deployment package, navigate to the deployment folder and run the script with your robot name:

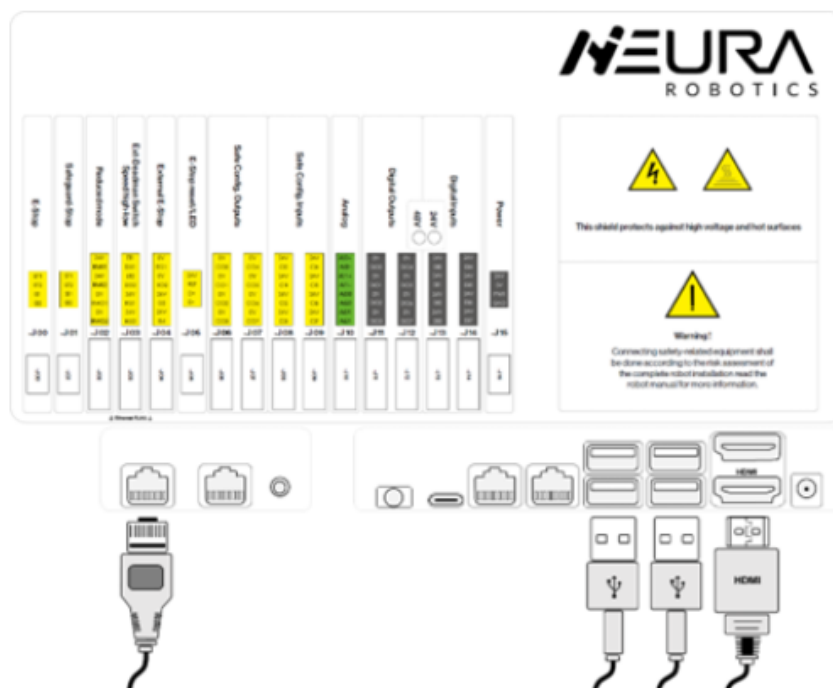
```
cd DEP_Deployment/neurapy_ubuntu18 # or neurapy_ubuntu20
. deploy_neurapy.sh LARA5           # Replace 'LARA5' with your robot's
↪name
```

4.3 Network Configuration

To control the robot from an external machine via Ethernet, configure the network as follows:

1. Connect the Ethernet cable between the robot control box and your external PC.

- Open the control box using the provided key.
- Locate the **Ethernet RJ45** port on the lower left side of the control box.
- Connect one end of the cable to this port and the other to your PC.

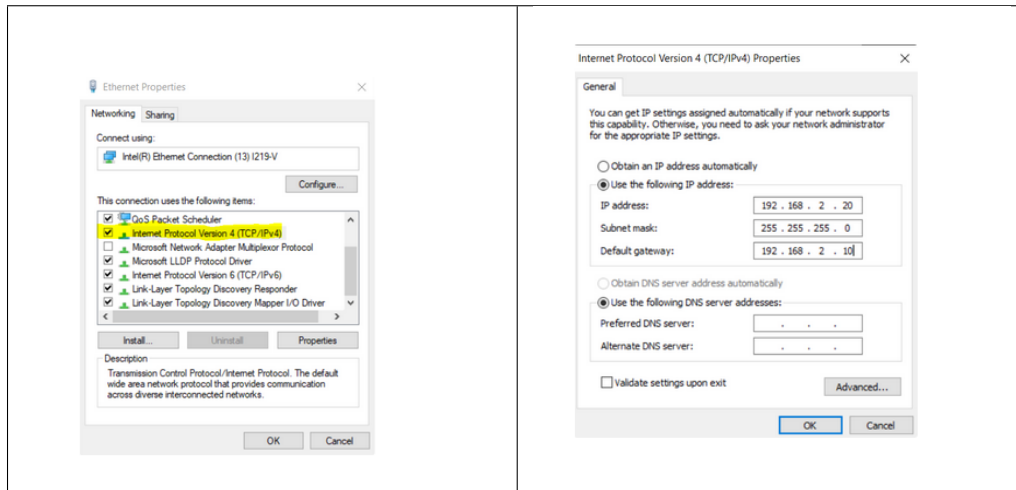
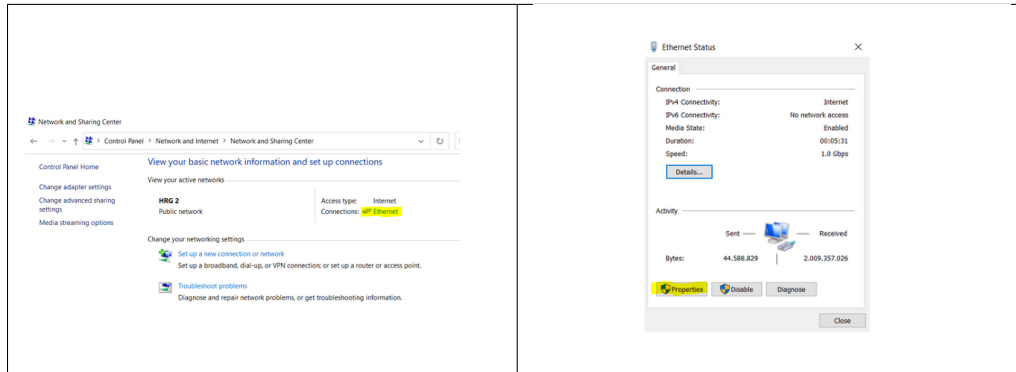


2. Set the static IP address on your PC.

Windows:

Navigate to:

Control Panel → Network and Sharing Center → Change adapter settings → Ethernet → Properties → Internet Protocol Version 4 (TCP/IPv4)



Ubuntu:

Use your system settings to manually assign a static IP address (e.g. *192.168.2.99*) in the same subnet as the robot (default: *192.168.2.13*).

The screenshot shows the 'netplan-enp2s0' configuration window with tabs for Details, Identity, IPv4, IPv6, and Security. The IPv4 tab is active, showing the 'IPv4 Method' set to 'Manual'. Under 'Addresses', a table lists the IP address '192.168.2.20', netmask '255.255.255.0', and gateway '192.168.2.10'. The 'DNS' section has an 'Automatic' toggle set to 'ON'. The 'Routes' section also has an 'Automatic' toggle set to 'ON'.

Address	Netmask	Gateway
192.168.2.20	255.255.255.0	192.168.2.10

4.3.1 Configuring Client IP Address

If your robot uses a **non-default IP address** (i.e., other than *192.168.2.13*), you can pass it directly when creating the *Robot* instance:

```
from neurapy.robot import Robot

r = Robot(socket_address="192.168.2.99")
```

This eliminates the need to set any environment variables manually.

4.4 Installing an IDE: Visual Studio Code (VSCode)

This guide walks you through installing **VSCode** on Windows, macOS, or Linux.

1. Check Requirements

- Internet connection
- Enough storage space

2. Download VSCode

- Visit <https://code.visualstudio.com/>
- Click the download button (auto-detected for your OS)

3. Install VSCode

Windows:

- Run the *.exe* file

- Follow the wizard (use default options unless needed)
- Click **Install**, then **Finish**

Linux (Ubuntu):

```
sudo dpkg -i <file-name>.deb  
sudo apt-get install -f
```

Replace *<file-name>* with the actual *.deb* filename.

4. Launch VSCode

Open VSCode from the app menu or type *code* in the terminal.

 Tip:

For the best experience, install the **Python extension** in VSCode.

You're all set to start developing with NeuraPy!

API REFERENCE

5.1 Robot Class

class neurapy.robot.Robot(*args, **kwargs)

Bases: object

A Python class to access and control the robot's software functionalities. It serves as an interface for users to interact with the robot's capabilities.

Parameters

socket_address – (Optional[str]) The IP address of the robot, default is 192.168.2.13.

Sample Usage:

Example 1 (Default communication address):

```
from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.kURL)
```

Example 2 (Custom communication IP):

```
from neurapy.robot import Robot

r = Robot(socket_address='192.168.2.20')
print(r.robot_name)
print(r.dof)
print(r.kURL)
```

activate_ethercat_interface()

Activate the Ethercat/el6695 interface.

Returns

True if the EtherCAT interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.activate_ethercat_interface()
```

activate_ethernetIP_interface(*args)

Activate the Ethernet/IP interface.

Parameters

path (*str*) – The path of the YAML file to activate the Ethernet/IP interface.

Returns

True if the Ethernet/IP interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.activate_ethernetIP_interface('/path/to/yaml')
```

activate_modbus_interface(*args)

Activate the Modbus interface.

Parameters

path (*str*) – The path of the YAML file to activate the Modbus interface.

Returns

True if the Modbus interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.activate_modbus_interface('/path/to/yaml')
```

activate_opcua_interface(*args)

Activate the OPC UA interface.

Parameters

path (*str*) – The path of the YAML file to activate the opcua interface.

Returns

True if the OPC UA interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.activate_opcua_interface('/path/to/yaml')
```

activate_profinet_interface()

Activate the Profinet interface.

Returns

True if the Profinet interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.activate_profinet_interface()
```

activate_ros_interface(*args)

Activate the ROS interface.

Parameters

mode (*str*) – The mode to activate the ROS interface. Options:

- 'position': Activate the ROS interface in position mode.
- 'torque': Activate the ROS interface in torque mode.

Returns

True if the ROS interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.activate_ros_interface('position')
```

activate_servo_interface(*args)

Activate the Servo interface.

Parameters

mode (*str*) – The mode to activate the Servo interface. Options:

- 'position': Activate the Servo interface in position mode.
- 'torque': Activate the Servo interface in torque mode.

Returns

True if the Servo interface is activated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.activate_servo_interface('position')
```

check_circular_feasibility(*args, **kwargs)

Checks if circular motion over all poses is feasible.

This method takes the following arguments/keyword arguments:

Parameters

- **target_pose** (*list*) – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **target_joint** (*list*) – List of joint configurations, needs current_joint_angles if not provided. (type: Joint configuration - [], float, units: Joint values at each position, required: No)
- **motion_id** (*int*) – id provided for the motion. (units: -, default: 0, required: No)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)

- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 100.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.4, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 0.4, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 100.0, required: No)
- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration, required if target_joint not provided. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. Required when weaving is enabled.
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)

- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.

- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

list of feasible (*bool*) and joint configuration (*List(float)*)

Return type

list

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "static",
    "blend_radius": 0.01,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
    ]
}
```

(continues on next page)

(continued from previous page)

```

        0.5190337951593321,
        -0.1969996948428492,
        0.23267853691809767,
        3.1414194107055664,
        -0.00017726201622281224,
        -0.48750609159469604
    ]
],
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"weaving_parameters": [
    {
        "start_target_pose_number": 1,
        "end_target_pose_number": 3,
        "frequency": 0.01,
        "pattern": "sine",
        "amplitude_left": 0.006,
        "amplitude_right": 0.006,
        "dwell_time_left": 0,
        "dwell_time_right": 0,
        "azimuth": 0.0,
        "elevation": 0.0
    }
],
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}

```

(continues on next page)

(continued from previous page)

```
feasible, joint_angles = r.check_circular_feasibility(**circular_
↳ property)
```

Example 2:

```
from neurapy.robot import Robot

r = Robot()
feasible, joint_angles = r.check_circular_feasibility(["P1", "P2",
↳ "P3"])
```

check_composite_feasibility(*args, **kwargs)

Checks if a combination of linear and circular motions is feasible.

This method takes the following arguments/keyword arguments:

Parameters

- **commands** (*list*) – List of linear and circular command combinations. Each element is a dictionary with a single key “linear” or “circular”, and the value is a dictionary containing the command parameters.

Each command dictionary structure:

- linear command: Dictionary with key “linear” and value containing:

- * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
- * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- * **velocity** (*float*): Translation Velocity, required when use_children_velocities set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
- * **rotation_velocity** (*float*): Rotational Velocity, required when use_children_velocities set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)

- circular command: Dictionary with key “circular” and value containing:

- * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
- * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- * **velocity** (*float*): Translation Velocity, required when use_children_velocities set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
- * **rotation_velocity** (*float*): Rotational Velocity, required when use_children_velocities set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)

- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)

- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec², default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec³, default: 500.0, required: No)
- **use_children_velocities** (*bool*) – Use individual speed and rotation speeds for each child motion, as set in commands parameter. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'static', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)

- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.

- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

list of feasible (*bool*) and joint configuration (*List(float)*)

Return type

list

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "dynamic",
    "blend_radius": 0.01,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [
        {
            "linear": {
                "blend_radius": 0.005,
                "target_pose": [
                    [
                        -0.000259845199876027,
                        -0.5211437049195536,
                        0.4429382717719519,
                        3.14123272895813,
                        -0.0007908568368293345,
                        -1.570908784866333
                    ],
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ]
                ]
            }
        }
    ],
}
```

(continues on next page)

(continued from previous page)

```

        {
            "circular": {
                "target_pose": [
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ],
                    [
                        -0.16540090985202305,
                        -0.3983552679378624,
                        0.44267608017426174,
                        3.1407113075256348,
                        -0.00036628879024647176,
                        -1.5714884996414185
                    ],
                    [
                        -0.33446498807559716,
                        -0.3989652352814891,
                        0.4421152856242009,
                        3.1402060985565186,
                        0.00030071483342908323,
                        -1.572899580001831
                    ]
                ]
            }
        },
        "motion_id": 234,
        "current_joint_angles": r.get_current_joint_angles(),
    }
    feasible, joint_angles = r.check_composite_
    ↪ feasibility(**composite_motion_property)

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
composite_motion_property = {
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "static",
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "use_children_velocities": True,
    "commands": [
        {
            "linear": {
                "target_pose": [

```

(continues on next page)

(continued from previous page)

```

        0.7006,
        0.0001,
        0.6875,
        3.1416,
        0,
        3.1416
    ],
    [
        0.7006,
        0.0001,
        0.6875,
        3.1416,
        0,
        3.1416
    ],
    [
        0.7509102896938852,
        0.27460943971123764,
        0.6864972996055627,
        3.141592653589793,
        -7.71435533383267e-06,
        -3.1415926535897882
    ],
    [
        0.7747260445464177,
        0.15721988912449852,
        0.8931631506500376,
        -3.1415926535897745,
        -7.714355718231237e-06,
        -3.1415926535897354
    ],
    [
        0.7006,
        0.0001,
        0.6875,
        3.1416,
        0,
        3.1416
    ]
],
"velocity": 0.1,
"rotation_velocity": 0.174533
}
},
{
    "circular": {
        "target_pose": [
            0.7006,
            0.0001,
            0.6875,
            3.1416,
            0,
            3.1416
        ],
    },
    [

```

(continues on next page)

(continued from previous page)

```

0.6999986609380567,
-0.25149522267751856,
0.6864972999318248,
-3.141592653589793,
-7.71435539238761e-06,
3.1415926535897905
],
[
0.7747260445464177,
0.15721988912449852,
0.8931631506500376,
-3.1415926535897745,
-7.714355718231237e-06,
-3.1415926535897354
]
],
"velocity": 0.4,
"rotation_velocity": 0.436332
}
},
"weaving": False
}
feasible, joint_angles = r.check_composite_
↪ feasibility(**composite_motion_property)

```

check_joint_feasibility(*args, **kwargs)

This method takes the following arguments/keyword arguments:

Parameters

- **target_joint** (*list*) – List of joint configurations. (type: List of Joint Values - float, units: radians, required: Yes)
- **target_pose** – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: No)

Alternative: List of Strings (Type: [String], names of existing points). If not provided, the poses will be automatically generated from the current_joint_angles. :type target_pose: list :param speed: Angular Speed. (units: % of maximum angular speed, scale: 0 to 100, default: 25%, required: No) :type speed: float :param acceleration: Angular Acceleration. (units: % of maximum angular acceleration, scale: 0 to 100, default: 25%, required: No) :type acceleration: float :param jerk: Angular Jerk. (units: % of maximum angular jerk, default: 100.0, required: No) :type jerk: float :param current_joint_angles: Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No) :type current_joint_angles: list

Parameters

- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **motion_id** (*int*) – identifier to the stored plan. (if not provided a random will be assigned, which will be returned after the function execution)

Returns

list of feasible and last joint configuration

Return type

list

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

Example 1:

```
from neurapy.robot import Robot
r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_blending": True,
    "target_joint": [
        [
            2.5995838308821924,
            0.24962416292345468,
            -1.8654403327490414,
            0.04503286318691005,
            -1.1740563715454926,
            0.10337461241185522
        ],
        [
            2.1372059994827075,
            0.24939733788589463,
            -1.8651270179353125,
            0.044771940725327274,
            -1.173860821592129,
            0.10315646291502645
        ],
        [
            1.9180047887810003,
            -0.24855170101601043,
            -1.3680228668892351,
            0.12404421791100637,
            -1.1914147150222498,
            -0.13255713717112075
        ]
    ],
    "motion_id": 234,
    "current_joint_angles": r.get_current_joint_angles(),
}
motion_id = r.check_joint_feasibility(**joint_property)
execute = r.executor()
```

check_joint_relative_feasibility(*args, **kwargs)

To move the robot relative to the current position by the given cartesian offset. The motion is executed as joint motion.

Parameters

- **speed** (*float*) – Angular Speed. (units: % of maximum angular speed, default: 25, required: No)
- **acceleration** (*float*) – Angular Acceleration. (units: % of maximum angular acceleration, default: 25, required: No)

- **jerk** (*float*) – Angular Jerk. (units: % of maximum angular jerk, default: 25, required: No)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **offset_frame** (*str*) – Frame for applying relative offset. (units: N/A, default: "Base", required: No)
 - 'Base' : Applies relative offset to the base frame
 - 'Tool' : Applies relative offset to the tool frame
- **cartesian_offset** (*list*) – The cartesian offset that is applied. (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m - [R,P,Y] in radian, default: [0.0,0.0,0.0,0.0,0.0,0.0], required: Yes)
- **target_joint** (*list*) – The joint configuration at which the relative motion should start from (type: list - [a1, a2, a3, a4, a5, a6] for lara and [a1, a2, a3, a4, a5, a6, a7] for maira, units: axis values in radian, default: current joint angles, required: No) If the target_joint is used, the target_pose also needs to be configured, else the current joint angles and current pose are used
- **target_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No) If the target_pose is used, the target_joint also needs to be configured, else the current joint angle and current pose are used

Returns

list of feasible (bool) and joint configuration (List(float))

Return type

list

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed" : 50.0,
    "acceleration" : 50,
    "jerk": 50,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi / 4],
    "current_joint_angles": r.get_current_joint_angles()
}
```

(continues on next page)

(continued from previous page)

```
feasible, joint_angles = r.check_joint_relative_feasibility(**joint_
→property)
```

check_linear_feasibility(*args, **kwargs)

To plan a move linear path across the given poses.

This method takes the following arguments/keyword arguments:

Parameters

- **target_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 500.0, required: No)
- **enable_blending** – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. (required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.

- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)

Returns

list of feasible (bool) and joint configuration (List(float))

Return type

list

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "enable_blending": True,
    "blending_mode": "static",
    "blend_radius": 0.01,
    "target_pose": [
        [
            0.3287228886,
            -0.1903355329,
            0.4220780352,
            0.08535207028439847,
            -2.797181496822229,
            2.4713321627410485
        ],
        [
            0.2093363791501374,
```

(continues on next page)

(continued from previous page)

```

        -0.31711250784165884,
        0.422149168855134,
        -3.0565555095672607,
        -0.3447442352771759,
        -1.1323236227035522
    ],
    [
        0.2090521916195534,
        -0.5246753336643587,
        0.4218773613553828,
        -3.0569007396698,
        -0.3448921740055084,
        -1.1323626041412354
    ],
    [
        0.3287228886,
        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ]
],
"motion_id":234,
"current_joint_angles": r.get_current_joint_angles(),
"weaving":False,
"weaving_parameters": [
    {
        "start_target_pose_number": 1,
        "end_target_pose_number": 4,
        "frequency": 0.01,
        "pattern": "sine",
        "amplitude_left": 0.006,
        "amplitude_right": 0.006,
        "dwell_time_left": 0,
        "dwell_time_right": 0,
        "azimuth": 0.0,
        "elevation": 0.0
    }
]
}
feasible, joint_angles = r.check_linear_feasibility(**linear_property)

```

check_linear_relative_feasibility(*args, **kwargs)

To move the robot relative to the current position by the given cartesian offset. The motion is executed as Cartesian motion.

Parameters

- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)

- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec², default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec³, default: 500.0, required: No)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)
- **cartesian_offset** (*list*) – (type: offset list - [X,Y,Z,R,P,Y], units: [X,Y,Z] offset in m [R,P,Y] offset in radian, default: [0.0,0.0,0.0,0.0,0.0], required: Yes)
- **offset_frame** (*str*) – Frame for applying relative offset. (units: N/A, default: "Base", required: No)
 - 'Base' : Applies relative offset to the base frame
 - 'Tool' : Applies relative offset to the tool frame
- **target_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)

Returns

list of feasible (bool) and joint configuration (List(float))

Return type

list

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()

linear_property = {
    "speed" : 4.5,
    "acceleration" : 10.0,
    "jerk": 1000.0,
    "rotation_speed" : 1.5,
    "rotation_acceleration" : 10.0,
    "rotation_jerk" : 1000.0,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi/4],
    "current_joint_angles": r.get_current_joint_angles(),
}

r.check_linear_relative_feasibility(**linear_property)
```

check_move_spline_feasibility(*args, **kwargs)

To check whether the motion is feasible with the inputs provided.

This method takes the following arguments/keyword arguments:

Parameters

- **motion_id** (*int*) – identifier for motion (units: -, default: 5, required: No)
- **interpolation_type** (*str*) – Type of cubic spline interpolation. Available types: akima and bspline.(required: yes)
- **file_name** (*str*) – File name with full path. The input file should be a JSON file with two KV pairs i.e. “time_stamps” : ... and “joint_angles” : ... Please use the same naming convention. The value should be a list for time_stamps and for joint_angles a list of lists(size: dof). (required: Yes)
- **is_approximated** (*bool*) – (Only for BSpline motion) True if the spline path should be approximated, False if the spline should be interpolated i.e. should exactly pass through the recorded points. Parameter only included in B-Spline. (units: N/A, default: True, required: No)

Raises

- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
move_spline_property = {
    "interpolation_type"
    "file_name": "../test_paths/test_path1.json",
}
success, joint_angles_finish = r.check_move_spline_feasibility(**move_
↪spline_property)
print("Success : " , success)
```

check_recorded_path_feasibility(*args, **kwargs)

To check if given motion is feasible.

This method takes the following arguments/keyword arguments:

Parameters

- **file_name** (*str*) – File name, if the file is saved from GUI or full path of the custom recorded path file. (required: Yes)
- **is_const_velocity** (*bool*) – True if constant velocity is needed during the motion. (required: No)
- **speed** (*float*) – Linear Speed. (units: m/sec, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **motion_id** (*int*) – identifier to store the plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (type: Bool - True/False, default_value: False)

Returns

motion_id (equal to motion_id if provided in inputs, else an identifier function has generated to store the plan)

Return type

int

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "file_name": "./test_paths/test_path1.txt",
    "speed": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
feasible, joint_angles = r.check_recorded_path_feasibility(**recorded_
↪path_property)

r.stop()
```

compute_forward_kinematics(joint_angles, target_frame='tool', representation='rpy', tool_params=None)

Compute forward kinematics (end-effector's Cartesian pose) for the given joint configuration.

Parameters

- **joint_angles** (*list*) – list of joint angles in radians.
- **target_frame** (*str, optional*) – the node for which the pose is requested. Default: 'tool'. Options: ['flange', 'wrist', 'elbow', 'link6', 'link5', ...]
- **Default** (*representation: string defining the return type of the rotational part of the pose.*) – 'rpy'. Options: ['rpy', 'quaternion']

Raises

ValueError – If the size of the argument joint_angles is not equal to the number of joints on the robot or if the representation is not valid

Returns

list representing the TCP pose in XYTRPY format.

Return type

list

Sample Usage

```
from neurapy.robot import Robot

r = Robot()
target_joint_angles = [0.2] * r.dof
tcp_pose = r.compute_forward_kinematics(joint_angles=target_joint_angles)
```

compute_inverse_kinematics(*target_pose*, *reference_joint*, *representation*='rpy',
tool_params=None)

Compute inverse kinematics(joint configuration) for the given end-effector's cartesian pose.

Parameters

- **target_pose** (*list*) – list representing the target end_effector pose in XYZRPY format if representation == “rpy”. For the representation “quaternion” the format is XYZQWQ1Q2Q3
- **reference_joint** (*list*) – list of reference joint angles (of the robot configuration) in radians.
- **representation** (*string*, *optional*) – string defining the input type of the rotational part of the target_pose. Default: ‘rpy’. Options: [‘rpy’, ‘quaternion’]

Raises

- **IKNotFound** – If IK solution could not be found.
- **ValueError** – If the size of the argument joint_angles is not equal to the number of joints on the robot or if the representation is not valid

Returns

list of joint angles in radians that achieve the target end-effector pose

Return type

list

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
target_end_effector_pose = [0.4983978832107115, -0.03249999999999999, 0.
↪ 3344977006474223, -3.0510184133354567e-06, 3.141585287645838, 2.
↪ 7488999999887627]
reference_joint_angles = [-1.3326468647873386, 0.0, 1.570800183972526, 0.
↪ 0, 1.5708001839725365, 0.0]
joint_angle_solution = r.compute_inverse_kinematics(target_pose=target_
↪ end_effector_pose, reference_joint=reference_joint_angles)
```

connect_external_device(*file_path*: str, *ignore_no_connection*: bool = False) → bool

Connects to an external device.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **ignore_no_connection** (*bool*, *optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

True if connection is successful, False otherwise.

Return type

bool

Raises

DriverError – If the connection fails and ignore_no_connection is False.

Sample Usage


```
from neurapy.robot import Robot
r = Robot()
is_connected = r.connect_external_device("external_device.json")
```

convert_euler_to_quaternion_pose(*cartesian_pose*)

Convert a pose with Euler angles to quaternions.

If the input pose is already in quaternion format (length 7), it is returned unchanged.

Parameters

cartesian_pose (*list*) – The pose to be converted to quaternion format ([X, Y, Z, R, P, Y], required: Yes).

Returns

The pose in quaternion format ([X, Y, Z, W, EX, EY, EZ]).

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
euler_pose = [1.0, 2.0, 3.0, 0.1, 0.2, 0.3] # [X, Y, Z, R, P, Y]
quaternion_pose = r.convert_euler_to_quaternion_pose(euler_pose)
print(quaternion_pose) # Output: [X, Y, Z, W, EX, EY, EZ] with
↳ quaternion values.
```

convert_quaternion_to_euler_pose(*cartesian_pose*)

Convert a pose with quaternion representation to a pose with Euler angles.

If the input pose is already in Euler format (length 6), it is returned unchanged.

Parameters

cartesian_pose (*list*) – The pose to be converted to Euler angles ([X, Y, Z, W, EX, EY, EZ], required: Yes).

Returns

The pose in Euler angle format ([X, Y, Z, R, P, Y]).

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
quaternion_pose = [1.0, 2.0, 3.0, 0.707, 0.0, 0.707, 0.0] # [X, Y, Z,
↳ W, EX, EY, EZ]
euler_pose = r.convert_quaternion_to_euler_pose(quaternion_pose)
print(euler_pose) # Output: [X, Y, Z, R, P, Y] with Euler angle
↳ values.
```

copy_pose(*pose_name, new_pose_name, new_frame_name*)

Replicate a pose(Point/Frame) in a new parent frame.

Parameters

- **pose_name** (*str*) – Name of the pose to be replicated.
- **new_pose_name** (*str*) – Name of the new pose to be created.
- **new_frame_name** (*str*) – Name of the new frame to be created.

Returns

True if the pose was replicated successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If user frame deletion fails with the given input data.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.copy_pose(pose_name="point_1", new_pose_name="new_point", new_frame_
↳ name="new_frame")
```

create_external_device(*file_path: str, ignore_no_connection: bool = False*) → ExternalDeviceApp

Factory method to create an ExternalDeviceApp object, which can be used to interact with external devices. Check the “External device tab in settings page to get an overview of the supported devices including their functions and parameters.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **ignore_no_connection** (*bool, optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Raises

DriverError – If the creation of the external device app is unsuccessful.

Returns

An instance of ExternalDeviceApp, allowing various operations to be executed with the external device.

Return type

ExternalDeviceApp

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
# Using full path
external_device = r.create_external_device("~/data/process/external_
↳ device.json", ignore_no_connection=False)
# Using just filename
external_device = r.create_external_device("external_device.json",
↳ ignore_no_connection=False)
```

create_external_source_app(*file_path: str, ignore_no_connection: bool = False*) → ExternalDeviceApp

Warning:

This function is deprecated and will be removed in future versions. Use the following function instead:

- `create_external_device()`

Factory method to create an ExternalDeviceApp object, which can be used to interact with external devices. Check the “External source tab in settings page to get an overview of the supported devices including their functions and parameters.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external source. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **ignore_no_connection** (*bool, optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Raises

DriverError – If the creation of the external source app is unsuccessful.

Returns

An instance of ExternalDeviceApp, allowing various operations to be executed with the external source.

Return type

ExternalDeviceApp

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
# Using full path
external_device = r.create_external_source_app("~/data/process/
↪external_device.json", ignore_no_connection=False)
# Using just filename
external_device = r.create_external_source_app("external_device.json",
↪ ignore_no_connection=False)
```

create_point(*name*, *reference_frame_name*='World', *target_end_effector_pose*=None, *target_joint_angles*=None, *tool_name*=None)

Store a point in the database.

Parameters

- **name** (*str*) – Name of the point to be saved (required: Yes).
- **reference_frame_name** – Name of the reference_frame (default: “World”).
- **target_end_effector_pose** (*list*) – A list containing Cartesian points in XYZRPY format (required only if target_joint_angles is not provided).
- **target_joint_angles** (*list*) – A list containing joint angles for the point (required only if target_end_effector_pose is not provided).
- **tool_name** – Name of the tool for the kinematics needed to save a valid input point. (default: Active tool in GUI).

Returns

True if the point was added successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If point creation fails with the given input data.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.create_point(name='new_point', target_end_effector_pose=[0.36, 0.36,
↪ 0.1, 3.14, 0, 0, 3.14])
```

create_reference_frame(*name*, *pose*, *parent_frame*='Base', *description*='')

Store a user-defined reference frame in the database.

Parameters

- **name** (*str*) – Name of the reference frame to be saved (required: Yes).
- **pose** (*list*) – A list containing the Cartesian coordinates and orientation of the reference frame in euler or quaternion format (required).
- **parent_frame** (*str*) – Name of the parent reference frame (default: “Base”).
- **description** (*str*) – Description of the reference frame (default: “”).

Returns

True if the user frame was added successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If user frame creation fails with the given input data.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.create_reference_frame(name='new_frame', pose=[0.36, 0.36, 0.1, 3.
↪ 14, 0, 0, 3.14])
```

create_tool(*tool_data*)

Create a new tool in the robot’s database using the provided tool data.

Parameters

tool_data (*dict*) – A dictionary containing the data for the new tool (required: Yes).

Returns

True if the tool was added successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If tool creation fails with the given input data.

Tool Description

Key/Value	De- fault Value	Description	Required or Not
_controlOA	False	True, if the tool is controlled by controlbox analog outputs	Not Required
_controlOD	False	True, if the tool is controlled by controlbox digital outputs	Not Required
_toolOA	False	True, if the tool is controlled by analog outputs of port present on robot tool flange	Not Required
_toolOD	False	True, if the tool is controlled by digital outputs of port present on robot tool flange	Not Required
autoM	0	Mass of the tool	Required
autoMeasureX	0	tool center of gravity (COG) in X direction, measured from robot's flange frame	Not Required
autoMeasureY	0	tool center of gravity (COG) in Y direction, measured from robot's flange frame	Not Required
autoMeasureZ	0	tool center of gravity (COG) in Z direction, measured from robot's flange frame	Not Required
closeInput	0	percentage of gripper width for close action	Not Required
cmdID	16	Input specific for certain kind of grippers	Not Required
force	0	Gripper closing force	Not Required
gripper		Gripper type	Not Required
gripper-type	Standard Gripper	Type of gripper Modbus/Standard	Not Required
inertiaXX	0	Ixx of tool	Not Required
inertiaXY	0	Ixy of tool	Not Required
inertiaXZ	0	Ixz of tool	Not Required
inertiaYY	0	Iyy of tool	Not Required
inertiaYZ	0	Iyz of tool	Not Required
inertiaZZ	0	Izz of tool	Not Required
name	N/A	Name of the tool	Required
offCOA	[0, 0, 0, 0, 0, 0]		Not Required
offCOD1	0	if the tool is controlled via control box digital outputs, offCOD1 is the pin mapped to turn off (close) the tool	Not Required
offCOD2	0		Not Required
offTOA	[0, 0]		Not Required
offTOD	0	if the tool is controlled via tool digital outputs, offTOD is the pin mapped to turn off (close) the tool	Not Required
offsetA	0	TCP roll offset	Not Required
offsetB	0	TCP pitch offset	Not Required
offsetC	0	TCP yaw offset	Not Required
offsetX	0	TCP offset in X	Not Required
offsetY	0	TCP offset in Y	Not Required
offsetZ	0	TCP offset in Z	Not Required
onCOA	[0, 0, 0, 0, 0, 0]		Not Required

continues on next page

Table 1 – continued from previous page

Key/Value	De- fault Value	Description	Required or Not
onCOD1	0	if the tool is controlled via control box digital outputs, onCOD1 is the pin mapped to turn on (open) the tool	Not Required
onCOD2	0		Not Required
onTOA	[0, 0]		Not Required
onTOD	0	if the tool is controlled via tool digital outputs, onTOD is the pin mapped to turn on (open) the tool	Not Required
openInput	0		Not Required
portID	0		Not Required
protocol	0		Not Required
robot_type	Tool		Required
slaveID	0		Not Required
speed	0		Not Required

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
tool_data = { '_control0A': False,
               '_control0D': False,
               '_tool0A': False,
               '_tool0D': False,
               'autoM': 0,
               'autoMeasureX': 0,
               'autoMeasureY': 0,
               'autoMeasureZ': 0,
               'closeInput': 0,
               'cmdID': 16,
               'description': 'Tool Description',
               'force': 0,
               'gripper': '',
               'grippertype': 'Standard Gripper',
               'inertiaXX': 0,
               'inertiaXY': 0,
               'inertiaXZ': 0,
               'inertiaYY': 0,
               'inertiaYZ': 0,
               'inertiaZZ': 0,
               'name': 'somerandomtool',
               'offCOA': [0, 0, 0, 0, 0, 0, 0, 0, 0],
               'offCOD1': 0,
               'offCOD2': 0,
               'offTOA': [0, 0],
               'offTOD': 0,
               'offsetA': 0,
               'offsetB': 0,
               'offsetC': 0,
               'offsetX': 0,
               'offsetY': 0,
               'offsetZ': 0,
               'onCOA': [0, 0, 0, 0, 0, 0, 0, 0, 0],

```

(continues on next page)

(continued from previous page)

```

'onCOD1': 0,
'onCOD2': 0,
'onTOA': [0, 0],
'onTOD': 0,
'openInput': 0,
'portID': '',
'protocol': 0,
'robot_type': 'Tool',
'slaveID': 0,
'speed': 0}

tools_data = r.create_tool(tool_data)

```

deactivate_ethercat_interface()

Deactivate the Ethercat interface.

Returns

True if the Ethercat interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.deactivate_ethercat_interface()

```

deactivate_ethernetIP_interface()

Deactivate the Ethernet/IP interface.

Returns

True if the Ethernet/IP interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.deactivate_ethernetIP_interface()

```

deactivate_modbus_interface()

Deactivate the Modbus interface.

Returns

True if the Modbus interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.deactivate_modbus_interface()

```

deactivate_opcua_interface()

Deactivate the OPCUA interface.

Returns

True if the OPCUA interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_opcua_interface()
```

deactivate_profinet_interface()

Deactivate the Profinet interface.

Returns

True if the Profinet interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_profinet_interface()
```

deactivate_ros_interface()

Deactivate the ROS interface.

Returns

True if the ROS interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_ros_interface()
```

deactivate_servo_interface()

Deactivate the Servo interface.

Returns

True if the Servo interface is deactivated successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.deactivate_servo_interface()
```

delete_point(*name*)

Delete a point from the database.

Parameters

name (*str*) – The name of the point to delete.

Returns

True if the point was deleted successfully.

Return type

bool

delete_points_in_frame(*frame_name*)

Delete all points in a user-defined reference frame in the database.

Parameters**frame_name** (*str*) – Name of the reference frame to be deleted.**Returns**

True if the points were deleted successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If user frame deletion fails with the given input data.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.delete_points_in_frame(frame_name='new_frame')
```

delete_pre_planned_motion(*args, **kwargs)

Deletes pre planned motion. If this function is executed a already pre planned motion is removed and cannot be executed again.

Parameters**motion_id** (*int*) – the id which should be removed (type: identifier - int, units: N/A, required: Yes)**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()

r.init_program()
r.pre_plan_move_joint([0.0] * r.dof, motion_id = 1)
r.delete_pre_planned_motion(motion_id = 1)

r.stop()
```

delete_reference_frame(*name*, *new_parent_name*='World')

Delete a user-defined reference frame in the database.

Parameters

- **name** (*str*) – Name of the reference frame to be deleted.
- **new_parent_name** (*str*) – Name of the new parent frame to be set for the points in deleted frame.

Returns

True if the user frame was deleted successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.

- **ValueError** – If user frame deletion fails with the given input data.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.delete_reference_frame(name='new_frame')
```

disable_collision_detection()

Disable collision detection on the robot.

Returns

True if reflex collision avoidance is disabled, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.disable_collision_detection()
```

disable_friction_mode()

To turn off friction mode (smooth free drive mode).

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **WrongMode** – If the Robot is either not in ‘teach’ mode, in simulation, or if free drive mode is not enabled.
- **ConnectionError** – If the Robot is powered off or if an internal error occurs.
- **RuntimeError** – If friction mode cannot be controlled in the current version of the Control Software.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.turn_off_friction_mode()
```

disable_reflex()

Disable reflex after collision.

Returns

True if reflex collision avoidance is disabled, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.disable_reflex()
```

disconnect_external_device(*file_path: str, ignore_no_connection: bool = False*) → bool

Disconnects from an external device.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~data/process/).
- **ignore_no_connection** (*bool, optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

True if disconnection was successful, False otherwise.

Return type

bool

Raises

DriverError – If the disconnection fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
success = r.disconnect_external_device("external_device.json")
```

enable_collision_detection()

Enable collision detection on the robot.

Returns

True if reflex collision avoidance is enabled, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.enable_collision_detection()
```

enable_friction_mode()

To turn on friction mode (smooth free drive mode). Free drive mode needs to be activated beforehand.



Warning:

This might lead to the robot falling!

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **WrongMode** – If the Robot is either not in ‘teach’ mode, in simulation, or if free drive mode is not enabled.
- **ConnectionError** – If the Robot is powered off or if an internal error occurs.

- **RuntimeError** – If friction mode cannot be controlled in the current version of the Control Software.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.turn_on_free_drive_mode()
r.turn_on_friction_mode()
```

enable_reflex()

Enable reflex after collision.

Returns

True if reflex collision avoidance is enabled, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.enable_reflex()
```

encoder2rad(*impulses*, *offset=False*)

Convert encoder impulses to joint angles in radians.

Parameters

- **impulses** (*list*) – A list of encoder impulses for each joint (Required).
- **offset** (*bool*) – If True considers robot offset values for calculation (Optional).

Returns

A list of joint angles in radians corresponding to the provided encoder impulses.

Return type

list

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.get_current_load_side_encoder_values()
joint_angles = r.encoder2rad(encoder_ticks)
print(joint_angles)
```

execute_closest_orthogonal_tcp_align(*args, **kwargs)

Execute the closest orthogonal axes-aligned TCP alignment motion.

Parameters

args (*list or str*) – Target pose, which can be either:

- A list in the format [x, y, z, a, b, c] representing the 6D pose.
- A string specifying the name of a saved point, e.g., “Home”.

Returns

True if the TCP alignment is executed successfully, False otherwise.

Return type

bool

Sample Usage:

Example 1:

```

from neurapy.robot import Robot
robot = Robot()

target_pose = [0.4983978832107115, -0.03249999999999999, 0.
↪ 3344977006474223, -3.0510184133354567e-06, 3.141585287645838, 2.
↪ 7488999999887627] # list of floating point values denoting the
↪ 6D pose.
robot.execute_closest_orthogonal_tcp_align(target_pose)

```

Example 2:

```

from neurapy.robot import Robot
robot = Robot()

target_pose = "Home" # string of the name of a saved point.
robot.execute_closest_orthogonal_tcp_align(target_pose)

```

execute_exact_tcp_align(*args, **kwargs)

Execute the exact axes-aligned TCP alignment motion.

Parameters

args (*list or str*) – Target pose, which can be either:

- A list in the format [x, y, z, a, b, c] representing the 6D pose.
- A string specifying the name of a saved point, e.g., “Home”.

Returns

True if the TCP alignment is executed successfully, False otherwise.

Return type

bool

Sample Usage:**Example 1:**

```

from neurapy.robot import Robot
robot = Robot()

target_pose = [0.4983978832107115, -0.03249999999999999, 0.
↪ 3344977006474223, -3.0510184133354567e-06, 3.141585287645838, 2.
↪ 7488999999887627] # list of floating point values denoting the
↪ 6D pose.
robot.execute_exact_tcp_align(target_pose)

```

Example 2:

```

from neurapy.robot import Robot
robot = Robot()

target_pose = "Home" # string of the name of a saved point.
robot.execute_exact_tcp_align(target_pose)

```

execute_external_device_function(file_path: str, function_name: str, function_parameters: dict = {}, ignore_no_connection: bool = False) → Dict[str, str]

Executes a function of an external device.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **function_name** (*str*) – The name of the function to execute on the external device (required: Yes).
- **function_parameters** (*dict*) – A dictionary containing values for the function's parameters (optional: Yes).
- **ignore_no_connection** (*bool*, *optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

A dictionary containing the function's return values. Each key corresponds to an attribute name.

Return type

Dict[str, str]

Raises

DriverError – If the execution of the function fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
function_return_values = r.execute_device_function("external_device.
↪ json", "my_function", {"param1": "string_value", "param2": 123})
```

execute_external_device_safety_stop (*file_path: str*, *function_parameters: dict = {}*, *ignore_no_connection: bool = False*) → bool

Executes the safety stop function on an external device.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **function_parameters** (*dict*) – A dictionary containing values for the safety stop function's parameters (optional: Yes).
- **ignore_no_connection** (*bool*, *optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

True if safety stop was executed successfully, False otherwise.

Return type

bool

Raises

DriverError – If executing the safety stop function fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
success = r.execute_external_device_safety_stop("external_device.json
↪ ", {"param1": "value1"})
```

execute_pre_plan_motion(*args, **kwargs)

Executes pre planned motion from the motion storage.

This function allows the execution of pre planned motions. For this the executor needs to get the specific motion id of the motion, so that it can be executed at a later time.

Parameters

- **motion_id** (*int*) – The motion id which should be executed. (units: N/A, default: None, required: Yes)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.init_program()
planned_motion_id = r.pre_plan_move_joint([0.0] * r.dof, motion_id = 1)
r.execute_pre_plan_motion(motion_id=1) # or r.pre_plan_executor(motion_
↪ id=pre_plan_motion_id)
r.stop()
```

execute_pre_plan_motion_from_json(*args, **kwargs)

Executes pre planned motion from json file.

This function allows the execution of pre planned motions. To execute the motion a json file needs to be specified.

Parameters

- **file_path_and_name** (*str*) – location and name of json file (type: location and name - string, units: N/A, required: Yes)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()

file_path_and_name = "/home/Desktop/motion_1"

r.init_program()
r.pre_plan_move_joint([0.0] * r.dof, motion_id = 1)
r.save_pre_planned_motion_to_json(motion_id = 1, file_path_and_name = ↵
↪ file_path_and_name)
r.execute_pre_plan_motion_from_json(file_path_and_name = file_path_and_
↪ name)
r.stop()
```

execute_program(name)

To run a program created from teach pendant's tree program creator

Parameters

- **name** (*str*) – The name of the saved program to execute.

Returns

True if the program executed successfully, raises an exception if execution encountered an error.

Return type

bool

Raises

Exception – If an exception is raised during program execution, this function raises it with the exception message.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
try:
    result = r.execute_program("Program_001")
    if result:
        print("Program executed successfully.")
except Exception as e:
    print("Error: " + str(e))
```

execute_z_axis_tcp_align(*args, **kwargs)

Execute the z-axis-aligned TCP alignment motion.

Parameters

args (*list or str*) – Target pose, which can be either:

- A list in the format [x, y, z, a, b, c] representing the 6D pose.
- A string specifying the name of a saved point, e.g., “Home”.

Returns

True if the z-axis alignment motion is executed successfully, False otherwise.

Return type

bool

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot
robot = Robot()

target_pose = [0.4983978832107115, -0.03249999999999999, 0.
↪ 3344977006474223, -3.0510184133354567e-06, 3.141585287645838, 2.
↪ 7488999999887627] # list of floating point values denoting the
↪ 6D pose.
robot.execute_z_axis_tcp_align(target_pose)
```

Example 2:

```
from neurapy.robot import Robot
robot = Robot()

target_pose = "Home" # string of the name of a saved point.
robot.execute_z_axis_tcp_align(target_pose)
```

executor(*args, **kwargs)

Execute already planned motion.

This function allows the execution of planned motions in order they are planned. For this to work the motions need to be planned with plan_move_circular/joint/linear/composite functions. Note that also adding a move_circular/joint/linear/composite is executing the already planned motions in the order they are planned in.

Parameters

blocking (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
planned_motion_ids = r.plan_move_joint([0]*r.dof) # Replace with
↳ your planned motion IDs
r.executor()
```

get_analog_input(*io_name*)

Retrieve the value of an analog input.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*int*) – The identifier of the analog input.

Returns

The value of the specified analog input.

Return type

float

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_analog_input(1)
print(io_get)
```

get_analog_output(*io_name*)

Retrieve the value of an analog output.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*int*) – The identifier of the analog output.

Returns

The value of the specified analog output.

Return type

float

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_analog_output(1)
print(io_get)
```

get_commanded_joint_angles()

Get commanded joint angles of the robot.

Returns

A list of joint angles in radians.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_angles = r.get_commanded_joint_angles()
```

get_commanded_joint_angles_with_timestamp()

Get the comamnded joint angles of the robot along with UTC timestamp.

Returns

Tuple of a list of joint angles in radians and UTC timestamp

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_angles, timestamp = r.get_commanded_joint_angles_with_
↪timestamp()
```

get_current_joint_angles()

Get current joint angles of the robot.

Returns

A list of joint angles in radians.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_angles = r.get_current_joint_angles()
```

get_current_joint_angles_with_timestamp()

Get the current joint angles of the robot along with UTC timestamp.

Returns

Tuple of a list of robot joint angles in radians and UTC timestamp

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_angles, timestamp = r.get_current_joint_angles_with_timestamp()
```

get_current_joint_torques()

Get the current joint torques of the robot.

Returns

List of joint torque values

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_torques = r.get_current_joint_torques()
```

get_current_joint_torques_with_timestamp()

Get the current joint torques of the robot along with UTC timestamp.

Returns

Tuple of a list of joint torque values and UTC timestamp.

Return type

tuple

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
joint_torques, timestamp = r.get_current_joint_torques_with_
    ↪ timestamp()
```

get_current_joint_velocities()

Get current joint velocities of the robot.

Returns

List of robot joint velocities in radians/sec

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_velocities = r.get_current_joint_velocities()
```

get_current_joint_velocities_with_timestamp()

Get the current joint velocities of the robot along with UTC timestamp.

Returns

Tuple of a list of robot joint velocities in radians/sec and UTC timestamp

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_velocities, timestamp = r.get_current_joint_velocities_with_
    ↪ timestamp()
```

get_current_load_side_encoder_values()

Get the current load side encoder ticks of the robot.

Returns

List of encoder tick values

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.get_current_load_side_encoder_values()
```

get_current_motor_side_encoder_values()

Get the current motor side encoder ticks of the robot .

Returns

List of encoder tick values

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.get_current_motor_side_encoder_values()
```

get_current_tool_cogs()

Get the current tool center of gravity (COG) in XYZ directions(measured from robot's flange frame/tool mounting point).

Returns

List representing the tool COG in XYZ directions.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool_cog = r.get_current_tool_cogs()
```

get_current_tool_inertias()

Get the current tool inertias Ixx, Iyy, Izz, Ixy, Ixz, and Iyz.

Returns

List representing the tool inertias Ixx, Iyy, Izz, Ixy, Ixz, and Iyz.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool_inertias = r.get_current_tool_inertias()
```

get_current_tool_mass()

Get the current tool mass.

Returns

The mass of the tool.

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool_mass = r.get_current_tool_mass()
```

get_current_tool_properties()

Get the current tool properties.

Returns

List containing the updated tool properties, i.e., [tool_mass, roll_offset, pitch_offset, yaw_offset, x_offset, y_offset, z_offset, COG_x, COG_y, COG_z, Ixx, Iyy, Izz, Ixy, Ixz, Iyz].

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool_properties = r.get_current_tool_properties()
```

get_current_tool_rpy_offsets()

Get the current tool roll, pitch, and yaw (RPY) offsets in radians.

Returns

List representing the tool RPY offsets in radians.

Return type

List

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool_rpy_offsets = r.get_current_tool_rpy_offsets()
```

get_current_tool_translation_offsets()

Get the current tool translation offsets in XYZ directions.

Returns

List representing the tool translation offsets in XYZ directions.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool_offsets = r.get_current_tool_translation_offsets()
```

get_diagnostics()

Method to query current robot diagnostics

Returns

If critical, a dictionary containing diagnostics values (i.e., critical, powered_off, collision_status); otherwise, an empty dictionary.

Return type

dict

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
print(r.get_diagnostics())
```

get_digital_input(*io_name*)

Get the value of a digital input.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*int*) – The name (pin number) of the digital input to retrieve the value from.

Returns

The value of the digital input, either 0 or 1.

Return type

int

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_digital_input(1)
print(io_get)
```

get_digital_output(*io_name*)

Get the value of a digital output.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*int*) – The name (pin number) of the digital output to retrieve the value from.

Returns

The value of the digital output, either 0 or 1.

Return type

int

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_digital_output(1)
print(io_get)
```

get_doc(*function_name*)

Get the description and sample usage of the specified function.

Parameters

function_name (*str*) – The name of the function for which description is needed.

Returns

Short description and usage of the function.

Return type

str

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
description = r.get_doc("move_joint")
print(description)
```

get_encoder_offsets()

To retrieve the encoder offsets of a robot.

Returns

A list containing the encoder offsets for the robot's encoders.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
encoder_offsets = r.get_encoder_offsets()
```

get_errors(*args, **kwargs)

Query the list of errors present on the robot.

Returns

A list of errors present on the robot.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
print(r.get_errors())
```

get_ext_digital_input(io_name)

Get the value of an external digital input.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*int*) – The pin number of the external digital input to retrieve the value from.

Returns

The value of the digital input, either 0 or 1.

Return type

int

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_digital_input(1)
print(io_get)
```

get_ext_digital_output(io_name)

Get the value of an external digital output.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*int*) – The pin number of the external digital output to retrieve the value from.

Returns

The value of the digital output, either 0 or 1.

Return type

int

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_ext_digital_output(1)
print(io_get)
```

get_flange_pose(*timestamp=False, representation='rpy'*)

Get the current flange pose in XYZRPY format.

Returns

List representing the flange pose with XYZRPY values.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
flange_pose = r.get_flange_pose()
```

get_flange_pose_quaternion()

Get the current flange pose in XYZwxyz format.

Returns

List representing the flange pose with [X,Y,Z,w,x,y,z].

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
flange_pose_quaternion = r.get_flange_pose_quaternion()
```

get_flange_pose_quaternion_with_timestamp()

Get the current flange pose in XYZwxyz format with UTC timestamp.

Returns

Tuple representing the flange pose with [X,Y,Z,w,x,y,z] and UTC timestamp.

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
flange_pose_quaternion, timestamp = r.get_flange_pose_quaternion_with_
    ↪ timestamp()
```


get_flange_pose_with_timestamp()

Get the current tool flange pose in XYZRPY format with UTC timestamp.

Returns

Tuple of the flange pose with XYZRPY values and UTC timestamp.

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
flange_pose, timestamp = r.get_flange_pose_with_timestamp()
```

get_io_configuration()

Get the number of available IOs.

Returns

A dictionary containing the type and number of the IOs.

Return type

dict

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_configuration = r.get_io_configuration()
print(io_configuration)
```

get_io_status()

Get all the available IOs and it's values

Returns

A dictionary containing all the IOs and it's available values

Return type

dict

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_io_status()
print(io_get)
```

get_joint_acceleration()

Get the current acceleration value for joint motion.

Returns

The current acceleration value for joint motion (percentage of maximum angular acceleration).

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
```

(continues on next page)

(continued from previous page)

```
acceleration = r.get_joint_acceleration()
print(f"Current joint acceleration: {acceleration}")
```

get_joint_speed()

Get the current speed value for joint motion.

Returns

The current speed value for joint motion (percentage of maximum angular speed).

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
speed = r.get_joint_speed()
print(f"Current joint speed: {speed}")
```

get_linear_acceleration()

Get the current acceleration value for linear motion.

Returns

The current acceleration value for linear motion (m/s²).

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
current_acceleration = r.get_linear_acceleration()
print(f"Current linear acceleration: {current_acceleration} m/s2")
```

get_linear_speed()

Get the current speed value for linear motion.

Returns

The current speed value for linear motion (m/s).

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
current_speed = r.get_linear_speed()
print(f"Current linear speed: {current_speed} m/s")
```

get_load_side_encoder_values_with_timestamp()

Get the current load side encoder ticks of the robot along with UTC timestamp.

Returns

Tuple of a list of encoder tick values and UTC timestamp

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks, timestamp = r.get_load_side_encoder_values_with_
    ↳ timestamp()
```

get_motor_side_encoder_values_with_timestamp()

Get the current motor side encoder ticks of the robot along with UTC timestamp.

Returns

Tuple of a list of encoder tick values and UTC timestamp

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks, timestamp = r.get_motor_side_encoder_values_with_
    ↳ timestamp()
```

get_override()

Get the current override value.

Returns

The current override value.

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
override_value = r.get_override()
print(f"Current override value: {override_value}")
```

get_point(name, representation='Joint')

Retrieves a point from the database based on the given name.

Parameters

- **name** (*str*) – The name/id of the point to retrieve from the database.
- **representation** (*str*) – To specify the point representation type.
 - **Joint** - Default - returns Joint values of the stored point
 - **Cartesian** - returns Cartesian pose of the stored point

Returns

A list containing the details of the point. For Joint representation - List contains the values of joint angles in radian. For Cartesian representation - List contains X,Y,Z,R,P,Y values

Return type

list

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If no point is saved in the database with the given name or the given representation type is invalid.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
point = r.get_point("P1",representation="Joint")
print(point)
point = r.get_point("P1",representation="Cartesian")
print(point)
```

get_point_names()

Retrieves a list of names of all saved points in the database.

Returns

A list of names of the saved points.

Return type

list

Raises

ConnectionError – If connection to the remote database cannot be established.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
point_names = r.get_point_names()
print(point_names)
```

get_power_status()

Query the power status of the robot.

Returns

The power status of the robot.

- 'POWERED_OFF': If the robot is powered off.
- 'POWERED_ON' : If the robot is powered on.

Return type

str

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
print(r.get_power_status())
```

get_program_names()

Retrieves list of program names from the robot's database

Returns

A list of names of the programs available in the robot's database.

Return type

list

Raises

ConnectionError – If connection to the remote database cannot be established.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
program_names = r.get_program_names()
print(program_names)
```

get_reference_frame(*name*)

Retrieve the coordinates and orientations of a user-defined reference frame.

Parameters

name (*str*) – The name of the reference frame to retrieve.

Returns

A list containing the values of the reference frame ([X,Y,Z,R,P,Y]).

Return type

list

Raises

ConnectionError – If there is a failure to connect to the robot's database.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
frame = r.get_reference_frame("tool_frame")
print(frame)
```

get_reference_frame_with_offset(*name, offset*)

Retrieve a reference frame with an applied offset to its position.

Parameters

- **name** (*str or list*) – The name of the reference frame or the reference frame as a list.
- **offset** (*list*) – A list containing the offset values [X, Y, Z] to apply to the frame's position.

Returns

A list containing the [X, Y, Z, A, B, C] values (coordinates and angles) of the translated reference frame.

Return type

list

Raises

ValueError – If the name argument is neither a valid string nor a list. If the offset argument is not a list of length 3.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
x_offset = 0.02 # in meters
y_offset = 0.1 # in meters
z_offset = 0.0 # in meters
frame = r.get_reference_frame_with_offset("world", [x_offset, y_offset,
↪ z_offset])
print(frame)
```

get_register_bytes(*tag_name, first_element, last_element, T2O=False*)

Returns a python “bytes” object from a T2O or O2T register specified by its tag name and index range from first to last element.

Parameters

- **tag_name** (*str*) – The register's tag name (without any element index) as found in the official EDS files or in the detailed mapping in EtherNET/IP users documentation.
- **first_element** (*int*) – Index of the first requested register element.

- **last_element** (*int*) – Index of the last requested register element.
- **T20** (*bool*) – Use True if the register is in Target-to-Originator registry instead of Originator-to-Target.

Returns

Encoded values or None on failure (invalid range, communication failure).

Return type

bytes

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
octet_list = r.get_register_bytes("INTIN", 1, 4) # return second to
↪ fifth integer elements from
# input (O2T) integer register
```

get_register_values(tag_name, first_element, last_element, T2O=False)

Returns values from a T2O or O2T register specified by its tag name and index range from first to last element

Parameters

- **tag_name** (*str*) – The register's tag name (without any element index) as found in the official EDS files or in the detailed mapping in EtherNET/IP users documentation.
- **first_element** (*int*) – Index of the first requested register element.
- **last_element** (*int*) – Index of the last requested register element.
- **T20** (*bool*) – Use True if the register is in Target-to-Originator registry instead of Originator-to-Target.

Returns

List of Python values or an empty list on failure (invalid range, communication failure).

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
octet_list = r.get_register_values("INTIN", 1, 4) # return second to
↪ fifth integer elements from
# input (O2T) integer register
```

get_safety()

Get the current status of Safety Mode.

Returns

True if the current mode is safety mode.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.get_safety()
```

get_selected_fts_data()

Retrieves the selected external fts from the robot's database.

Returns

selected_fts_data

Return type

dict

Raises

ConnectionError – If connection to the remote database cannot be established.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
selected_fts_data = r.get_selected_fts_data()
```

get_selected_tool_data()

Retrieves the selected tool data from the robot's database.

Returns

The selected tool data from the robot's database.

Return type

list

Raises

ConnectionError – If connection to the remote database cannot be established.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tool = r.get_selected_tool_data()
print(tool)
```

get_selected_tool_name()

Retrieves the selected tool name from the robot's database.

Returns

The name of the selected tool.

Return type

str

Raises

- **KeyError** – If the retrieved tool data does not contain a valid tool name.
- **ConnectionError** – If connection to the remote database cannot be established.

get_servo_trajectory_scaling_factor(*args, **kwargs)

Method to get the trajectory scaling factor from ServoJ

Returns

The scaling factor.

Return type

float

Sample Usage:

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

dof = 6

otg = Ruckig(dof, 0.001) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]
inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
res = Result.Working

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code < 3):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

get_tcp_force()

Get the current TCP (Tool Center Point) force values of the robot.

Returns

List of TCP force values (Fx, Fy, Fz)

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_forces = r.get_tcp_force()
print("TCP Force Values:", tcp_forces)
```

get_tcp_pose(timestamp=False, representation='rpy')

to get the current tcp pose in XYZRPY format

Returns

List of TCP pose as [X, Y, Z, R, P, Y]

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_pose = r.get_tcp_pose()
print(tcp_pose)
```

get_tcp_pose_quaternion()

to get the current tcp pose in XYZwxyz format

Returns

List of TCP pose as [X, Y, Z, w, x, y, z].

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_pose_quaternion = r.get_tcp_pose_quaternion()
```

get_tcp_pose_quaternion_with_timestamp()

Get the current tcp pose in XYZwxyz format along with UTC timestamp

Returns

Tuple of TCP pose as [X, Y, Z, w, x, y, z] and UTC timestamp.

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_pose_quaternion = r.get_tcp_pose_quaternion()
```

get_tcp_pose_with_timestamp()

Get the current robot's TCP pose in XYZRPY format along with UTC timestamp.

Returns

Tuple of the robot's TCP pose and UTC timestamp.

Return type

tuple

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_pose, timestamp = r.get_tcp_pose_with_timestamp()
```

get_tcp_speed()

Get the current speed of the robot's TCP (Tool Center Point).

Returns

The current TCP speed.

Return type

float

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_speed = r.get_tcp_speed()
```

get_tcp_torque()

Get the current TCP (Tool Center Point) torque values of the robot.

Returns

List of TCP torque values (Mx, My, Mz)

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_torques = r.get_tcp_torque()
print("TCP torque Values:", tcp_torques)
```

get_tcp_wrench()

Get the current TCP (Tool Center Point) wrench values, including both force and torque components.

Returns

List containing TCP force values (Fx, Fy, Fz) followed by torque values (Mx, My, Mz).

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
tcp_wrench = r.get_tcp_wrench()
print("TCP Wrench Values:", tcp_wrench)
```

get_tool_analog_input(*io_name*)

Get the value of a tool analog input.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*str*) – The name of the tool analog input.

Returns

The value of the tool analog input.

Return type

float

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_tool_analog_input(1)
print(io_get)
```

get_tool_digital_input(*io_name*)

Get the value of a tool digital input.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*str*) – The name of the tool digital input.

Returns

The value of the tool digital input, typically represented as 0 or 1.

Return type

int

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_tool_digital_input(1)
print(io_get)
```

get_tool_digital_output(*io_name*)

Get the value of a tool digital output.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

io_name (*str*) – The name of the tool digital output.

Returns

The value of the tool digital output, typically represented as 0 or 1.

Return type

int

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.get_tool_digital_output(1)
print(io_get)
```

get_tools()

Retrieves list of tools from the robot's database.

Returns

A list of tools available in the robot's database.

Return type

list

Raises**ConnectionError** – If connection to the remote database cannot be established.**Sample Usage:**

```
from neurapy.robot import Robot
r = Robot()
tools = r.get_tools()
print(tools)
```

get_warnings(*args, **kwargs)

Query the list of warnings present on the robot.

Returns

A list of warnings present on the robot.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
print(r.get_warnings())
```

grasp()

To grasp the object with the attached gripper

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.grasp()
```

gripper(*args, **kwargs)

Method to control the gripper attached to the robot

Sample Usage:

```
from time import sleep
from neurapy.robot import Robot

r = Robot()
r.gripper('on')
sleep(2)
r.gripper('off')
```

init_program(*args, **kwargs)

To allow running motion commands, without this, the planning of motion will be blocked and no motion will take place.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
r.move_joint('Home')
r.stop()
```

Deprecated since version 5.0.9: The `init_program` method is deprecated and handled automatically by the robot. Manual invocation is no longer required.

io(*args, **kwargs)

Access and manipulate (output) I/O values on the robot.

This method allows for retrieving or setting the values of I/O operations.

Parameters

- **operation_type** (*str*) – The type of operation to perform.
 - **get**: Retrieve the I/O value.
 - **set**: Set the output I/O value.
- **io_name** (*str*) – The name of the I/O. (A list of available I/O names can be found in the documentation.)
- **target_value** (*Any*) – The target value of the I/O. This parameter is required only for the 'set' operation.

Returns

- For 'get' operation: The value of the queried *io_name*.
- For 'set' operation: True if the operation succeeded, False otherwise.

Return type

Union[Any, bool]

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_get = r.io("get", io_name = "DO_1")
io_set = r.io("set", io_name = "DO_2", target_value = True)
print(io_get, io_set)
```

is_collision_enabled()

Check if reflex after collision is enabled for the robot.

Returns

True if reflex collision avoidance is enabled, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.is_collision_enabled():
    print("Collision detection is enabled.")
else:
    print("Collision detection is not enabled.")
```

is_external_device_connected(file_path: str, ignore_no_connection: bool = False) → bool

Checks if an external device is connected.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~data/process/).

- **ignore_no_connection** (*bool, optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

True if the device is connected, False otherwise.

Return type

bool

Raises

DriverError – If checking the connection status fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
is_connected = r.is_external_device_connected("external_device.json")
```

is_external_device_in_error(*file_path: str, ignore_no_connection: bool = False*) → bool

Checks if an external device is in error state.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **ignore_no_connection** (*bool, optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

True if the device is in error state, False otherwise.

Return type

bool

Raises

DriverError – If checking the error state fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
is_in_error = r.is_external_device_in_error("external_device.json")
```

is_free_drive_mode_enabled()

Get the status of the robot free drive mode.

Returns

The status of the free drive mode. True if turned, else False.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
status = r.is_free_drive_mode_enabled()
```

is_reflex_enabled()

Check if reflex after collision is enabled for the robot.

Returns

True if reflex collision avoidance is enabled, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.is_reflex_enabled():
    print("Reflex collision avoidance is enabled.")
else:
    print("Reflex collision avoidance is not enabled.")
```

is_robot_in_automatic_mode()

Method to check whether the robot is in automatic mode or not

Returns

True if the robot is in automatic mode, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_automatic_mode():
    print("Robot is in automatic mode")
else:
    print("Robot is not in automatic mode")
```

is_robot_in_collision()

Check if the robot is currently in collision.

Returns

True if the robot is in collision, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_collision():
    print("Robot is in collision.")
else:
    print("Robot is not in collision.")
```

is_robot_in_semi_automatic_mode()

Method to check whether the robot is in semi automatic mode or not

Returns

True if the robot is in semi-automatic mode, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_semi_automatic_mode():
    print("Robot is in semi automatic mode")
else:
    print("Robot is not in semi automatic mode")

```

is_robot_in_simulation()

Method to check whether the robot is in simulation or not

Returns

True, if the robot is in simulation, False if it is in real mode.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_simulation():
    print("Robot is in simulation")
else:
    print("Robot is in real")

```

is_robot_in_teach_mode()

Method to check whether the robot is in teach mode or not

Returns

True if the robot is in teach mode, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
if r.is_robot_in_teach_mode():
    print("Robot is in teach mode")
else:
    print("Robot is not in teach mode")

```

is_robot_powered_on()

Check if the robot is powered on.

Returns

True if the robot is powered on, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
print(r.is_robot_powered_on())

```

jog(*args, **kwargs)

Method to jog programatically. This needs to be used in conjunction with turn_on_jog, turn_off_jog methods.

Parameters

set_jogging_external_flag (*int*) – A flag to enable external jogging mode.

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()

"""
jog_velocity - velocity ranging from [-1,1] for all joints. Signifies
↳ percentage of total speed used (1 = 100%)
jog_type - can be either cartesian or joint jogging
turn_on_jog sets the parameters of external jogging, jogging velocity,
↳ jogging type and sets the mode for jogging to external mode.
"""

r.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
↳ 'Joint')
r.jog(set_jogging_external_flag=1)
i = 0
"""
Requires minimum number of cycles in the loop for performing jogging.
Depends upon jogging velocity, override.
"""
while i < 500:
    #command to set flag to start jogging in external mode. This
    ↳ command has to be used each time external jog command has to be sent
    r.jog(set_jogging_external_flag=1)
    i += 1

#Command to switch back to internal jogging mode (GUI)
r.turn_off_jog()
```

list_methods()

To list the available functions in the API

Returns

A table listing the available functions and short descriptions of the functions.

Return type

PrettyTable

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
methods = r.list_methods()
print(methods)
```

list_recorded_path()

To list the recorded path using ZeroG.

Returns

List of file names. Empty list, if no files are found.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
recorded_path = r.list_recorded_path()
print(recorded_path)
```

load_pre_planned_motion_from_json(*args, **kwargs)

Loads pre planned motion from json file and stores it.

Parameters

- **motion_id** (*int*) – the id with which the motion should be stored (type: identifier - int, units: N/A, required: No)
- **file_path_and_name** (*string*) – location and name of json file (type: location and name - string, units: N/A, required: Yes)

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()

file_path_and_name = "/home/Desktop/motion_1"

r.init_program()
r.pre_plan_move_joint([0.0] * r.dof, motion_id = 1)
r.save_pre_planned_motion_to_json(motion_id = 1, file_path_and_name =
    ↪file_path_and_name)

r.load_pre_planned_motion_from_json(motion_id = 2, file_path_and_name =
    ↪file_path_and_name)
r.execute_pre_plan_motion(motion_id = 2)

r.stop()
```

lock_cartesian_axes(cartesian_axes)

Lock specific Cartesian axes while in free drive mode.

Parameters

cartesian_axes (*List[str]*) – A list of Cartesian axes (“X”, “Y”, “Z”, “A”, “B”, “C”) to be locked.

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **ValueError** – If an unknown axis is provided. For example, if an invalid axis like “D” is passed when locking Cartesian axes.
- **ValueError** – If the number of provided Cartesian axes does not match the robot’s available axes.
- **RuntimeError** – If the robot is in motion and axis lock cannot be applied.
- **RuntimeError** – If axis lock cannot be controlled in this version of the control software.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.lock_cartesian_axes(["X", "Z"])
```

lock_joint_axes(*joint_axes*)

Lock specific joint axes while in free drive mode.

Parameters

joint_axes (*List[bool]*) – A list of booleans indicating which joint axes should be locked (True for locked, False for free).

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **ValueError** – If the number of provided joint axes does not match the robot's degrees of freedom. For example, if there are more or fewer values than the robot's DOF (degrees of freedom).
- **ValueError** – If an unknown axis is provided when locking Cartesian axes.
- **RuntimeError** – If the robot is in motion and axis lock cannot be applied.
- **RuntimeError** – If axis lock cannot be controlled in this version of the control software.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.lock_joint_axes([True, False, True, False, False, True])
```

motion_status()

Query the motion status of the robot.

Returns

The motion status of the robot.

- 'NOT_RUNNING': If the robot is not running.
- 'RUNNING' : If the robot is running.
- 'PAUSED' : If the robot is in a paused state.

Return type

str

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
print(r.motion_status())
```

move_circular(*args, **kwargs)

To move the robot in a circular path across the given poses.

This method takes the following arguments/keyword arguments:

Parameters

- **target_pose** (*list*) – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **target_joint** (*list*) – List of joint configurations, needs current_joint_angles if not provided. (type: Joint configuration - [], float, units: Joint values at each position, required: No)
- **motion_id** (*int*) – id provided for the motion. (units: -, default: 0, required: No)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 100.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.4, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 0.4, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 100.0, required: No)
- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration, required if target_joint not provided. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. Required when weaving is enabled.
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)

- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.

- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "dynamic",
    "target_pose": [
        0.3744609827431085,
        -0.3391784988266481,
        0.23276604279256016,
        3.14119553565979,
        -0.00017731254047248513,
        -0.48800110816955566
    ],
}
```

(continues on next page)

(continued from previous page)

```

[
    0.37116786741831503,
    -0.19686307684994242,
    0.23300456855796453,
    3.141423225402832,
    -0.00020668463548645377,
    -0.48725831508636475
],
[
    0.5190337951593321,
    -0.1969996948428492,
    0.23267853691809767,
    3.1414194107055664,
    -0.00017726201622281224,
    -0.48750609159469604
]
],
"current_joint_angles": r.get_current_joint_angles(),
"weaving_parameters": [
    {
        "start_target_pose_number": 1,
        "end_target_pose_number": 3,
        "frequency": 0.01,
        "pattern": "sine",
        "amplitude_left": 0.006,
        "amplitude_right": 0.006,
        "dwell_time_left": 0,
        "dwell_time_right": 0,
        "azimuth": 0.0,
        "elevation": 0.0
    }
],
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    },
    "Mz": {
        "is_active": False,
        "value": 0.0
    }
}

r.move_circular(**circular_property)
r.stop() # if there are multiple motions than, this needs to be
        ↪ called only once at the end of the script

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
r.move_circular(["P1", "P2", "P3"])
r.stop()

```

move_composite(*args, **kwargs)

To move the robot in the specified linear and circular motion combinations.

Parameters

- **commands** (*list*) – List of linear and circular command combinations. Each element is a dictionary with a single key “linear” or “circular”, and the value is a dictionary containing the command parameters.

Each command dictionary structure:

– linear command: Dictionary with key “linear” and value containing:

- * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
- * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- * **velocity** (*float*): Translation Velocity, required when use_children_velocities set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
- * **rotation_velocity** (*float*): Rotational Velocity, required when use_children_velocities set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)

– circular command: Dictionary with key “circular” and value containing:

- * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
- * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- * **velocity** (*float*): Translation Velocity, required when use_children_velocities set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)

- * **rotation_velocity** (*float*): Rotational Velocity, required when **use_children_velocities** set to True. (units: rad/sec, default: **rotation_speed** set for entire MoveComposite, required: No)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 500.0, required: No)
- **use_children_velocities** (*bool*) – Use individual speed and rotation speeds for each child motion, as set in commands parameter. (units: N/A, default: False, required: No)
- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: True, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given **blend_radius**.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. (required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - **sine** : SINE, if selected uses a sine wave, allows configuration with sine values
 - **circle** : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)

- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.

- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "static",
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [
        {
            "linear": {
                "target_pose": [
                    -0.000259845199876027,
                    -0.5211437049195536,
                    0.4429382717719519,
                    3.14123272895813,
                    -0.0007908568368293345,
```

(continues on next page)

(continued from previous page)

```

        -1.570908784866333
    ],
    [
        -0.16633498440272945,
        -0.5201452059140722,
        0.4427486025872017,
        3.140937089920044,
        -0.0005319403717294335,
        -1.571555495262146
    ]
]
}
},
{
    "circular": {
        "target_pose": [
            [
                -0.16633498440272945,
                -0.5201452059140722,
                0.4427486025872017,
                3.140937089920044,
                -0.0005319403717294335,
                -1.571555495262146
            ],
            [
                -0.16540090985202305,
                -0.3983552679378624,
                0.44267608017426174,
                3.1407113075256348,
                -0.00036628879024647176,
                -1.5714884996414185
            ],
            [
                -0.33446498807559716,
                -0.3989652352814891,
                0.4421152856242009,
                3.1402060985565186,
                0.00030071483342908323,
                -1.572899580001831
            ]
        ]
    }
},
],
"weaving": False,
"weaving_parameters": [
    {
        "start_target_pose_number": 1,
        "end_target_pose_number": 4,
        "frequency": 0.01,
        "pattern": "sine",
        "amplitude_left": 0.006,
        "amplitude_right": 0.006,
        "dwell_time_left": 0,
        "dwell_time_right": 0,
        "azimuth": 0.0,
    }
]

```

(continues on next page)

(continued from previous page)

```

        "elevation": 0.0
    }
],
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}
r.move_composite(**composite_motion_property)
r.stop() # if there are multiple motions than, this needs to be
        ↪ called only once at the end of the script

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
composite_motion_property = {
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "static",
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "use_children_velocities": True,
    "commands": [
        {
            "linear": {

```

(continues on next page)

(continued from previous page)

```

        "target_pose": [
            [
                0.388698464,
                -0.297404085,
                0.490878791,
                -3.057496505,
                0.055407655,
                2.624106295
            ],
            [
                0.257535229,
                -0.30505092,
                0.491253917,
                -3.032448111,
                0.052689034,
                2.621777631
            ]
        ],
        "velocity": 0.1,
        "rotation_velocity": 0.174533
    },
    {
        "circular": {
            "target_pose": [
                [
                    0.257535229,
                    -0.30505092,
                    0.491253917,
                    -3.032448111,
                    0.052689034,
                    2.621777631
                ],
                [
                    0.033258337,
                    -0.397837013,
                    0.484987274,
                    -3.032448113,
                    0.052688999,
                    2.004043036
                ],
                [
                    -0.216170016,
                    -0.335635114,
                    0.484987274,
                    -3.032448113,
                    0.052688999000000002,
                    1.3484491189999999
                ]
            ],
            "velocity": 0.4,
            "rotation_velocity": 0.436332
        }
    },
    "weaving": False

```

(continues on next page)

(continued from previous page)

```

}
r.move_composite(**composite_motion_property)
r.stop()

```

move_joint(*args, **kwargs)

To move the robot to specified joint configuration in joint space.

Parameters

- **target_joint** (*list*) – List of joint configurations (type: List of Joint Values - float or int, units: radians, required: Yes) Alternative: (List of) String (type: String, name of existing point)
- **target_pose** – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: No)

Alternative: List of Strings (Type: [String], names of existing points). If not provided, the poses will be automatically generated from the current_joint_angles. :type target_pose: list :param speed: Angular Speed. (units: % of maximum angular speed, scale: 0 to 100, default: 25%, required: No) :type speed: float :param acceleration: Angular Acceleration. (units: % of maximum angular acceleration, scale: 0 to 100, default: 25%, required: No) :type acceleration: float :param jerk: Angular Jerk. (units: % of maximum angular jerk, default: 100.0, required: No) :type jerk: float :param current_joint_angles: Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No) :type current_joint_angles: list

Parameters

- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).

- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot
r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        2.5995838308821924,
```

(continues on next page)

(continued from previous page)

```

        0.24962416292345468,
        -1.8654403327490414,
        0.04503286318691005,
        -1.1740563715454926,
        0.10337461241185522
    ],
    [
        2.1372059994827075,
        0.24939733788589463,
        -1.8651270179353125,
        0.044771940725327274,
        -1.173860821592129,
        0.10315646291502645
    ],
    [
        1.9180047887810003,
        -0.24855170101601043,
        -1.3680228668892351,
        0.12404421791100637,
        -1.1914147150222498,
        -0.13255713717112075
    ]
],
"current_joint_angles": r.get_current_joint_angles(),
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}

```

(continues on next page)

(continued from previous page)

```
r.move_joint(**joint_property)
r.stop() # if there are multiple motions than, this needs to be
↳ called only once at the end of the script
```

Example 2:

```
from neurapy.robot import Robot
r = Robot()
r.move_joint(["Home"])
r.move_joint(["P1", "P2"])
r.stop()
```

move_joint_relative(*args, **kwargs)

To move the robot relative to the current position by the given cartesian offset. The motion is executed as joint motion.

Parameters

- **speed** (*float*) – Angular Speed. (units: % of maximum angular speed, default: 25, required: No)
- **acceleration** (*float*) – Angular Acceleration. (units: % of maximum angular acceleration, default: 25, required: No)
- **jerk** (*float*) – Angular Jerk. (units: % of maximum angular jerk, default: 25, required: No)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **offset_frame** (*str*) – Frame for applying relative offset. (units: N/A, default: "Base", required: No)
 - 'Base' : Applies relative offset to the base frame
 - 'Tool' : Applies relative offset to the tool frame
- **cartesian_offset** (*list*) – The cartesian offset that is applied. (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m - [R,P,Y] in radian, default: [0.0,0.0,0.0,0.0,0.0,0.0], required: Yes)
- **target_joint** (*list*) – The joint configuration at which the relative motion should start from (type: list - [a1, a2, a3, a4, a5, a6] for lara and [a1, a2, a3, a4, a5, a6, a7] for maira, units: axis values in radian, default: current joint angles, required: No) If the target_joint is used, the target_pose also needs to be configured, else the current joint angles and current pose are used
- **target_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No) If the target_pose is used, the target_joint also needs to be configured, else the current joint angle and current pose are used
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.

- 'hybrid' : Hybrid Motion/Force control mode.
- 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.

- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go_till_force_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **go_till_force_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **go_till_force_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **go_till_force_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **go_till_force_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **go_till_force_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **go_till_force_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed" : 50.0,
    "acceleration" : 50,
    "jerk": 50,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi / 4],
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}
```

(continues on next page)

(continued from previous page)

```

    },
    "go_till_force_parameters": {
        "is_enabled": True,
        "enable_reflex": True,
        "extra_search_offset": 0,
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": True,
                "value": 10
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

r.move_joint_relative(**joint_property)
r.wait_motion_finished() # only needed if blocking is False
r.stop() # if there are multiple motions than, this needs to be called,
→ only once at the end of the script

```

move_linear(*args, **kwargs)

To move the robot to specified poses in Cartesian/Task space.

Parameters

- **target_pose** (*list*) – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **target_joint** (*list*) – List of joint angles. (type: List of Joint Values - float, units: Joint angles in radians, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)

- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 500.0, required: No)
- **enable_blending** – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given `blend_radius`.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. (required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)

- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.

- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go_till_force_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **go_till_force_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **go_till_force_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **go_till_force_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **go_till_force_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "rotation_speed": 0.5,
    "enable_blending": True,
    "blending_mode": "static",
    "blend_radius": 0.005,
    "target_pose": [
        [
            0.3287228886,
            -0.1903355329,
            0.4220780352,
            0.08535207028439847,
            -2.797181496822229,
            2.4713321627410485
        ],
        [
            0.2093363791501374,
            -0.31711250784165884,
            0.422149168855134,
            -3.0565555095672607,
            -0.3447442352771759,
            -1.1323236227035522
        ],
        [
            0.2090521916195534,
            -0.5246753336643587,
            0.4218773613553828,
            -3.0569007396698,
            -0.3448921740055084,
```

(continues on next page)

(continued from previous page)

```

        -1.1323626041412354
    ],
    [
        0.3287228886,
        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ]
],
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"weaving_parameters": [
    {
        "start_target_pose_number": 1,
        "end_target_pose_number": 4,
        "frequency": 0.01,
        "pattern": "sine",
        "amplitude_left": 0.006,
        "amplitude_right": 0.006,
        "dwell_time_left": 0,
        "dwell_time_right": 0,
        "azimuth": 0.0,
        "elevation": 0.0
    }
],
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
  },
  "go_till_force_parameters": {
    "is_enabled": True,
    "enable_reflex": True,
    "extra_search_offset": 0,
    "force_vector": {
      "Fx": {
        "is_active": False,
        "value": 0
      },
      "Fy": {
        "is_active": False,
        "value": 0
      },
      "Fz": {
        "is_active": True,
        "value": 10
      }
    },
    "torque_vector": {
      "Mx": {
        "is_active": False,
        "value": 0.0
      },
      "My": {
        "is_active": False,
        "value": 0.0
      },
      "Mz": {
        "is_active": False,
        "value": 0.0
      }
    }
  }
}

r.move_linear(**linear_property)
r.stop() # if there are multiple motions than, this needs to be
         ↪ called only once at the end of the script

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
r.move_linear(["P1", "P2"])
r.stop()

```

move_linear_relative(*args, **kwargs)

To move the robot relative to the current position by the given cartesian offset. The motion is executed as Cartesian motion.

Parameters

- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)

- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec², default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec³, default: 500.0, required: No)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)
- **cartesian_offset** (*list*) – (type: offset list - [X,Y,Z,R,P,Y], units: [X,Y,Z] offset in m [R,P,Y] offset in radian, default: [0,0,0,0,0,0], required: Yes)
- **offset_frame** (*str*) – Frame for applying relative offset. (units: N/A, default: "Base", required: No)
 - 'Base' : Applies relative offset to the base frame
 - 'Tool' : Applies relative offset to the tool frame
- **target_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.

- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.

- `go_till_force_parameters["force_vector"]["Fy"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["force_vector"]["Fz"]` (*Dict[str, Any]*) – Force control setting in the Z direction.
- `go_till_force_parameters["force_vector"]["Fz"]["is_active"]` (*bool*) – Whether force control is active for the Z direction.
- `go_till_force_parameters["force_vector"]["Fz"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["torque_vector"]` (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- `go_till_force_parameters["torque_vector"]["Mx"]` (*Dict[str, Any]*) – Torque control setting around the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()
linear_property = {
    "speed" : 4.5,
```

(continues on next page)

(continued from previous page)

```

"acceleration" : 10.0,
"jerk": 1000.0,
"rotation_speed" : 1.5,
"rotation_acceleration" : 10.0,
"rotation_jerk" : 1000.0,
"blocking": True,
"offset_frame" : "Base",
"cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi/4],
"current_joint_angles": r.get_current_joint_angles(),
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
},
"go_till_force_parameters": {
    "is_enabled": True,
    "enable_reflex": True,
    "extra_search_offset": 0,
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": True,
            "value": 10
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}
r.move_linear_relative(**linear_property)
r.wait_motion_finished() # only needed if blocking is False
r.stop() # only needs to be executed at the end of the program

```

move_pose(pose_name, new_frame_name)

Method to move the input pose(point/frame), which is defined with respect to a parent frame, to a new parent frame. This shall edit the existing pose which retains the local coordinates but with respect to the new parent frame.

Parameters

- **pose_name** (*str*) – The name of the pose to be moved.
- **new_frame_name** (*str*) – The name of the new frame to be set for the input pose.

Raises

- **ValueError** – If the input{pose_name} is not a string.
- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If user frame deletion fails with the given input data.

Returns

True if operation successfull.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.move_pose(pose_name="point_1", new_frame_name="new_frame")

```

move_recorded_path(*args, **kwargs)

To move the robot in a pre-recorded path.

This method takes the following arguments/keyword arguments:

Parameters

- **file_name** (*str*) – File name, if the file is saved from GUI or full path of the custom recorded path file. (required: Yes)

- **is_const_velocity** (*bool*) – True if constant velocity is needed during the motion. (required: No)
- **speed** (*float*) – Linear Speed. (units: m/sec, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "file_name": "<path to file location>",
    "speed": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
result = r.move_recorded_path(**recorded_path_property)
r.stop() # if there are multiple motions than, this needs to be called_
↪ only once at the end of the script
```

move_spline(*args, **kwargs)

To move the robot along a pre-defined set of joint angles with a Spline trajectory.

 Warning:**1. Please use check feasibility feature to know if the motion generated from the provided input data points with joint angles is safe to use or not.**

It is important to ensure that robot does not go into or close to joint limits. If feasibility check fails, DO NOT execute spline motion on the real robot it may crash or break the robot or even cause rapid motions.

2. This motion is not meant to be used with any pair of consecutive points too far away from each other expected to move in short time dt expecting high jump in velocity and acceleration.

It is recommended to give fairly dense points for the trajectory to perform a good and feasible motion.

This method takes the following arguments/keyword arguments:

Parameters

- **motion_id** (*int*) – identifier for motion (units: -, default: 5, required: No)
- **interpolation_type** (*str*) – Type of cubic spline interpolation. Available types: akima and bspline. (required: yes)

- **file_name** (*str*) – File name with full path. The input file should be a JSON file with two KV pairs i.e. “time_stamps” : and “joint_angles” : Please use the same naming convention. The value should be a list for time_stamps and for joint_angles a list of lists(size: dof). (required: Yes)
- **is_approximated** (*bool*) – (Only for BSpline motion) True if the spline path should be approximated, False if the spline should be interpolated i.e. should exactly pass through the recorded points. Parameter only included in B-Spline. (units: N/A, default: True, required: No)

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
move_spline_property = {
    "interpolation_type" : "akima",
    "file_name": "./test_paths/test_path1.json"
}
result = r.move_spline(**move_spline_property)
r.stop() # if there are multiple motions then, this needs to be called
↪ only once at the end of the script
```

```
{
  "time_stamps": [
    0.0,
    1.0,
    .....
  ],
  "joint_angles": [
    [
      0.0,
      0.0,
      0.0,
      0.0,
      0.0,
      0.0
    ],
    [
      0.1,
      0.1,
      0.1,
      0.1,
      0.1,
      0.1
    ],
    .....
  ]
}
```

(continues on next page)

(continued from previous page)

```
    ]
}
```

movelinear_online(*args, **kwargs)

Method to do Servo in Cartesian Space

Parameters

- **servo_target_position** (*list[float]*) – List of Cartesian target positions in meters. Length should be 7, containing [X, Y, Z, qw, qx, qy, qz].
- **servo_target_velocity** (*list[float]*) – List of maximum Cartesian velocities in meters per second. Length should be 6, containing [x_dot, y_dot, z_dot, omega_x, omega_y, omega_z].
- **servo_target_acceleration** (*list[float]*) – List of maximum Cartesian accelerations in meters per second squared. Length should be 6, containing [x_ddot, y_ddot, z_ddot, alpha_x, alpha_y, alpha_z].

Returns

Servo warning and error codes.

Return type

int

Sample Usage:

```
from neurapy.robot import Robot
import time
import copy

r = Robot()

target_1 = 0.3
target_2 = 0.25

#Switch to external servo mode
r.activate_servo_interface('position')
cart_pose_length = 7 # X,Y,Z,qw,qx,qy,qz
target = copy.deepcopy(r.get_tcp_pose())
print(target)

# Move target_1 unit in -X direction
target[0] -= target_1
velocity = [0.15]*6
acceleration = [2.]*6
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion.
time.sleep(5)

target = copy.deepcopy(r.get_tcp_pose())
#Move target_2 units in +Z direction
target[2] += target_2
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion
time.sleep(5)

print("Robot stopped")
```

(continues on next page)

(continued from previous page)

```
r.deactivate_servo_interface()
r.stop()
```

notify_error(*msg*)

To notify an error message to teach pendant .

Parameters

msg (*str*) – The error message to be sent.

Returns

True if successful, else False

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.notify_error("Something went wrong!"):
    print("Error message sent successfully.")
else:
    print("Failed to send error message.")
```

notify_info(*msg*)

To notify a info message to teach pendant.

Parameters

msg (*str*) – The error message to be sent.

Returns

True if successful, else False

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.notify_info("This is an info!"):
    print("Info message sent successfully.")
else:
    print("Failed to send info message.")
```

notify_warning(*msg*)

To notify a warning message to teach pendant.

Parameters

msg (*str*) – The error message to be sent.

Returns

True if successful, else False

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.notify_warning("This is a warning!"):
    print("Warning message sent successfully.")
```

(continues on next page)

(continued from previous page)

```

    print("Warning message sent successfully.")
else:
    print("Failed to send warning message.")

```

pause(*args, **kwargs)

To pause the robot's motion.

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.pause()

```

plan_move_circular(*args, **kwargs)

Plans a motion in a circular path across the given poses.

This method takes the following arguments/keyword arguments:

Parameters

- **target_pose** (*list*) – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **target_joint** (*list*) – List of joint configurations, needs current_joint_angles if not provided. (type: Joint configuration - [], float, units: Joint values at each position, required: No)
- **motion_id** (*int*) – id provided for the motion. (units: -, default: 0, required: No)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 100.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.4, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 0.4, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 100.0, required: No)
- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration, required if target_joint not provided. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)

- **weaving_parameters** (*list*) – List of weaving_parameter. Required when weaving is enabled.
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).

- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

motion_id (equal to motion_id if provided in inputs, else an identifier function has generated to store the plan)

Return type

int

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:**Example 1:**


```

from neurapy.robot import Robot

r = Robot()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "dynamic",
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
        [
            0.5190337951593321,
            -0.1969996948428492,
            0.23267853691809767,
            3.1414194107055664,
            -0.00017726201622281224,
            -0.48750609159469604
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles(),
    "weaving": False,
    "weaving_parameters": [
        {
            "start_target_pose_number": 1,
            "end_target_pose_number": 3,
            "frequency": 0.01,
            "pattern": "sine",
            "amplitude_left": 0.006,
            "amplitude_right": 0.006,
            "dwell_time_left": 0,
            "dwell_time_right": 0,
            "azimuth": 0.0,
            "elevation": 0.0
        }
    ],
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {

```

(continues on next page)

(continued from previous page)

```

        "is_active": False,
        "value": 0
    },
    "Fy": {
        "is_active": False,
        "value": 0
    },
    "Fz": {
        "is_active": False,
        "value": 0
    }
},
"torque_vector": {
    "Mx": {
        "is_active": False,
        "value": 0.0
    },
    "My": {
        "is_active": False,
        "value": 0.0
    },
    "Mz": {
        "is_active": False,
        "value": 0.0
    }
}
}

r.plan_move_circular(**circular_property)
r.executor()
r.stop() # if there are multiple motions than, this needs to be
↪ called only once at the end of the script

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
r.plan_move_circular(["P1", "P2", "P3"])
r.executor()
r.stop()

```

plan_move_composite(*args, **kwargs)

Plans a motion in which linear and circular motions are combined.

This method takes the following arguments/keyword arguments:

Parameters

- **commands** (*list*) – List of linear and circular command combinations. Each element is a dictionary with a single key “linear” or “circular”, and the value is a dictionary containing the command parameters.

Each command dictionary structure:

- linear command: Dictionary with key “linear” and value containing:

- * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes)
Alternative: targets or List of Strings (Type: [String], names of existing points)

- * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- * **velocity** (*float*): Translation Velocity, required when `use_children_velocities` set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
- * **rotation_velocity** (*float*): Rotational Velocity, required when `use_children_velocities` set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)
- **circular command**: Dictionary with key “circular” and value containing:
 - * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
 - * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
 - * **velocity** (*float*): Translation Velocity, required when `use_children_velocities` set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
 - * **rotation_velocity** (*float*): Rotational Velocity, required when `use_children_velocities` set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 500.0, required: No)
- **use_children_velocities** (*bool*) – Use individual speed and rotation speeds for each child motion, as set in commands parameter. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: ‘static’, required: No)
 - ‘dynamic’ : dynamic blending, blending based on velocity and acceleration.
 - ‘static’ : static blending, blending based on the given `blend_radius`.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. (required: No)

- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.

- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

motion_id (equal to motion_id if provided in inputs, else an identifier function has generated to store the plan)

Return type

int

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
```

(continues on next page)

(continued from previous page)

```

composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "dynamic",
    "blend_radius": 0.01,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [
        {
            "linear": {
                "blend_radius": 0.005,
                "target_pose": [
                    [
                        -0.000259845199876027,
                        -0.5211437049195536,
                        0.4429382717719519,
                        3.14123272895813,
                        -0.0007908568368293345,
                        -1.570908784866333
                    ],
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ]
                ]
            },
        },
        {
            "circular": {
                "target_pose": [
                    [
                        -0.16633498440272945,
                        -0.5201452059140722,
                        0.4427486025872017,
                        3.140937089920044,
                        -0.0005319403717294335,
                        -1.571555495262146
                    ],
                    [
                        -0.16540090985202305,
                        -0.3983552679378624,
                        0.44267608017426174,
                        3.1407113075256348,
                        -0.00036628879024647176,
                        -1.5714884996414185
                    ],
                    [
                        -0.33446498807559716,
                        -0.3989652352814891,

```

(continues on next page)

(continued from previous page)

```

0.4421152856242009,
3.1402060985565186,
0.00030071483342908323,
-1.572899580001831
    ]
    ]
    }
    ],
    "motion_id":234,
    "current_joint_angles": r.get_current_joint_angles(),
    "weaving":False,
    "weaving_parameters": [
        {
            "start_target_pose_number": 1,
            "end_target_pose_number": 4,
            "frequency": 0.01,
            "pattern": "sine",
            "amplitude_left": 0.006,
            "amplitude_right": 0.006,
            "dwell_time_left": 0,
            "dwell_time_right": 0,
            "azimuth": 0.0,
            "elevation": 0.0
        }
    ],
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
}
motion_id = r.plan_move_composite(**composite_motion_property,
    ↪motion_id = 2)
execute = r.executor()

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
composite_motion_property = {
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "static",
    "blend_radius": 0.005,
    "current_joint_angles": r.get_current_joint_angles(),
    "use_children_velocities": True,
    "commands": [
        {
            "linear": {
                "target_pose": [
                    [
                        0.7006,
                        0.0001,
                        0.6875,
                        3.1416,
                        0,
                        3.1416
                    ],
                    [
                        0.7006,
                        0.0001,
                        0.6875,
                        3.1416,
                        0,
                        3.1416
                    ],
                    [
                        0.7509102896938852,
                        0.27460943971123764,
                        0.6864972996055627,
                        3.141592653589793,
                        -7.71435533383267e-06,
                        -3.1415926535897882
                    ],
                    [
                        0.7747260445464177,
                        0.15721988912449852,
                        0.8931631506500376,
                        -3.1415926535897745,
                        -7.714355718231237e-06,
                        -3.1415926535897354
                    ]
                ]
            }
        ]
    ]
}

```

(continues on next page)

(continued from previous page)

```

        [
            0.7006,
            0.0001,
            0.6875,
            3.1416,
            0,
            3.1416
        ],
        ],
        "velocity": 0.1,
        "rotation_velocity": 0.174533
    },
    {
        "circular": {
            "target_pose": [
                [
                    0.7006,
                    0.0001,
                    0.6875,
                    3.1416,
                    0,
                    3.1416
                ],
                [
                    0.6999986609380567,
                    -0.25149522267751856,
                    0.6864972999318248,
                    -3.141592653589793,
                    -7.71435539238761e-06,
                    3.1415926535897905
                ],
                [
                    0.7747260445464177,
                    0.15721988912449852,
                    0.8931631506500376,
                    -3.1415926535897745,
                    -7.714355718231237e-06,
                    -3.1415926535897354
                ]
            ],
            "velocity": 0.4,
            "rotation_velocity": 0.436332
        }
    },
    ],
    "weaving": False
}
motion_id = r.plan_move_composite(**composite_motion_property)
execute = r.executor()

```

plan_move_joint(*args, **kwargs)

This method takes the following arguments/keyword arguments:

Parameters

- **target_joint** (*list*) – List of joint configurations. (type: List of Joint Values - float, units: radians, required: Yes)

- **target_pose** – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: No)

Alternative: List of Strings (Type: [String], names of existing points). If not provided, the poses will be automatically generated from the current_joint_angles. :type target_pose: list :param speed: Angular Speed. (units: % of maximum angular speed, scale: 0 to 100, default: 25%, required: No) :type speed: float :param acceleration: Angular Acceleration. (units: % of maximum angular acceleration, scale: 0 to 100, default: 25%, required: No) :type acceleration: float :param jerk: Angular Jerk. (units: % of maximum angular jerk, default: 100.0, required: No) :type jerk: float :param current_joint_angles: Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No) :type current_joint_angles: list

Parameters

- **enable_blending** (*bool*) – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **motion_id** (*int*) – identifier to the stored plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.

- `controller_parameters["torque_vector"]["Mx"]` (*Dict[str, Any]*) – Torque control setting around the X-axis.
- `controller_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `controller_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `controller_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `controller_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

`plan_id` (equal to `motion_id` if provided in inputs, else an identifier function has generated to store the plan)

Return type

`int`

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot
r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_blending": True,
    "target_joint": [
        [
            2.5995838308821924,
            0.24962416292345468,
            -1.8654403327490414,
            0.04503286318691005,
            -1.1740563715454926,
            0.10337461241185522
        ],
        [
            2.1372059994827075,
            0.24939733788589463,
```

(continues on next page)

(continued from previous page)

```

        -1.8651270179353125,
        0.044771940725327274,
        -1.173860821592129,
        0.10315646291502645
    ],
    [
        1.9180047887810003,
        -0.24855170101601043,
        -1.3680228668892351,
        0.12404421791100637,
        -1.1914147150222498,
        -0.13255713717112075
    ]
],
"motion_id":234,
"current_joint_angles": r.get_current_joint_angles(),
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}
}
motion_id = r.plan_move_joint(**joint_property)
execute = r.executor()

```

plan_move_linear(*args, **kwargs)

To plan a move linear path across the given poses.

This method takes the following arguments/keyword arguments:

Parameters

- **target_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec², default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec³, default: 500.0, required: No)
- **enable_blending** – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. (required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)

- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go_till_force_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.

- `go_till_force_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

`motion_id` (equal to `motion_id` if provided in inputs, else an identifier function has generated to store the plan)

Return type

`int`

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "enable_blending": True,
    "blending_mode": "static",
    "blend_radius": 0.01,
    "target_pose": [
        [
            0.3287228886,
            -0.1903355329,
            0.4220780352,
            0.08535207028439847,
            -2.797181496822229,
            2.4713321627410485
        ]
    ],
```

(continues on next page)

(continued from previous page)

```

[
    0.2093363791501374,
    -0.31711250784165884,
    0.422149168855134,
    -3.0565555095672607,
    -0.3447442352771759,
    -1.1323236227035522
],
[
    0.2090521916195534,
    -0.5246753336643587,
    0.4218773613553828,
    -3.0569007396698,
    -0.3448921740055084,
    -1.1323626041412354
],
[
    0.3287228886,
    -0.1903355329,
    0.4220780352,
    0.08535207028439847,
    -2.797181496822229,
    2.4713321627410485
]
],
"motion_id":234,
"current_joint_angles": r.get_current_joint_angles(),
"weaving":False,
"weaving_parameters": [
    {
        "start_target_pose_number": 1,
        "end_target_pose_number": 4,
        "frequency": 0.01,
        "pattern": "sine",
        "amplitude_left": 0.006,
        "amplitude_right": 0.006,
        "dwell_time_left": 0,
        "dwell_time_right": 0,
        "azimuth": 0.0,
        "elevation": 0.0
    }
],
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": False,
            "value": 0
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    },
    "torque_vector": {
        "Mx": {
            "is_active": False,
            "value": 0.0
        },
        "My": {
            "is_active": False,
            "value": 0.0
        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
},
"go_till_force_parameters": {
    "is_enabled": True,
    "enable_reflex": True,
    "extra_search_offset": 0,
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": True,
            "value": 10
        }
    }
},
"torque_vector": {
    "Mx": {
        "is_active": False,
        "value": 0.0
    },
    "My": {
        "is_active": False,
        "value": 0.0
    },
    "Mz": {
        "is_active": False,
        "value": 0.0
    }
}
}

}

motion_id = r.plan_move_linear(**linear_property)
execute_motion = r.executor()

```

plan_move_recorded_path(*args, **kwargs)

To plan the given pre-recorded path.

This method takes the following arguments/keyword arguments:

Parameters

- **file_name** (*str*) – File name, if the file is saved from GUI or full path of the custom recorded path file. (required: Yes)
- **is_const_velocity** (*bool*) – True if constant velocity is needed during the motion. (required: No)
- **speed** (*float*) – Linear Speed. (units: m/sec, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **motion_id** (*int*) – identifier to store the plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **reusable** (*bool*) – to reuse the identifier for repeated motion. (type: Bool - True/False, default_value: False)

Returns

plan_id (equal to motion_id if provided in inputs, else an identifier function has generated to store the plan)

Return type

int

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "file_name": "./test_paths/test_path1.txt",
    "speed": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
motion_id = r.plan_move_recorded_path(**recorded_path_property)
execute_motion = r.executor()
r.stop()
```

plan_move_spline(*args, **kwargs)

To plan the given pre-defined path.

This method takes the following arguments/keyword arguments:

Parameters

- **motion_id** (*int*) – identifier for motion (units: -, default: 5, required: No)
- **interpolation_type** (*str*) – Type of cubic spline interpolation. Available types: akima and bspline.(required: yes)
- **file_name** (*str*) – File name with full path. The input file should be a JSON file with two KV pairs i.e. “time_stamps” : ... and “joint_angles” : ... Please use the same naming convention. The value should be a list for time_stamps and for joint_angles a list of lists(size: dof). (required: Yes)

- **is_approximated** (*bool*) – (Only for BSpline motion) True if the spline path should be approximated, False if the spline should be interpolated i.e. should exactly pass through the recorded points. Parameter only included in B-Spline. (units: N/A, default: True, required: No)

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
move_spline_property = {
    "interpolation_type" : "akima"
    "file_name": "./test_paths/test_path1.json",
}
plan_id = r.plan_move_spline(**move_spline_property)
execute_motion = r.executor() #To execute the planned id
r.stop()
```

plan_realtime_wait(*args, **kwargs)

Adds a wait command which is added to the execution pipeline. With this delays are minimized and should be closer than python waits. This is at this time planned but not added to the execution.

Parameters

- **motion_id** (*int*) – The id which the motion should get, this is needed for gui feedback. (units: N/A, default: 0, required: No)
- **blocking** (*bool*) – Block execution until wait is completed, if enabled. (units: N/A, default: True, required: No)
- **time_in_s** (*float*) – The time which the wait command should take. (units: sec, default: 0.0, required: No)

Sample Usage:

Example 1:

```
from neurapy.robot import Robot

r = Robot()
wait_property = {
    'motion_id': 1,
    'time_in_s': 0.010,
}
r.plan_realtime_wait(**wait_property)
r.executor() # this is blocking as long as not given a non_
↳ blocking command.
r.stop() # if there are multiple motions than, this needs to be_
↳ called only once at the end of the script
```

power_off()

To power off the robot.

Returns

True if the robot's power is successfully turned off, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.power_off():
    print("Robot powered off successfully.")
else:
    print("Failed to power off the robot.")
```

power_on()

To power on the robot.

Returns

True if the robot's power is successfully turned on, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.power_on():
    print("Robot powered on successfully.")
else:
    print("Failed to power on the robot.")
```

pre_plan_move_circular(*args, **kwargs)

Pre planning of motion, this allows to plan the motion without adding it for execution!

Parameters

- **target_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 500.0, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, required if static blending is selected. (units: m, default: None, required: No)

- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **motion_id** (*int*) – identifier to the stored plan. (if not provided a random will be assigned, which will be returned after the function execution)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.

- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

motion_id (equal to motion_id if provided in inputs, else an identifier function has generated to store the plan)

Return type

int

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "static",
    "blend_radius": 0.05,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
        [
            0.5190337951593321,
            -0.1969996948428492,
            0.23267853691809767,
            3.1414194107055664,
            -0.00017726201622281224,
            -0.48750609159469604
        ]
    ],
    "motion_id": 234,
    "current_joint_angles": r.get_current_joint_angles(),
    "weaving": False,
    "pattern": 1,
    "amplitude_left": 0.003,
    "amplitude_right": 0.003,
    "frequency": 1.5,
    "dwell_time_left": 0.0,
    "dwell_time_right": 0.0,
    "elevation": 0.0,
    "azimuth": 0.0,
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,

```

(continues on next page)

(continued from previous page)

```

        "value": 0
    },
    "Fy": {
        "is_active": False,
        "value": 0
    },
    "Fz": {
        "is_active": False,
        "value": 0
    }
},
"torque_vector": {
    "Mx": {
        "is_active": False,
        "value": 0.0
    },
    "My": {
        "is_active": False,
        "value": 0.0
    },
    "Mz": {
        "is_active": False,
        "value": 0.0
    }
}
}
}
plan_id = r.pre_plan_move_circular(**circular_property)
execute = r.execute_pre_plan_motion(plan_id)

```

pre_plan_move_composite(*args, **kwargs)

This function allows to pre plan a motion, without adding it to the list for execution. Allows to pre plan bigger motions and execute them at a later time.

This method takes the following arguments/keyword arguments:

Parameters

- **commands** (*list*) – List of linear and circular command combinations. Each element is a dictionary with a single key “linear” or “circular”, and the value is a dictionary containing the command parameters.

Each command dictionary structure:

– linear command: Dictionary with key “linear” and value containing:

- * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
- * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
- * **velocity** (*float*): Translation Velocity, required when use_children_velocities set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
- * **rotation_velocity** (*float*): Rotational Velocity, required when use_children_velocities set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)

- circular command: Dictionary with key “circular” and value containing:
 - * **target_pose** (*list*): List of target poses. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: targets or List of Strings (Type: [String], names of existing points)
 - * **target_joint** (*list*): List of target joints. (type: Joint configuration, float, units: joint angles in rad, required: No) Alternative: List of Strings (Type: [String], names of existing points)
 - * **velocity** (*float*): Translation Velocity, required when use_children_velocities set to True. (units: m/sec, default: speed set for entire MoveComposite, required: No)
 - * **rotation_velocity** (*float*): Rotational Velocity, required when use_children_velocities set to True. (units: rad/sec, default: rotation_speed set for entire MoveComposite, required: No)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec², default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec³, default: 500.0, required: No)
- **use_children_velocities** (*bool*) – Use individual speed and rotation speeds for each child motion, as set in commands parameter. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: ‘static’, required: No)
 - ‘dynamic’ : dynamic blending, blending based on velocity and acceleration.
 - ‘static’ : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)
- **weaving_parameters** (*list*) – List of weaving_parameter. (required: No)
- **weaving_parameter** (*Dict[str, Any]*) – Dict of weaving values. Required when weaving is enabled.
- **weaving_parameter["pattern"]** (*str*) – Weaving pattern for the segment. Required when weaving is enabled.
 - sine : SINE, if selected uses a sine wave, allows configuration with sine values
 - circle : CIRCLE, if selected uses circle wave, allows configuration with circle values
- **weaving_parameter["start_target_pose_number"]** (*int*) – point at which weaving should be started (count starts from 1). Required when weaving is enabled.

- **weaving_parameter["end_target_pose_number"]** (*int*) – point at which weaving should end (count starts from 1). Required when weaving is enabled.
- **weaving_parameter["frequency"]** (*float*) – Frequency for the weaving. Required when weaving is enabled. (units: Hz)
- **weaving_parameter["elevation"]** (*float*) – Rotation around the velocity vector. Required when weaving is enabled. (units: rad)
- **weaving_parameter["azimuth"]** (*float*) – Rotation around the z-Axis TCP. Required when weaving is enabled. (units: rad)
- **weaving_parameter["amplitude_left"]** (*float*) – amplitude on left side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["amplitude_right"]** (*float*) – amplitude on right side, defined from motion direction, sine value. Required when weaving is enabled. (units: m)
- **weaving_parameter["dwell_time_left"]** (*float*) – time to wait at left peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["dwell_time_right"]** (*float*) – time to wait at right peak, sine value. Required when weaving is enabled. (units: s)
- **weaving_parameter["radius"]** (*float*) – radius for the circle weaving, circle value. Required when weaving is enabled. (units: s)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.

- `controller_parameters["force_vector"]["Fz"]["value"]` (*float*) – Desired force value in Newtons (N).
- `controller_parameters["torque_vector"]` (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- `controller_parameters["torque_vector"]["Mx"]` (*Dict[str, Any]*) – Torque control setting around the X-axis.
- `controller_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `controller_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `controller_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `controller_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `controller_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `controller_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

`plan_id` (equal to `motion_id` if provided in inputs, else an identifier function has generated to store the plan)

Return type

`int`

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "jerk": 100,
    "rotation_speed": 1.57,
    "rotation_acceleration": 5.0,
    "rotation_jerk": 100,
    "blending_mode": "dynamic",
    "blend_radius": 0.01,
```

(continues on next page)

(continued from previous page)

```

"current_joint_angles": r.get_current_joint_angles(),
"commands": [
    {
        "linear": {
            "blend_radius": 0.005,
            "target_pose": [
                [
                    -0.000259845199876027,
                    -0.5211437049195536,
                    0.4429382717719519,
                    3.14123272895813,
                    -0.0007908568368293345,
                    -1.570908784866333
                ],
                [
                    -0.16633498440272945,
                    -0.5201452059140722,
                    0.4427486025872017,
                    3.140937089920044,
                    -0.0005319403717294335,
                    -1.571555495262146
                ]
            ]
        },
    },
    {
        "circular": {
            "target_pose": [
                [
                    -0.16633498440272945,
                    -0.5201452059140722,
                    0.4427486025872017,
                    3.140937089920044,
                    -0.0005319403717294335,
                    -1.571555495262146
                ],
                [
                    -0.16540090985202305,
                    -0.3983552679378624,
                    0.44267608017426174,
                    3.1407113075256348,
                    -0.00036628879024647176,
                    -1.5714884996414185
                ],
                [
                    -0.33446498807559716,
                    -0.3989652352814891,
                    0.4421152856242009,
                    3.1402060985565186,
                    0.00030071483342908323,
                    -1.572899580001831
                ]
            ]
        },
    },
],

```

(continues on next page)

(continued from previous page)

```

    "motion_id":234,
    "current_joint_angles": r.get_current_joint_angles(),
    "weaving":False,
    "pattern": 1,
    "amplitude_left": 0.003,
    "amplitude_right": 0.003,
    "frequency": 1.5,
    "dwell_time_left": 0.0,
    "dwell_time_right": 0.0,
    "elevation": 0.0,
    "azimuth": 0.0,
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}
plan_id= r.pre_plan_move_composite(**composite_motion_property,
    ↪motion_id = 1)
execute = r.execute_pre_plan_motion(plan_id)

```

Example 2:

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
composite_motion_property = {
    "acceleration": 0.1,
    "jerk": 100,

```

(continues on next page)

(continued from previous page)

```

"rotation_acceleration": 5.0,
"rotation_jerk": 100,
"blending_mode": "static",
"blend_radius": 0.005,
"current_joint_angles": r.get_current_joint_angles(),
"use_children_velocities": True,
"commands": [
    {
        "linear": {
            "target_pose": [
                [
                    0.7006,
                    0.0001,
                    0.6875,
                    3.1416,
                    0,
                    3.1416
                ],
                [
                    0.7006,
                    0.0001,
                    0.6875,
                    3.1416,
                    0,
                    3.1416
                ],
                [
                    0.7509102896938852,
                    0.27460943971123764,
                    0.6864972996055627,
                    3.141592653589793,
                    -7.71435533383267e-06,
                    -3.1415926535897882
                ],
                [
                    0.7747260445464177,
                    0.15721988912449852,
                    0.8931631506500376,
                    -3.1415926535897745,
                    -7.714355718231237e-06,
                    -3.1415926535897354
                ],
                [
                    0.7006,
                    0.0001,
                    0.6875,
                    3.1416,
                    0,
                    3.1416
                ]
            ],
            "velocity": 0.1,
            "rotation_velocity": 0.174533
        }
    },
    {

```

(continues on next page)

(continued from previous page)

```

        "circular": {
            "target_pose": [
                [
                    0.7006,
                    0.0001,
                    0.6875,
                    3.1416,
                    0,
                    3.1416
                ],
                [
                    0.6999986609380567,
                    -0.25149522267751856,
                    0.6864972999318248,
                    -3.141592653589793,
                    -7.71435539238761e-06,
                    3.1415926535897905
                ],
                [
                    0.7747260445464177,
                    0.15721988912449852,
                    0.8931631506500376,
                    -3.1415926535897745,
                    -7.714355718231237e-06,
                    -3.1415926535897354
                ]
            ],
            "velocity": 0.4,
            "rotation_velocity": 0.436332
        }
    ],
    "weaving": False
}
plan_id = r.pre_plan_move_composite(**composite_motion_property,
    ↪ motion_id = 2)
execute = r.execute_pre_plan_motion(plan_id)

```

pre_plan_move_joint(*args, **kwargs)

Allows to pre plan the motion and execute it at a later time.

This method takes the following arguments/keyword arguments:

Parameters

- **target_joint** (*list*) – List of joint configurations. (type: List of Joint Values - float, units: radians, required: Yes)
- **target_pose** – List of pose configurations. (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: No)

Alternative: List of Strings (Type: [String], names of existing points). If not provided, the poses will be automatically generated from the current_joint_angles. :type target_pose: list :param speed: Angular Speed. (units: % of maximum angular speed, scale: 0 to 100, default: 25%, required: No) :type speed: float :param acceleration: Angular Acceleration. (units: % of maximum angular acceleration, scale: 0 to 100, default: 25%, required: No) :type acceleration: float :param jerk: Angular Jerk. (units: % of maximum angular jerk, default: 100.0, required: No) :type jerk: float :param current_joint_angles: Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Con-

figuration obtained from Robot Status method, required: No) :type current_joint_angles: list :param enable_blending: Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No) :type enable_blending: bool :param motion_id: identifier to the stored plan. (if not provided a random will be assigned, which will be returned after the function execution) :type motion_id: int

Parameters

- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.

- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

plan_id (equal to motion_id if provided in inputs, else an identifier function has generated to store the plan)

Return type

int

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot
r = Robot()
r.init_program()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "enable_blending": True,
    "target_joint": [
        [
            2.5995838308821924,
            0.24962416292345468,
            -1.8654403327490414,
            0.04503286318691005,
            -1.1740563715454926,
            0.10337461241185522
        ],
        [
            2.1372059994827075,
            0.24939733788589463,
            -1.8651270179353125,
            0.044771940725327274,
            -1.173860821592129,
            0.10315646291502645
        ],
        [
            1.9180047887810003,
            -0.24855170101601043,
            -1.3680228668892351,
            0.12404421791100637,
            -1.1914147150222498,
            -0.13255713717112075
        ]
    ]
}
```

(continues on next page)

(continued from previous page)

```

    ]
    ],
    "motion_id": 234,
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

plan_id = r.pre_plan_move_joint(**joint_property, motion_id = 1)
execute = r.execute_pre_plan_motion(plan_id)

```

pre_plan_move_joint_relative(*args, **kwargs)

Allows to pre plan the motion and execute it at a later time.

Parameters

- **speed** (*float*) – Angular Speed. (units: % of maximum angular speed, default: 25, required: No)
- **acceleration** (*float*) – Angular Acceleration. (units: % of maximum angular acceleration, default: 25, required: No)
- **jerk** (*float*) – Angular Jerk. (units: % of maximum angular jerk, default: 25, required: No)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)

- **offset_frame** (*str*) – Frame for applying relative offset. (units: N/A, default: “Base”, required: No)
 - ‘Base’ : Applies relative offset to the base frame
 - ‘Tool’ : Applies relative offset to the tool frame
- **cartesian_offset** (*list*) – The cartesian offset that is applied. (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m - [R,P,Y] in radian, default: [0.0,0.0,0.0,0.0,0.0,0.0], required: Yes)
- **target_joint** (*list*) – The joint configuration at which the relative motion should start from (type: list - [a1, a2, a3, a4, a5, a6] for lara and [a1, a2, a3, a4, a5, a6, a7] for maira, units: axis values in radian, default: current joint angles, required: No) If the target_joint is used, the target_pose also needs to be configured, else the current joint angles and current pose are used
- **target_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No) If the target_pose is used, the target_joint also needs to be configured, else the current joint angle and current pose are used
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - ‘position’ : Position control mode.
 - ‘joint_impedance’ : Joint impedance control mode.
 - ‘hybrid’ : Hybrid Motion/Force control mode.
 - ‘admittance’ : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.

- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.

- `go_till_force_parameters["force_vector"]["Fz"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["torque_vector"]` (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- `go_till_force_parameters["torque_vector"]["Mx"]` (*Dict[str, Any]*) – Torque control setting around the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()
r.init_program()
joint_property = {
    "speed" : 50.0,
    "acceleration" : 50,
    "jerk": 50,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi / 4],
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
```

(continues on next page)

(continued from previous page)

```

"control_mode": "position",
"force_vector": {
  "Fx": {
    "is_active": False,
    "value": 0
  },
  "Fy": {
    "is_active": False,
    "value": 0
  },
  "Fz": {
    "is_active": False,
    "value": 0
  }
},
"torque_vector": {
  "Mx": {
    "is_active": False,
    "value": 0.0
  },
  "My": {
    "is_active": False,
    "value": 0.0
  },
  "Mz": {
    "is_active": False,
    "value": 0.0
  }
},
},
"go_till_force_parameters": {
  "is_enabled": True,
  "enable_reflex": True,
  "extra_search_offset": 0,
  "force_vector": {
    "Fx": {
      "is_active": False,
      "value": 0
    },
    "Fy": {
      "is_active": False,
      "value": 0
    },
    "Fz": {
      "is_active": True,
      "value": 10
    }
  },
  "torque_vector": {
    "Mx": {
      "is_active": False,
      "value": 0.0
    },
    "My": {
      "is_active": False,
      "value": 0.0
    }
  }
}

```

(continues on next page)

(continued from previous page)

```

        },
        "Mz": {
            "is_active": False,
            "value": 0.0
        }
    }
}

motion_id = r.pre_plan_move_joint_relative(**joint_property, motion_
↪ id = 1)
r.execute_pre_plan_motion(motion_id)
r.wait_motion_finished() # only needed if blocking is False
r.stop() # if there are multiple motions than, this needs to be called_
↪ only once at the end of the script

```

pre_plan_move_linear(*args, **kwargs)

Allows to pre plan the motion and execute it at a later time.

This method takes the following arguments/keyword arguments:

Parameters

- **target_pose** (*list*) – List of 3 pose configurations (starting, middle, and end points). (type: Pose configuration - [X,Y,Z,R,P,Y], float, units: Position values in meters and rotation values in radians, required: Yes) Alternative: List of Strings (Type: [String], names of existing points)
- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec², default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec³, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec², default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec³, default: 500.0, required: No)
- **enable_blending** – Enable blending mode, allowing motion to continue without stopping at every point. (units: N/A, default: False, required: No)
- **blending_mode** (*str*) – The blending type that is selected to blend between points. (units: N/A, default: 'dynamic', required: No)
 - 'dynamic' : dynamic blending, blending based on velocity and acceleration.
 - 'static' : static blending, blending based on the given blend_radius.
- **blend_radius** (*float*) – Blend Radius, if static blending is selected. (units: m, default: 0.01, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **weaving** (*bool*) – Enable weaving motion pattern. (units: N/A, default: False, required: No)

- **pattern** (*int*) – The Pattern which should be used for the Weaving (units: N/A, default: 2, required: No)
 - 1 : SINE, if selected uses a Sine wave, ignores dwell times
 - 2 : TRAPEZOIDAL, if selected uses a trapezoidal wave, implements dwell times
 - 3 : CIRCLE (experimental), if selected uses a circle wave, ignores dwell time and amplitude offset
- **amplitude_left** (*float*) – The amplitude on the left side (defined in motion direction) (units: m, default: 0.0, required: No)
- **amplitude_right** (*float*) – The amplitude on the right side (defined in motion direction) (units: m, default: 0.0, required: No)
- **frequency** (*float*) – the frequency for the wave form (units: Hz, default: 10.0, required: No)
- **dwell_time_left** (*float*) – the wait time, for trapezoidal waves at each peak (left defined by motion direction) (units: s, default: 0.0, required: No)
- **dwell_time_right** (*float*) – the wait time, for trapezoidal waves at each peak (right defined by motion direction) (units: s, default: 0.0, required: No)
- **elevation** (*float*) – the rotation around the velocity vector (units: rad, default: 0.0, required: No)
- **azimuth** (*float*) – the rotation around the z-Axis of the tool (units: rad, default: 0.0, required: No)
- **motion_id** (*int*) – identifier to store the plan. (if not provided this will return zero)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.

- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.
- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.

- `go_till_force_parameters["force_vector"]["Fy"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["force_vector"]["Fz"]` (*Dict[str, Any]*) – Force control setting in the Z direction.
- `go_till_force_parameters["force_vector"]["Fz"]["is_active"]` (*bool*) – Whether force control is active for the Z direction.
- `go_till_force_parameters["force_vector"]["Fz"]["value"]` (*float*) – Desired force value in Newtons (N).
- `go_till_force_parameters["torque_vector"]` (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- `go_till_force_parameters["torque_vector"]["Mx"]` (*Dict[str, Any]*) – Torque control setting around the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["is_active"]` (*bool*) – Whether torque control is active for the X-axis.
- `go_till_force_parameters["torque_vector"]["Mx"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["My"]` (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["is_active"]` (*bool*) – Whether torque control is active for the Y-axis.
- `go_till_force_parameters["torque_vector"]["My"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).
- `go_till_force_parameters["torque_vector"]["Mz"]` (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["is_active"]` (*bool*) – Whether torque control is active for the Z-axis.
- `go_till_force_parameters["torque_vector"]["Mz"]["value"]` (*float*) – Desired torque value in Newton-meters (Nm).

Returns

`plan_id` (equal to `motion_id` if provided in inputs, else an identifier function has generated to store the plan)

Return type

`int`

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
```

(continues on next page)

(continued from previous page)

```

"jerk": 100,
"rotation_speed": 1.57,
"rotation_acceleration": 5.0,
"rotation_jerk": 100,
"enable_blending": True,
"blending_mode": "static",
"blend_radius": 0.01,
"target_pose": [
    [
        0.3287228886,
        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ],
    [
        0.2093363791501374,
        -0.31711250784165884,
        0.422149168855134,
        -3.0565555095672607,
        -0.3447442352771759,
        -1.1323236227035522
    ],
    [
        0.2090521916195534,
        -0.5246753336643587,
        0.4218773613553828,
        -3.0569007396698,
        -0.3448921740055084,
        -1.1323626041412354
    ],
    [
        0.3287228886,
        -0.1903355329,
        0.4220780352,
        0.08535207028439847,
        -2.797181496822229,
        2.4713321627410485
    ]
],
"motion_id": 234,
"current_joint_angles": r.get_current_joint_angles(),
"weaving": False,
"pattern": 1,
"amplitude_left": 0.003,
"amplitude_right": 0.003,
"frequency": 1.5,
"dwell_time_left": 0.0,
"dwell_time_right": 0.0,
"elevation": 0.0,
"azimuth": 0.0,
"controller_parameters": {
    "control_mode": "position",
    "force_vector": {
        "Fx": {

```

(continues on next page)

(continued from previous page)

```

        "is_active": False,
        "value": 0
    },
    "Fy": {
        "is_active": False,
        "value": 0
    },
    "Fz": {
        "is_active": False,
        "value": 0
    }
},
"torque_vector": {
    "Mx": {
        "is_active": False,
        "value": 0.0
    },
    "My": {
        "is_active": False,
        "value": 0.0
    },
    "Mz": {
        "is_active": False,
        "value": 0.0
    }
},
},
"go_till_force_parameters": {
    "is_enabled": True,
    "enable_reflex": True,
    "extra_search_offset": 0,
    "force_vector": {
        "Fx": {
            "is_active": False,
            "value": 0
        },
        "Fy": {
            "is_active": False,
            "value": 0
        },
        "Fz": {
            "is_active": True,
            "value": 10
        }
    }
},
"torque_vector": {
    "Mx": {
        "is_active": False,
        "value": 0.0
    },
    "My": {
        "is_active": False,
        "value": 0.0
    },
    "Mz": {
        "is_active": False,

```

(continues on next page)

(continued from previous page)

```

        "value": 0.0
    }
}
}
}
plan_id = r.pre_plan_move_linear(**linear_property, motion_id = 1)
execute_motion = r.execute_pre_plan_motion(plan_id) #To execute the
↳planned id

```

pre_plan_move_linear_relative(*args, **kwargs)

Allows to pre plan the motion and execute it at a later time.

Parameters

- **speed** (*float*) – Translation Speed. (units: m/sec, default: 0.25, required: No)
- **acceleration** (*float*) – Translation Acceleration. (units: m/sec2, default: 0.25, required: No)
- **jerk** (*float*) – Translation Jerk (units: m/sec3, default: 500.0, required: No)
- **rotation_speed** (*float*) – Rotational Speed. (units: rad/sec, default: 0.5, required: No)
- **rotation_acceleration** (*float*) – Rotational Acceleration. (units: rad/sec2, default: 1.57, required: No)
- **rotation_jerk** (*float*) – Rotational Jerk. (units: rad/sec3, default: 500.0, required: No)
- **blocking** (*bool*) – Block execution until motion is completed, if enabled. (units: N/A, default: True, required: No)
- **cartesian_offset** (*list*) – (type: offset list - [X,Y,Z,R,P,Y], units: [X,Y,Z] offset in m [R,P,Y] offset in radian, default: [0.,0.,0.,0.,0.,0], required: Yes)
- **offset_frame** (*str*) – Frame for applying relative offset. (units: N/A, default: "Base", required: No)
 - 'Base' : Applies relative offset to the base frame
 - 'Tool' : Applies relative offset to the tool frame
- **target_pose** (*list*) – The pose at which the relative motion should start from (type: list - [X,Y,Z,R,P,Y], units: [X,Y,Z] in m and [R,P,Y] in radian, default: current pose, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)
- **controller_parameters** (*Dict[str, Any]*) – Dictionary defining the control mode and force/torque vectors.
- **controller_parameters["control_mode"]** (*str*) – Control mode used to execute the motion. Options include:
 - 'position' : Position control mode.
 - 'joint_impedance' : Joint impedance control mode.
 - 'hybrid' : Hybrid Motion/Force control mode.
 - 'admittance' : Admittance control mode.
- **controller_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.

- **controller_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **controller_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **controller_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **controller_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **controller_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **controller_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **controller_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **controller_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **controller_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **controller_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **controller_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **controller_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **controller_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **controller_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters** (*Dict[str, Any]*) – Dictionary defining go till force motion parameters.
- **go_till_force_parameters["is_enabled"]** (*bool*) – Whether go till force motion is enabled.
- **go_till_force_parameters["enable_reflex"]** (*bool*) – Whether reflex mode is enabled after go till forces are reached.
- **go_till_force_parameters["extra_search_offset"]** (*float*) – Adds an extra search offset to the last target pose for go-till-force motion. The offset is applied in

the direction of motion, computed from the last two points, affecting only x, y, and z while keeping the orientation unchanged.

- **go_till_force_parameters["force_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian force control settings.
- **go_till_force_parameters["force_vector"]["Fx"]** (*Dict[str, Any]*) – Force control setting in the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["is_active"]** (*bool*) – Whether force control is active for the X direction.
- **go_till_force_parameters["force_vector"]["Fx"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fy"]** (*Dict[str, Any]*) – Force control setting in the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["is_active"]** (*bool*) – Whether force control is active for the Y direction.
- **go_till_force_parameters["force_vector"]["Fy"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["force_vector"]["Fz"]** (*Dict[str, Any]*) – Force control setting in the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["is_active"]** (*bool*) – Whether force control is active for the Z direction.
- **go_till_force_parameters["force_vector"]["Fz"]["value"]** (*float*) – Desired force value in Newtons (N).
- **go_till_force_parameters["torque_vector"]** (*Dict[str, Dict[str, Any]]*) – Dictionary containing Cartesian torque control settings.
- **go_till_force_parameters["torque_vector"]["Mx"]** (*Dict[str, Any]*) – Torque control setting around the X-axis.
- **go_till_force_parameters["torque_vector"]["Mx"]["is_active"]** (*bool*) – Whether torque control is active for the X-axis.
- **go_till_force_parameters["torque_vector"]["Mx"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters["torque_vector"]["My"]** (*Dict[str, Any]*) – Torque control setting around the Y-axis.
- **go_till_force_parameters["torque_vector"]["My"]["is_active"]** (*bool*) – Whether torque control is active for the Y-axis.
- **go_till_force_parameters["torque_vector"]["My"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).
- **go_till_force_parameters["torque_vector"]["Mz"]** (*Dict[str, Any]*) – Torque control setting around the Z-axis.
- **go_till_force_parameters["torque_vector"]["Mz"]["is_active"]** (*bool*) – Whether torque control is active for the Z-axis.
- **go_till_force_parameters["torque_vector"]["Mz"]["value"]** (*float*) – Desired torque value in Newton-meters (Nm).

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
import math
from neurapy.robot import Robot

r = Robot()
r.init_program()

linear_property = {
    "speed" : 4.5,
    "acceleration" : 10.0,
    "jerk": 1000.0,
    "rotation_speed" : 1.5,
    "rotation_acceleration" : 10.0,
    "rotation_jerk" : 1000.0,
    "blocking": True,
    "offset_frame" : "Base",
    "cartesian_offset" : [0.0, 0.0, 0.1, 0.0, 0.0, math.pi/4],
    "current_joint_angles": r.get_current_joint_angles(),
    "controller_parameters": {
        "control_mode": "position",
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": False,
                "value": 0
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
},
```

(continues on next page)

(continued from previous page)

```

    "go_till_force_parameters": {
        "is_enabled": True,
        "enable_reflex": True,
        "extra_search_offset": 0,
        "force_vector": {
            "Fx": {
                "is_active": False,
                "value": 0
            },
            "Fy": {
                "is_active": False,
                "value": 0
            },
            "Fz": {
                "is_active": True,
                "value": 10
            }
        },
        "torque_vector": {
            "Mx": {
                "is_active": False,
                "value": 0.0
            },
            "My": {
                "is_active": False,
                "value": 0.0
            },
            "Mz": {
                "is_active": False,
                "value": 0.0
            }
        }
    }
}

motion_id = r.pre_plan_move_linear_relative(**linear_property, motion_id_
↪= 1)
r.execute_pre_plan_motion(motion_id)
r.wait_motion_finished() # only needed if blocking is False
r.stop() # only needs to be executed at the end of the program

```

pre_plan_move_recorded_path(*args, **kwargs)

Allows to pre plan the motion and adds it at a later time.

This method takes the following arguments/keyword arguments:

Parameters

- **file_name** (*str*) – File name, if the file is saved from GUI or full path of the custom recorded path file. (required: Yes)
- **is_const_velocity** (*bool*) – True if constant velocity is needed during the motion. (required: No)
- **speed** (*float*) – Linear Speed. (units: m/sec, required: No)
- **current_joint_angles** (*list*) – Current Robot Joint Configuration. (type: List of Joint Values - float, units: radians, default: Joint Configuration obtained from Robot Status method, required: No)

Returns

True if motion is executed successfully, False if motion is not executed successfully

Return type

bool

Raises

- **WrongMode** – If the robot is in Teach mode while executing this function.
- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible due to invalid motion parameters, excluding path or position constraints.
- **InterruptedError** – If there is any interruption during the execution of the motion.

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
r.init_program()
recorded_path_property = {
    "file_name": "./test_paths/test_path1.txt",
    "velocity": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
motion_id = r.pre_plan_move_recorded_path(**recorded_path_property,
↳ motion_id = 1)
r.execute_pre_plan_motion(motion_id)
r.stop() # if there are multiple motions than, this needs to be called,
↳ only once at the end of the script
```

pre_plan_move_spline(*args, **kwargs)

To pre plan the given pre-defined path.

this method takes the following arguments/keyword arguments:

Parameters

- **motion_id** (*int*) – identifier for motion (units: -, default: 5, required: No)
- **interpolation_type** (*str*) – Type of cubic spline interpolation. Available types: akima and bspline. (required: yes)
- **file_name** (*str*) – File name with full path. The input file should be a JSON file with two KV pairs i.e. “time_stamps”: and “joint_angles”: Please use the same naming convention. The value should be a list for time_stamps and for joint_angles a list of lists(size: dof). (required: Yes)
- **is_approximated** (*bool*) – (Only for BSpline motion) True if the spline path should be approximated, False if the spline should be interpolated i.e. should exactly pass through the recorded points. Parameter only included in B-Spline. (units: N/A, default: True, required: No)

Raises

- **ConnectionError** – If there is a failure to connect to the robot.
- **UnfeasibleMotion** – If motion is not possible with the given input parameters.

Sample Usage:

```

from neurapy.robot import Robot

r = Robot()
r.init_program()
move_spline_property = {
    "interpolation_type"
    "file_name": "./test_paths/test_path1.json",
}
motion_id = r.pre_plan_move_spline(**move_spline_property, motion_id=
↳ 2)
execute_motion = r.execute_pre_plan_motion(motion_id) #To execute the
↳ planned id

```

pre_plan_realtime_wait(*args, **kwargs)

Allows to pre plan the wait and execute it at a later time.

Parameters

- **motion_id** (*int*) – The id which the motion should get, this is needed for gui feedback. (units: N/A, default: 0, required: No)
- **blocking** (*bool*) – Block execution until wait is completed, if enabled. (units: N/A, default: True, required: No)
- **time_in_s** (*float*) – The time which the wait command should take. (units: sec, default: 0.0, required: No)

Sample Usage:

Example 1:

```

from neurapy.robot import Robot

r = Robot()
wait_property = {
    'motion_id': 1,
    'time_in_s': 0.010,
}
motion_id = r.pre_plan_realtime_wait(**wait_property)
r.execute_pre_plan_motion(motion_id) # this is blocking as long
↳ as not given a non blocking command.
r.stop() # if there are multiple motions than, this needs to be
↳ called only once at the end of the script

```

program_status()

Query the program status of the robot.

Returns

The program status of the robot.

- 'NOT_RUNNING': If the program is not running from the Teach pendant.
- 'RUNNING' : If the program is running from the Teach pendant.
- 'PAUSED' : If the program running from the Teach pendant is in a pause state.

Return type

str

Sample Usage:

```
from neurapy.robot import Robot

r = Robot()
print(r.program_status())
```

quaternion_to_rpy(*w, x, y, z*)

Convert a quaternion representation to roll-pitch-yaw (RPY) angles.

Parameters

- **w** (*float*) – Scalar component (real) of the quaternion.
- **x** (*float*) – First vector component of the quaternion.
- **y** (*float*) – Second vector component of the quaternion.
- **z** (*float*) – Third vector component of the quaternion.

Returns

A list containing [roll, pitch, yaw] angles in radians.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
quaternion = r.quaternion_to_rpy(0.85,0,0.52,0)
```

rad2encoder(*joint_angles, offset=False*)

Convert joint angles in radians to encoder impulses.

Parameters

- **joint_angles** (*list*) – A list of joint angles in radians (required : Yes).
- **offset** (*bool*) – If True considers robot offset values for calculation (Optional).

Returns

A list of encoder impulses corresponding to the provided joint angles.

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
joint_angles = r.get_current_joint_angles()
encoder_impulses = r.rad2encoder(joint_angles)
```

read_general_purpose_registers()

Method to read general purpose registers from a Modbus server.

Returns

The list of register values, or None if an error occurred.

Return type

list

read_safeio(*args, **kwargs)

Read the values of configurable/safe Input/Outputs.

Parameters

- **io_name** (*int*) – The identifier of the safe I/O channel to read.

Returns

Value of safe I/O.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
safe_io = r.read_safeio(1)
print(safe_io) #True/False
```

realtime_wait(*args, **kwargs)

This function can be used, to add a wait time to the motion buffer.

Parameters

- **motion_id** (*int*) – The id which the motion should get, this is needed for gui feedback. (units: N/A, default: 0, required: No)
- **blocking** (*bool*) – Block execution until wait is completed, if enabled. (units: N/A, default: True, required: No)
- **time_in_s** (*float*) – The time which the wait command should take. (units: sec, default: 0.0, required: No)

Sample Usage:**Example 1:**

```
from neurapy.robot import Robot

r = Robot()
wait_property = {
    'motion_id': 1,
    'blocking': True,
    'time_in_s': 0.010,
}
r.realtime_wait(**wait_property)
r.stop() # if there are multiple motions than, this needs to be
↔ called only once at the end of the script
```

release()

To release the grasped object

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.release()
```

remove_load()

Remove the load which was set in set_load by setting the load to NoLoad.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.remove_load()
```

reparent_pose(*pose_name*, *new_frame_name*)

Method which shall change the association of the input pose(point/frame) to a new frame. This shall retain the world coordinates of the input pose but only change its association to a new parent frame such that when the new parent is updated, this input pose shall also get itself updated.

Parameters

- **pose_name** (*str*) – The name of the pose to be reparented.
- **new_frame_name** (*str*) – The name of the new parent pose.

Raises

- **ValueError** – If the input{pose_name} is not a string.
- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If user frame deletion fails with the given input data.

Returns

True if operation successful.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.reparent_pose(pose_name="point_1", new_frame_name="new_frame")
```

reset_axes_lock()

Reset all joint and Cartesian axis locks in free drive mode.

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **RuntimeError** – If the robot is in motion and axis lock cannot be applied.
- **RuntimeError** – If axis lock cannot be controlled in this version of the control software.
- **ConnectionError** – If an internal error occurs while resetting axis locks.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.lock_cartesian_axes(["X", "Z"])
r.reset_axes_lock()
```

reset_collision()

Reset the collision state of the robot.

Returns

True if the robot is not in collision, False otherwise.

Return type

bool

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
r.reset_collision()
```

reset_control()

To restart the control software running on the robot. Equivalent to reset control option from Teach pendant

Returns

True if the control reset was successful, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.reset_control():
    print("Control reset successful.")
else:
    print("Control reset failed.")
```

reset_errors(*args, **kwargs)

Method to reset the errors on the robot

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
r.reset_error()
```

reset_external_device_actions() → bool

Resets action for all external devices

Returns

True if resetting of actions was successful

Return type

bool

Raises

DriverError – If resetting of actions fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()

r.reset_external_device_actions()
```

reset_external_device_error(file_path: str, ignore_no_connection: bool = False) → bool

Resets the error state of an external device.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (~/.data/process/).
- **ignore_no_connection** (*bool*, *optional*) – Flag to control behavior when connection fails. Defaults to False. If set to True, the robot tries to establish a connection

to the device, but continues the program execution even if the connection fails. In this case, no communication with the device will take place.

Returns

True if resetting the error succeeded, False otherwise.

Return type

bool

Raises

DriverError – If resetting the error state fails.

Sample Usage

```
from neurapy.robot import Robot
r = Robot()
reset_success = r.reset_external_device_error("external_device.json")
```

reset_led_mode()

Reset the LED mode to its default state.

Returns

Result of the LED reset execution.

Return type

bool

Raises

RuntimeError – If LED mode cannot be controlled in the current version of Control Software.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.reset_led_mode():
    print("LED reset to default state.")
else:
    print("Failed to reset LED.")
```

reset_warnings()

Method to reset/clear the warnings on the robot

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
r.reset_warnings()
```

rpy_to_quaternion(*r, p, y*)

Convert roll-pitch-yaw (RPY) angles to a quaternion representation.

Parameters

- **r** (*float*) – Roll angle in radians.
- **p** (*float*) – Pitch angle in radians.
- **y** (*float*) – Yaw angle in radians.

Returns

A list representing the quaternion [w, x, y, z].

Return type

list

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
rpy = r.rpy_to_quaternion(0,1.1,0)
```

save_pre_planned_motion_to_json(*args, **kwargs)

Saves pre planned motion as json file.

Note that a file name that already exists is overwritten and not checked.

Parameters

- **motion_id** (*int*) – the id which should be saved into a json (type: identifier - int, units: N/A, required: Yes)
- **file_path_and_name** (*string*) – location and name of json file (type: location and name - string, units: N/A, required: Yes)

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()

file_path_and_name = "/home/Desktop/motion_1"

r.init_program()
r.pre_plan_move_joint([0.0] * r.dof, motion_id = 1)
r.save_pre_planned_motion_to_json(motion_id = 1, file_path_and_name =
↪file_path_and_name)

r.stop()
```

servo_j(*args, **kwargs)

Method to do Servo in Joint Space

Parameters

- **servo_target_position** (*list[float]*) – List of joint target positions in radians. Length of list should be equal to the degree of freedom.
- **servo_target_velocity** (*list[float]*) – List of joint target velocities in radians per second. Length of list should be equal to the degree of freedom.
- **servo_target_acceleration** (*list[float]*) – List of joint target accelerations in radians per second squared. Length of list should be equal to the degree of freedom.
- **cycle_time** (*float*) – Cycle time for ServoJ in seconds. Default is 0.001 seconds.

Returns

Servo warning and error codes.

Return type

int

Sample Usage:

```
from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()
```

(continues on next page)

(continued from previous page)

```

#Switch to external servo mode
r.activate_servo_interface('position')

dof = 6

otg = Ruckig(dof, 0.001) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]
inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
res = Result.Working

error_code = 0

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code == 0 or error_code is None):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration)
        →optional cycle time can be added here
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001) #Sleep function should have the same cycle_
        →time as the ServoJ function
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

servo_x(*args, **kwargs)

Method to do Servo in Cartesian Space

Parameters

- **servo_target_position** (*list[float]*) – List of Cartesian target positions in meters. Length of list should be 7, containing [X, Y, Z, qw, qx, qy, qz].

- **servo_target_velocity** (*list[float]*) – List of Cartesian target velocities in meters per second and radians per second. The first three elements represent translational velocity (m/s), and the next three elements represent rotational velocity (rad/s).
- **servo_target_acceleration** (*list[float]*) – List of Cartesian target accelerations in meters per second squared and radians per second squared. The first three elements represent translational acceleration (m/s^2), and the next three elements represent rotational acceleration (rad/s^2).
- **gain_parameter** (*float*) – ServoX gain parameter used for the proportional controller; should be within the range [0.2, 100].

Returns

Servo warning and error codes.

Return type

int

Sample Usage:

```
from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig
import copy

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

cart_pose_length = 7 #X,Y,Z,qw,qx,qy,qz

otg = Ruckig(cart_pose_length, 0.001) # control cycle
inp = InputParameter(cart_pose_length)
out = OutputParameter(cart_pose_length)

inp.current_position = r.get_tcp_pose()
inp.current_velocity = [0.]*cart_pose_length
inp.current_acceleration = [0.]*cart_pose_length

target = copy.deepcopy(inp.current_position)
target[0] += 0.2 # Move 200mm in X direction
inp.target_position = target
inp.target_velocity = [0.]*cart_pose_length
inp.target_acceleration = [0.]*cart_pose_length

inp.max_velocity = [0.5]*cart_pose_length
inp.max_acceleration = [3]*cart_pose_length
inp.max_jerk = [10.]*cart_pose_length
res = Result.Working

servox_proportional_gain = 25

velocity = [0.] * 6 #Since ruckig does not provide rotational_
↳velocity if quaternion is input, we can send 0 rotational_
↳feedforward velocity
acceleration = [0.] * 6 #Since ruckig does not provide rotational_
↳acceleration if quaternion is input, we can send 0 rotational_
↳feedforward acceleration
```

(continues on next page)

(continued from previous page)

```

error_code = 0

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    error_code = 0
    if(error_code == 0 or error_code is None):

        res = otg.update(inp, out)

        position = out.new_position

        for i in range(0,3): # Updating target translation velocity,
        ↪and accelerations
            velocity[i] = out.new_velocity[i]
            acceleration[i] = out.new_acceleration[i]

        error_code = r.servo_x(position, velocity, acceleration,
        ↪servox_proportional_gain)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

set_analog_output(*io_name*, *target_value*)

Set the analog outputs of the control box.

Use the `get_io_configuration()` function to determine the number of available I/Os.

Parameters

- **io_name** (*int*) – The pin number of the analog output to set.
- **target_value** (*float*) – The target value to set for the analog output.

Returns

True if the operation succeeded, False otherwise.

Return type

bool

Sample Usage:

```

import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_analog_output(1,3.2)
print(io_set)

```

set_current_position_as_zero(*list_of_axis*, *pass_phrase=None*)

Sets the current joint position of the given axis to zero.

⚠ Warning:

This function sets the current joint position to zero. Incorrect usage of this function, such as setting incorrect encoder values, may result in serious unexpected behavior of the robot.

Parameters

- **list_of_axis** (*list*) – A list of axes that need to be set to zero with values (bool) for each joint.
- **pass_phrase** (*str*) – The password which is required to call this function.

Returns

True if the joint position was successfully updated.

Return type

bool

Raises

- **ValueError** – If the length of the provided encoder offsets list is not equal to the number of joints on the robot.
- **ConnectionError** – If there is a failure to connect to the robot or the database, or if there is an issue updating the encoder offsets, or if the pass_phrase is not correct.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_current_position_as_zero([False, True, False, True, False,
↪ False], pass_phrase = userpassword)
```

set_dblout_register(*first_element*, *last_element*, **values*)

Writes values into an element range of the DBLOUT register

Parameters

- **first_element** (*int*) – Index of the first integer DBLOUT element to write into.
- **last_element** (*int*) – Index of the last integer DBLOUT element to write into.
- **values** (*float*) – Target values as arguments.

Returns

Number of elements written, or 0 in case of failure (invalid range, communication failure).

Return type

int

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
num_written = r.set_dblout_register(0, 2, 3.3, 6.6, 9.9) # writes
↪ three floating point values
# into the first three elements of DBLOUT
```

set_digital_output(*io_name*, *target_value*)

Set the digital outputs of the control box.

Use the [get_io_configuration\(\)](#) function to determine the number of available I/Os.

Parameters

- **io_name** (*int*) – The number of the digital output to set.
- **target_value** (*bool*) – The target value to set for the digital output (True or False).

Raises

TypeError – If *io_name* is not an integer or *target_value* is not a boolean.

Returns

True if the digital output is set successfully, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_digital_output(1, True)
print(io_set)
```

set_encoder_offsets(*list_of_encoder_offsets*, *relative=False*, *pass_phrase=None*)

Sets the encoder offsets for the robot's joints relative or absolute.

 Warning:

This function sets the encoder values of joints. Incorrect usage of this function, such as setting incorrect encoder values, may result in serious unexpected behavior of the robot.

Parameters

- **list_of_encoder_offsets** (*list*) – A list of encoder offset values (int) for each joint. The length of the list must be equal to the number of joints on the robot.
- **relative** (*bool*) – Defines if the given encoder offset should be added relative to the current encoder offsets or set as absolute encoder values.
- **pass_phrase** (*str*) – The password which is required to call this function.

Returns

True if the encoder offsets were successfully updated, False otherwise.

Return type

bool

Raises

- **ValueError** – If the length of the provided encoder offsets list is not equal to the number of joints on the robot.
- **ConnectionError** – If there is a failure to connect to the robot or the database, or if there is an issue updating the encoder offsets, or if the *pass_phrase* is not correct, or if the encoder offset should be applied relative.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_encoder_offsets([#encoder offsets from the provided robot_
↳ manual/data sheet], relative = False, pass_phrase = userpassword)
```

set_ext_digital_output(*io_name*, *target_value*)

Set the digital outputs of the external I/O module.

Use the `get_io_configuration()` function to determine the number of available I/Os.

Parameters

- **io_name** (*int*) – The pin number of the digital output to set.
- **target_value** (*bool*) – The target value to set for the digital output (True or False).

Raises

TypeError – If *io_name* is not an integer or *target_value* is not a boolean.

Returns

True if the digital output is set successfully, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_ext_digital_output(1, True)
print(io_set)
```

set_external_device_actions(*file_path*: *str*, *actions*: *List[Dict]*) → bool

Sets actions for an external device. An action consists of a trigger pose and an associated function (including any parameter values) to execute. Multiple actions can be registered at once. The robot maintains these in a queue and continuously monitors whether the trigger pose of the currently queued action has been reached. When the trigger pose is reached, the corresponding external device function is executed, the action is removed from the queue, and the robot proceeds to wait for the next trigger condition.

In essence, actions allow certain device functions to be automatically triggered when the robot reaches specific poses. This mechanism is particularly useful for tasks such as dynamically adjusting welding parameters or other process-specific settings based on the robot pose.

Parameters

- **file_path** (*str*) – The path to the JSON configuration file of the external device. Can be either a full path or just the filename. If only a filename is provided, it is assumed to be in the process folder (`~/data/process/`).
- **actions** – list of dictionary (each containing action details), each dictionary has the keys `trigger_pose`, `function_name`, and `function_parameters`. The `trigger_pose` key must contain a value which is the pose of the robot at which this actions must be triggered, it contains 6 values(x,y,z,r,p,y) The `function_name` key contains a string value The `function_parameters` key contains all the parameters (in key value pair format) required to execute the `function_name` argument. If the function does not need any parameters, then an empty dictionary(`{}`) must be sent.

Returns

True if setting of actions was successful

Return type

bool

Raises

DriverError – If setting of actions fails.

Sample Usage


```

from neurapy.robot import Robot
r = Robot()
p1 = [0.00, 0.00, 0.00, 0.00, 0.00, 0.00] # representing the first
↳point in actions
p2 = [0.10, 0.10, 0.10, 0.00, 0.00, 0.00] # representing the second
↳point in actions
p3 = [0.10, 0.20, 0.10, 0.00, 0.00, 0.00] # representing the third
↳point in actions
function_name = "job" # represents the function
↳to be executed
function_param1 = {"job": "1", "current": 10, "voltage": 15, "current2
↳":20, "voltage2":30, "correction":30, "arc":20, "control":8, "power
↳":0}
function_param2 = {"job": "2", "current": 20, "voltage": 25, "current2
↳":20, "voltage2":30, "correction":30, "arc":20, "control":8, "power
↳":0}
function_param3 = {"job": "3", "current": 15, "voltage": 10, "current2
↳":20, "voltage2":30, "correction":30, "arc":20, "control":8, "power
↳":0}
action1 = {"trigger_pose":p1, "function_name":function_name,
↳"function_parameters":function_param1}
action2 = {"trigger_pose":p2, "function_name":function_name,
↳"function_parameters":function_param2}
action3 = {"trigger_pose":p3, "function_name":function_name,
↳"function_parameters":function_param3}

actions_list = [action1, action2, action3]

r.set_external_device_actions("iROB401.json",actions=actions_list)

```

set_intout_register(first_element, last_element, *values_or_bytes)

Writes values or octets into an element range of the INTOUT register

Parameters

- **first_element** (int) – Index of the first integer INTOUT element to write into.
- **last_element** (int) – Index of the last integer INTOUT element to write into.
- **values_or_bytes** (bytes/int) – Either one Python “bytes” argument or target values as arguments.

Returns

Number of elements written, or 0 in case of failure (invalid range, communication failure).

Return type

int

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
num_written = r.set_intout_register(2, 3, b'{{{yyyyy}}}') # writes 8
↳octets
# given as a binary bytes() into the register
↳storage of INTOUT elements 2 and 3
num_written = r.set_intout_register(0, 2, 14, 42, 84) # writes three
↳integers into the first three
# elements of INTOUT

```

set_joint_acceleration(*acceleration*)

Set the acceleration value for joint motion.

Parameters

acceleration (*float*) – The acceleration value for joint motion, ranging from 0 to 100 (percentage of maximum angular acceleration).

Raises

- **TypeError** – If acceleration is not a numeric value (int or float).
- **ValueError** – If acceleration is outside the range of 0 to 100.

Returns

True if successful, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_joint_acceleration(50) # Set joint acceleration to 50% of
↪ maximum
```

set_joint_speed(*speed*)

Set the speed value for joint motion.

Parameters

speed (*float*) – The speed value for joint motion, ranging from 0 to 100 (percentage of maximum angular speed).

Raises

- **TypeError** – If speed is not a numeric value (int or float).
- **ValueError** – If speed is outside the range of 0 to 100.

Returns

True if successful, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_joint_speed(75) # Set joint speed to 75% of maximum
```

set_led_mode(*mode*, *color*, *frequency*)

Turn on the LED of the robot using the specified mode, a list of colors, and frequency.

Parameters

- **mode** (*int*) – LED operating mode. Allowed values:
 0 - LED_MODE_FLASH 1 - LED_MODE_HEARTBEAT 2 - LED_MODE_CYCLE (only mode that supports multiple colors) 3 - LED_MODE_SOLID
- **color** (*list[int]*) – List of LED colors. - If mode is LED_MODE_CYCLE (2), multiple colors may be provided and will be cycled. - For all other modes, the list must contain exactly one color. Allowed values for each item:

0 - LED_COLORLESS 1 - LED_GREEN 2 - LED_RED 3 - LED_YELLOW 4 - LED_BLUE 5 - LED_LIGHTBLUE 6 - LED_MAGENTA 7 - LED_WHITE

- **frequency** (*float*) – Flashing or cycling frequency (in Hz). Relevant when mode is FLASH or CYCLE.

Returns

Result of the LED mode execution.

Return type

bool

Raises

- **TypeError** – If input types are invalid.
- **ValueError** – If values are outside accepted ranges or structure.
- **RuntimeError** – If LED mode cannot be controlled in the current version of Control Software.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
# Cycle through GREEN, RED, and BLUE at 2.5 Hz
if r.set_led_mode(2, [1, 2, 4], 2.5):
    print("LED pattern applied.")
else:
    print("Failed to apply LED pattern.")
```

set_linear_acceleration(*acceleration*)

Set the acceleration value for linear motion.

Parameters

acceleration (*float*) – The acceleration value for linear motion (m/s²). Minimum value is 0.0. Maximum value can be found in GUI settings page.

Raises

- **TypeError** – If acceleration is not a numeric value (int or float).
- **ValueError** – If acceleration is less than 0.0.

Returns

True if successful, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_linear_acceleration(0.5) # Set linear acceleration to 0.5 m/s^2
```

set_linear_speed(*speed*)

Set the speed value for linear motion.

Parameters

speed (*float*) – The speed value for linear motion (m/s). Minimum value is 0.0. Maximum value can be found in GUI settings page.

Raises

- **TypeError** – If speed is not a numeric value (int or float).
- **ValueError** – If speed is less than 0.0.

Returns

True if successful, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_linear_speed(0.75) # Set linear speed to 0.75 m/s
```

set_load(load_name, load_mass=None, load_cog=None, load_inertia=None)

Sets a load, e.g., an object that is grasped. If the load is present in the database, it is updated. If it is not present, it is first added to the database and then updated.

Parameters

- **load_name** (*string, required*) – String representing the payload name
- **load_mass** (*float, optional*) – Mass of load in kg
- **load_cog** (*list of float, optional*) – Center of gravity of the payload, specified as [x, y, z] coordinates in meters.
- **load_inertia** (*list of float, optional*) – Upper triangular part of the inertia matrix, specified in the order [xx, yy, zz, xy, xz, yz].

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_load(load_name = 'NewLoad', load_mass=1.1, load_cog=[0.02, 0.03, ↵
↵0.04], load_inertia=[0.01, 0.02, 0.03, 0.04, 0.05, 0.06])
```

set_opcua_msg(*args, **kwargs)

Send a message to the OPC UA server running on the control box.

Parameters

- **message** (*str*) – The message to be sent to the OPC UA server.
- **opcua_register_index** (*int*) – The index of the OPC UA register where the message will be stored.

Returns

True if the message is successfully sent and stored in the specified register, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
message = "Hello, OPC UA Server!"
register_index = 1 # Replace with the index of the target OPC UA ↵
↵register
if r.set_opcua_msg(message, register_index):
    print("Message sent and stored in OPC UA register successfully.")
else:
    print("Failed to send the message or store it in the OPC UA ↵
↵register.")
```

set_override(value)

Set the override value.

Parameters

value (*int*) – The new override value to set.

Returns

True if the override value is successfully set, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
new_override_value = 0.5 # it should be from 0 to 1
if r.set_override(new_override_value):
    print(f"Override value set to {new_override_value} successfully.")
else:
    print("Failed to set the override value.")
```

set_safety(safety: bool)

Method to enable or disable the safety mode. If set to True, max speed is slashed to 25%.

Parameters

safety (*bool*) – Set to True to enable safety mode, or False to disable it.

Returns

True if the safety mode was set successfully, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_safety(True) # Enable safety mode
```

set_tool(*args, **kwargs)

Notify the gripper/tool change to software components after the physical change.

Parameters

tool_name (*str*) – The name of the tool defined from the GUI.

Returns

True if the operation succeeds, False if the operation fails.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.set_tool(tool_name="name given during tool creation in GUI")
```

set_tool_digital_output(io_number, target_value)

Set the target value on the specified tool digital output.

Use the `get_io_configuration()` function to determine the number of available I/Os.

Parameters

- **io_number** (*int*) – The number of the tool digital output to be triggered.
- **target_value** (*bool*) – The target value to set for the tool digital output.

Returns

True if the operation succeeded, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_tool_digital_output(1, True) # To set tool digital
↪ output 1 to high
print(io_set)
```

set_tool_digital_outputs(target_value)

Set digital outputs of the tool.

Use the `get_io_configuration()` function to determine the number of available I/Os.

Parameters

target_value (*list of float*) – A list of target values to set for the tool digital outputs.

Returns

True if the operation succeeded, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_set = r.set_tool_digital_outputs([0.0, 1.0, 0.0]) # To set tool
↪ digital output to high
print(io_set)
```

speed_j(*args, **kwargs)

Method to do Joint Velocity Control

Parameters

- **speedj_target_velocity** (*list[float]*) – List of Joint target velocities in radians per second. Length of list should be equal to the degree of freedom.
- **speedj_target_acceleration** (*list[float]*) – List of Joint target accelerations in radians per second². Length of list should be equal to the degree of freedom.

Returns

Servo warning and error codes.

Return type

int

Sample Usage:

```
from neurapy.robot import Robot from neurapy.commands.motion.servo import Servo import
time import numpy as np import copy
```

```

r = Robot() r.activate_servo_interface('position') # Activate servo interface before sending
trajectories.

dof = 6 joint_angles = np.array(r.get_current_joint_angles()) initial_joint_angle =
np.array([0.] * dof) vel = [0.]*dof acc = [0.]*dof

while(i < 1000):
    error_code = r.serveo_j(initial_joint_angle.tolist(), vel, acc) time.sleep(0.001)

    target_1 = [0.5, 0., 0.5, 0., 0.5, 0.] target_2 = [-0.5, 0., -0.5, 0., -0.5, 0.]

    change_target = False

    i = 0

while(True):

    if(i % 500 == 0):
        change_target = not change_target

    if(change_target):
        velocity = target_1 acceleration = [0.] * dof

    else:
        velocity = target_2 acceleration = [0.] * dof

    error_code = r.speed_j(velocity, acceleration)

    time.sleep(0.001) i+=1

print("Servo has ended") r.deactivate_servo_interface() # deactivate servo interface to resume
internal primitive execution r.stop()

speed_x(*args, **kwargs)

```

Method to do Cartesian Velocity Control in Cartesian Space

Parameters

- **speedx_target_velocity** (*list[float]*) – List of Cartesian target velocities in meters per second and radians per second. The first three elements represent translational velocity (m/s), and the next three elements represent rotational velocity (rad/s).
- **speedx_target_acceleration** (*list[float]*) – List of Cartesian target accelerations in meters per second squared and radians per second squared. The first three elements represent translational acceleration (m/s^2), and the next three elements represent rotational acceleration (rad/s^2).

Returns

Servo warning and error codes.

Return type

int

Sample Usage:

```


```

```

from neurapy.robot import Robot from neurapy.commands.motion.servo import Servo import
time import numpy as np import copy

```

```

r = Robot() r.activate_servo_interface('position') # Activate servo interface before sending
trajectories.

```

```

joint_angles = np.array(r.get_current_joint_angles()) initial_joint_angle = np.array([0., 0.,
1.57, 0., 1.57, 0.]) #For 6 DoF Robot vel = [0.]*6 acc = [0.]*6

```

```

while(i < 1000):
    error_code = r.serveo_j(initial_joint_angle.tolist(), vel, acc) time.sleep(0.001)

```

```

print("Starting Servo Motion")
target_1 = [0.0, 0.35, 0.0, 0., 0.0, 0.] target_2 = [0.0, -0.35, 0.0, 0., 0., 0.]
change_target = False
i = 0
while(True):
    if(i % 350 == 0):
        change_target = not change_target
    if(change_target):
        velocity = target_1 acceleration = [0.] * 6
    else:
        velocity = target_2 acceleration = [0.] * 6
    error_code = r.speed_x(velocity, acceleration)
    time.sleep(0.001) i+=1

print("Servo has ended") r.deactivate_servo_interface() # deactivate servo interface to resume
internal primitive execution r.stop()

```

stop(*args, **kwargs)

To stop the robot's motion and to terminate the script execution. This needs to be added at the end of the script(only once) for a proper script termination

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.stop()

```

stop_movelinear_online(*args, **kwargs)

Method to stop MoveLinearOnline motion if running

Sample Usage

```

from neurapy.robot import Robot
import time
import copy

r = Robot()

target_1 = 0.3
target_2 = 0.25

#Switch to external servo mode
r.activate_servo_interface('position')
cart_pose_length = 7 # X,Y,Z,qw,qx,qy,qz
target = copy.deepcopy(r.get_tcp_pose())
print(target)

# Move target_1 unit in -X direction
target[0] -= target_1
velocity = [0.15]*6
acceleration = [2.]*6
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion.
time.sleep(5)

```

(continues on next page)

(continued from previous page)

```

target = copy.deepcopy(r.get_tcp_pose())
#Move target_2 units in +Z direction
target[2] += target_2
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 2 sec to complete the motion
time.sleep(2)

#Stop the movelinear online motion
r.stop_movelinear_online()
time.sleep(3)
print("Robot stopped")
r.deactivate_servo_interface()
r.stop()

```

switch_to_automatic_mode()

Switch the robot to Automatic mode.

Returns

True if the operation is executed successfully.

Return type

bool

Raises

ControllerException – If it is unable to switch to automatic mode.

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
if r.switch_to_automatic_mode():
    print("Robot switched to Automatic mode.")
else:
    print("Failed to switch to Automatic mode.")

```

switch_to_real()

Switches the robot's running context to real mode.

Returns

True if the switch was successful, False otherwise.

Return type

bool

Sample Usage:

```

from neurapy.robot import Robot
r = Robot()
r.switch_to_real()

```

switch_to_semi_automatic_mode()

Switch the robot to Semi Automatic mode.

Returns

True if the operation is executed successfully.

Return type

bool

Raises

ControllerException – If it is unable to switch to SemiAutomatic mode.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.switch_to_semi_automatic_mode():
    print("Robot switched to Semi Automatic mode.")
else:
    print("Failed to switch to Semi Automatic mode.")
```

switch_to_simulation()

Switches the robot's running context to simulation mode.

Returns

True if the switch was successful, False otherwise.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.switch_to_simulation()
```

switch_to_teach_mode()

Switch the robot to Teach mode.

Returns

True if the operation is executed successfully.

Return type

bool

Raises

ControllerException – If it is unable to switch to Teach mode.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
if r.switch_to_teach_mode():
    print("Robot switched to Teach mode.")
else:
    print("Failed to switch to Teach mode.")
```

tare_fts()

Set the property for taring force torque sensor to true when called and clears the force torque sensor offsets.

Returns

True if successful, False if an exception occurs

terminate_current_motion()

This function terminates the execution of current motion.

Incase you use this function make sure that next motion is planned dynamically.

Returns

True if successful, False if an exception occurs

turn_off_free_drive_mode()

To turn off free drive mode

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **WrongMode** – If the Robot is not in ‘teach’ mode or in simulation.
- **ConnectionError** – If the Robot is powered off.

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_off_free_drive_mode()
```

turn_off_jog(*args, **kwargs)

Method to disable jogging programatically. This needs to be used in conjunction with jog, turn_on_jog methods.

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
    ↪ 'Joint')
r.jog(set_jogging_external_flag=1)
i = 0
while i < 500:
    r.jog(set_jogging_external_flag=1)
    i += 1
r.turn_off_jog()
```

turn_on_free_drive_mode()

To turn on free drive mode

Returns

True if the operation is executed successfully, False otherwise.

Return type

bool

Raises

- **WrongMode** – If the Robot is not in ‘teach’ mode or in simulation.
- **ConnectionError** – If the Robot is powered off.

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_free_drive_mode()
```

turn_on_jog(*args, **kwargs)

Method to enable jogging programatically. This needs to be used in conjunction with jog, turn_off_jog methods.

Args: - jog_velocity (list of float): A list of joint velocities ranging from -1 to 1 for all joints. - jog_type (str): Specifies the type of jogging, which can be 'Cartesian' or 'Joint'.

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
    ↪ 'Joint')
r.jog(set_jogging_external_flag=1)
i = 0
while i < 500:
    r.jog(set_jogging_external_flag=1)
    i += 1
r.turn_off_jog()
```

unpause(*args, **kwargs)

To unpause the robot's motion.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.unpause()
```

update_current_tool_parameters(*args, **kwargs)

Update the current gripper/tool parameters in the tool list and the selected tool properties.

Parameters

- **_controlOA** (*bool*) – True if the tool is controlled by controlbox analog outputs; False otherwise. (Not Required)
- **_controlOD** (*bool*) – True if the tool is controlled by controlbox digital outputs; False otherwise. (Not Required)
- **_toolOA** (*bool*) – True if the tool is controlled by analog outputs of port present on robot tool flange; False otherwise. (Not Required)
- **_toolOD** (*bool*) – True if the tool is controlled by digital outputs of port present on robot tool flange; False otherwise. (Not Required)
- **autoM** (*float*) – Mass of the tool. (Required)
- **autoMeasureX** (*float*) – Tool center of gravity (COG) in X direction, measured from robot's flange frame. (Not Required)
- **autoMeasureY** (*float*) – Tool center of gravity (COG) in Y direction, measured from robot's flange frame. (Not Required)
- **autoMeasureZ** (*float*) – Tool center of gravity (COG) in Z direction, measured from robot's flange frame. (Not Required)
- **closeInput** (*float*) – Percentage of gripper width for close action. (Not Required)
- **cmdID** (*int*) – Input specific for certain kinds of grippers. (Not Required)
- **force** (*float*) – Gripper closing force. (Not Required)
- **gripper** (*str*) – Gripper type. (Not Required)

- **grippertype** (*str*) – Type of gripper Modbus/Standard. (Not Required)
- **inertiaXX** (*float*) – Ixx of tool. (Not Required)
- **inertiaXY** (*float*) – Ixy of tool. (Not Required)
- **inertiaXZ** (*float*) – Ixz of tool. (Not Required)
- **inertiaYY** (*float*) – Iyy of tool. (Not Required)
- **inertiaYZ** (*float*) – Iyz of tool. (Not Required)
- **inertiaZZ** (*float*) – Izz of tool. (Not Required)
- **offCOA** (*list*) – Offset for control analog outputs. (Not Required)
- **offCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offCOD2** (*int*) – (Not Required)
- **offTOA** (*list*) – Offset for tool analog outputs. (Not Required)
- **offTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offsetA** (*float*) – TCP roll offset. (Not Required)
- **offsetB** (*float*) – TCP pitch offset. (Not Required)
- **offsetC** (*float*) – TCP yaw offset. (Not Required)
- **offsetX** (*float*) – TCP offset in X. (Not Required)
- **offsetY** (*float*) – TCP offset in Y. (Not Required)
- **offsetZ** (*float*) – TCP offset in Z. (Not Required)
- **onCOA** (*list*) – Offset for control analog outputs when turning on. (Not Required)
- **onCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **onCOD2** (*int*) – (Not Required)
- **onTOA** (*list*) – Offset for tool analog outputs when turning on. (Not Required)
- **onTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **openInput** (*float*) – Percentage of gripper width for open action. (Not Required)
- **portID** (*int*) – (Not Required)
- **protocol** (*int*) – (Not Required)
- **robot_type** (*str*) – Type of robot tool. (Not Required)
- **slaveID** (*int*) – (Not Required)
- **speed** (*float*) – (Not Required)

Returns

True if the operation succeeds, False if the operation fails.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.update_current_tool_parameters(offsetZ=0.1, autoM=0.5)
```

update_reference_frame(*name*, *pose*, *parent_frame*='Base', *description*='')

Update a user-defined reference frame in the database.

Parameters

- **name** (*str*) – Name of the reference frame to be updated.
- **pose** (*list*) – A list containing the Cartesian coordinates and orientation of the reference frame in euler or quaternion format (required).
- **parent_frame** (*str*) – Name of the parent reference frame (default: “Base”).
- **description** (*str*) – Description of the reference frame (default: “”).

Returns

True if the user frame was updated successfully.

Return type

bool

Raises

- **ConnectionError** – If a connection to the remote database cannot be established.
- **ValueError** – If user frame update fails with the given input data.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.update_reference_frame(name='new_frame', pose=[0.36, 0.36, 0.1, 3.
↪14, 0, 0, 3.14])
```

update_tool_parameters(*args, **kwargs)

Update the gripper/tool parameters in the tool list with a given tool_name.

Parameters

- **tool_name** (*str*) – The name of the tool defined from the GUI.
- **_controlOA** (*bool*) – True if the tool is controlled by controlbox analog outputs; False otherwise. (Not Required)
- **_controlOD** (*bool*) – True if the tool is controlled by controlbox digital outputs; False otherwise. (Not Required)
- **_toolOA** (*bool*) – True if the tool is controlled by analog outputs of port present on robot tool flange; False otherwise. (Not Required)
- **_toolOD** (*bool*) – True if the tool is controlled by digital outputs of port present on robot tool flange; False otherwise. (Not Required)
- **autoM** (*float*) – Mass of the tool. (Required)
- **autoMeasureX** (*float*) – Tool center of gravity (COG) in X direction, measured from robot’s flange frame. (Not Required)
- **autoMeasureY** (*float*) – Tool center of gravity (COG) in Y direction, measured from robot’s flange frame. (Not Required)
- **autoMeasureZ** (*float*) – Tool center of gravity (COG) in Z direction, measured from robot’s flange frame. (Not Required)
- **closeInput** (*float*) – Percentage of gripper width for close action. (Not Required)
- **cmdID** (*int*) – Input specific for certain kinds of grippers. (Not Required)
- **force** (*float*) – Gripper closing force. (Not Required)
- **gripper** (*str*) – Gripper type. (Not Required)

- **grippertype** (*str*) – Type of gripper Modbus/Standard. (Not Required)
- **inertiaXX** (*float*) – Ixx of tool. (Not Required)
- **inertiaXY** (*float*) – Ixy of tool. (Not Required)
- **inertiaXZ** (*float*) – Ixz of tool. (Not Required)
- **inertiaYY** (*float*) – Iyy of tool. (Not Required)
- **inertiaYZ** (*float*) – Iyz of tool. (Not Required)
- **inertiaZZ** (*float*) – Izz of tool. (Not Required)
- **offCOA** (*list*) – Offset for control analog outputs. (Not Required)
- **offCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offCOD2** (*int*) – (Not Required)
- **offTOA** (*list*) – Offset for tool analog outputs. (Not Required)
- **offTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn off (close) the tool. (Not Required)
- **offsetA** (*float*) – TCP roll offset. (Not Required)
- **offsetB** (*float*) – TCP pitch offset. (Not Required)
- **offsetC** (*float*) – TCP yaw offset. (Not Required)
- **offsetX** (*float*) – TCP offset in X. (Not Required)
- **offsetY** (*float*) – TCP offset in Y. (Not Required)
- **offsetZ** (*float*) – TCP offset in Z. (Not Required)
- **onCOA** (*list*) – Offset for control analog outputs when turning on. (Not Required)
- **onCOD1** (*int*) – If controlled via control box digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **onCOD2** (*int*) – (Not Required)
- **onTOA** (*list*) – Offset for tool analog outputs when turning on. (Not Required)
- **onTOD** (*int*) – If controlled via tool digital outputs, this pin is mapped to turn on (open) the tool. (Not Required)
- **openInput** (*float*) – Percentage of gripper width for open action. (Not Required)
- **portID** (*int*) – (Not Required)
- **protocol** (*int*) – (Not Required)
- **robot_type** (*str*) – Type of robot tool. (Not Required)
- **slaveID** (*int*) – (Not Required)
- **speed** (*float*) – (Not Required)

Returns

True if the operation succeeds, False if the operation fails.

Return type

bool

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()
r.update_tool_parameters(tool_name="name given during tool creation",
↳in GUI", offsetZ=0.1, autoM=0.5)
```

wait(*args, **kwargs)

Wait for a specified signal.

Parameters

- **port_name** (*str*) – The name or identifier of the tool-specific analog input.
- **expected_value** (*float*) – The expected value of the analog input. If None, the function waits for any change in the input value.

Returns

True if the specified condition or event is met, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait(port_name="DI_2", expected_value=1.0, wait_for_
↳signal=True, wait_time=0.0) #waits till digital input 2 is True
print(io_wait)
```

wait_for_analog_input(io_name, expected_value, wait_time=0.0)

Wait for an analog input signal to match the expected value.

Parameters

- **io_name** (*str*) – The name or identifier of the analog input.
- **expected_value** (*float*) – The expected value of the analog input.
- **wait_time** (*float*) – The amount of time (in seconds) to wait before checking the analog input signal. Default is 0.0 seconds.

Returns

True if the analog input matches the expected value, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_analog_input(1,2.3) #waits till analog input 1
↳is 2.3
print(io_wait)
```

wait_for_digital_input(io_name, expected_value, wait_time=0.0)

Wait for a digital input signal to match the expected value.

Parameters

- **io_name** (*str*) – The name or identifier of the digital input.
- **expected_value** (*bool*) – The expected value of the digital input.

- **wait_time** (*float*) – The amount of time (in seconds) to wait before returning. Default is 0.0 seconds.

Returns

True if the digital input matches the expected value, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_digital_input(1,True) #waits till digital input
↪ 1 is True
print(io_wait)
```

wait_for_digital_input_timer_off_delay(*io_name*, *delay*=0.0)

Wait for a given delay and returns after the specified digital input signal reaches low.

Parameters

- **io_name** (*str*) – The name or identifier of the digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

Returns

True if the digital input signal reaches low, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_digital_input_timer_off_delay(1,10) #waits for
↪ 10sec and returns after digital input 1 reaches low
print(io_wait)
```

wait_for_digital_input_timer_on_delay(*io_name*, *delay*=0.0)

Wait for a given delay and returns after the specified digital input signal reaches high.

Parameters

- **io_name** (*str*) – The name or identifier of the digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

Returns

True if the digital input signal reaches high, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_digital_input_timer_on_delay(1,10) #waits for
```

(continues on next page)

(continued from previous page)

```
↪ 10sec and returns after digital input 1 reaches high
print(io_wait)
```

wait_for_tool_analog_input(*io_name*, *expected_value*)

Wait for a tool-specific analog input signal to match the expected value.

Parameters

- **io_name** (*str*) – The name or identifier of the tool-specific analog input.
- **expected_value** (*float/None*) – The expected value of the analog input. If None, the function waits for any change in the input value.

Returns

True if the tool-specific analog input matches the expected value, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_analog_input(1,2.3v) #waits till tool_
↪ analog input 1 is 2.3v
print(io_wait)
```

wait_for_tool_digital_input(*io_name*, *expected_value*, *wait_time=0.0*)

Wait for a tool-specific digital input signal to match the expected value.

Parameters

- **io_name** (*str*) – The name or identifier of the tool-specific digital input.
- **expected_value** (*bool*) – The expected value of the digital input (True or False).
- **wait_time** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

Returns

True if the tool-specific digital input matches the expected value, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_digital_input(1,True) #waits till tool_
↪ digital input 1 is True
print(io_wait)
```

wait_for_tool_digital_input_timer_off_delay(*io_name*, *delay=0.0*)

Wait for a given delay and return after the specified tool digital input signal reaches low.

Parameters

- **io_name** (*str*) – The name or identifier of the tool digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

Returns

True if the tool digital input signal reaches low, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_digital_input_timer_off_delay(1,10) #waits_
↳for 10sec and returns after tool digital input 1 reaches low
print(io_wait)
```

wait_for_tool_digital_input_timer_on_delay(io_name, delay=0.0)

Wait for a given delay and return after the specified tool digital input signal reaches high.

Parameters

- **io_name** (*str*) – The name or identifier of the tool digital input.
- **delay** (*float*) – The amount of time (in seconds) to wait before checking the digital input signal. Default is 0.0 seconds.

Returns

True if the tool digital input signal reaches high, False otherwise.

Return type

bool

Sample Usage:

```
import time
from neurapy.robot import Robot
r = Robot()
io_wait = r.wait_for_tool_digital_input_timer_on_delay(1,10) #waits_
↳for 10sec and returns after tool digital input 1 reaches high
print(io_wait)
```

wait_motion_finished()

This method allows to wait for a motion to complete. If there is no motion queued or all motions are already done this motion, checks once and skips waiting. Note that this only is effective if the motion is called with the non blocking option.

Sample Usage:

```
from neurapy.robot import Robot
r = Robot()

joint_command = {
    "speed": 10,
    "acceleration": 10,
    "jerk": 100,
    "blocking": False,
    "target_joint": [
        r.get_current_joint_angles(),
        [0.0] * 6
    ]
}
"current_joint_angles": r.get_current_joint_angles()
}
```

(continues on next page)

(continued from previous page)

```
r.move_joint(**joint_command) # Does not block until motion is_
↪completed
r.wait_motion_finished() # waits till motion is done
```

EXAMPLES

6.1 Examples for Lara 5

1. Robot Object Initialization

```
from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.platform)
print(r.payload)
print(r.kURL)
print(r.robot_urdf_path)
print(r.current_tool)
print(r.connection)
print(r.version)
```

2. Move joint

```
from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            0.24133577249211113,
            -0.5953164320218438,
            1.9929891407680798,
            -3.1090657476289074,
            -1.7553445455327545,
            0
        ],
        [
            0.7865353588222422,
            -0.028662542184221113,
            1.5519639871284383,
            -3.2960054585898284e-11,
            1.579307188208187,
            0
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
```

(continues on next page)

(continued from previous page)

```
}
r.move_joint(**joint_property)
```

3. Move linear

```
from neurapy.robot import Robot

r = Robot()
linear_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "blend_radius": 0.005,
    "target_pose": [
        [
            0.24448961019605933, 0.3415022886246084, 0.4429983799900599, ↵
↵ 3.141592653589793, -7.7143552935579e-06, -2.1921296394194334
        ],
        [
            0.37280933837762326, 0.19342228374880036, 0.4429983800173495, ↵
↵ -7.392776508693445E-17, -3.1415849392994755, 0.4785940474487451
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_linear(**linear_property)
```

4. Move circular

```
from neurapy.robot import Robot

r = Robot()
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "target_pose": [
        [
            -0.486438899299542, 0.29460704493457296, 0.5165016193028429, ↵
↵ -3.0434630455585077, -0.06638303849596608, 2.588201061874784
        ],
        [
            0.35373071422698277, 0.4452886561213558, 0.5165008179534564, ↵
↵ -3.043525300927239, -0.06631793019551589, 0.8906657281862649
        ],
        [
            0.5626726577180612, -0.0825038527245115, 0.5165008179534564, ↵
↵ 0.09806735266255437, -3.0752747233942777, 2.987178917849476
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_circular(**circular_property)
```

5. Move composite

```
from neurapy.robot import Robot
```

(continues on next page)

(continued from previous page)

```

r = Robot()
composite_motion_property = {
    "speed": 0.432,
    "acceleration": 0.2,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [
        {
            "linear": {
                "blend_radius": 0.005,
                "targets": [
                    [
                        0.24448961019605933, 0.3415022886246084, 0.
↪ 4429983799900599, 3.141592653589793, -7.7143552935579e-06, -2.
↪ 1921296394194334
                    ],
                    [
                        0.37280933837762326, 0.19342228374880036, 0.
↪ 4429983800173495, -7.392776508693445E-17, -3.1415849392994755, 0.
↪ 4785940474487451
                    ]
                ]
            },
            {
                "circular": {
                    "targets": [
                        [
                            0.37280933837762326, 0.19342228374880036, 0.
↪ 4429983800173495, -7.392776508693445E-17, -3.1415849392994755, 0.
↪ 4785940474487451
                        ],
                        [
                            0.22361395388972127, -0.3555218434954414, 0.
↪ 44299838001725345, 3.141592653589793, -7.714290317667482E-06, 2.
↪ 1322507768765044
                        ],
                        [
                            -0.16898439959241998, -0.38450390670444534, 0.
↪ 44299838001725345, -3.141592653589793, -7.714290317599941E-06, 1.
↪ 1567220902055733
                        ]
                    ]
                }
            }
        ]
    }
}
r.move_composite(**composite_motion_property)

```

6. Move recorded path

```

from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "file_name": "<path to file loaction>",
    "speed": 0.25,

```

(continues on next page)

(continued from previous page)

```

        "current_joint_angles": r.get_current_joint_angles(),
    }
    r.move_recorded_path(**recorded_path_property)

```

7. Power

```

from time import sleep
from neurapy.robot import Robot

r = Robot()
r.power_on()
sleep(2)
r.power_off()

```

8. Motion status

```

from neurapy.robot import Robot

r = Robot()
print(r.motion_status())

```

9. Program status

```

from neurapy.robot import Robot

r = Robot()
print(r.program_status())

```

10. Warnings

```

from neurapy.robot import Robot

r = Robot()
print(r.get_warnings())

```

11. Errors

```

from neurapy.robot import Robot

r = Robot()
print(r.get_errors())

```

12. Get point

```

from neurapy.robot import Robot

r = Robot()
point = r.get_point("P1")
print(point)

```

13. IOs

```

import time
from neurapy.robot import Robot

r = Robot()
io_get = r.io("get", io_name = "DO_1")
io_set = r.io("set", io_name = "DO_2", target_value = True)
print(io_get, io_set)

```


14. Gripper

```
from time import sleep
from neurapy.robot import Robot
r = Robot()
r.grasp()
sleep(2)
r.release()
```

15. Set tool

```
from neurapy.robot import Robot
r = Robot()
r.set_tool(tool_name="name given during tool creation in GUI")
```

16. Zero G

```
import time
from neurapy.robot import Robot
r = Robot()
r.turn_on_free_drive_mode()
time.sleep(1)
r.turn_off_free_drive_mode()
```

17. Forward/Inverse kinematics

```
from neurapy.robot import Robot
r = Robot()
target_pose = r.compute_forward_kinematics(joint_angles = [-1.
↪ 3326468647873386, 0.0, 1.570800183972526, 0.0, 1.5708001839725365, 0.
↪ 0])
target_angle = r.compute_inverse_kinematics(target_pose = [0.
↪ 4983978832107115, -0.03249999999999999, 0.3344977006474223, -3.
↪ 0510184133354567e-06, 3.141585287645838, 2.7488999999887627],reference_
↪ joint = [-1.3326468647873386, 0.0, 1.570800183972526, 0.0, 1.
↪ 5708001839725365, 0.0])
```

18. Override

```
import time
from neurapy.robot import Robot
r = Robot()
override_value = r.get_override()
print("override_value : " + str(override_value))
time.sleep(1)

override_value = r.set_override(0.4)
print("Setting Override to 0.4")
time.sleep(1)

override_value = r.get_override()
print("override_value : " + str(override_value))
```

19. Reset Error

```
import time
from neurapy.robot import Robot
r = Robot()
r.reset_errors()
```

20. Get tools

```
import time
from neurapy.robot import Robot
r = Robot()
tools_data = r.get_tools()
```

21. Create tool

```
from neurapy.robot import Robot
r = Robot()
tool_data = { '_control0A': False,
               '_control0D': False,
               '_tool0A': False,
               '_tool0D': False,
               'autoM': 0,
               'autoMeasureX': 0,
               'autoMeasureY': 0,
               'autoMeasureZ': 0,
               'closeInput': 0,
               'cmdID': 16,
               'description': 'Tool Description',
               'force': 0,
               'gripper': '',
               'grippertype': 'Standard Gripper',
               'inertiaXX': 0,
               'inertiaXY': 0,
               'inertiaXZ': 0,
               'inertiaYY': 0,
               'inertiaYZ': 0,
               'inertiaZZ': 0,
               'name': 'NoTool',
               'offCOA': [0, 0, 0, 0, 0, 0, 0, 0],
               'offCOD1': 0,
               'offCOD2': 0,
               'offTOA': [0, 0],
               'offTOD': 0,
               'offsetA': 0,
               'offsetB': 0,
               'offsetC': 0,
               'offsetX': 0,
               'offsetY': 0,
               'offsetZ': 0,
               'onCOA': [0, 0, 0, 0, 0, 0, 0, 0],
               'onCOD1': 0,
               'onCOD2': 0,
               'onTOA': [0, 0],
               'onTOD': 0,
               'openInput': 0,
               'portID': '',
               'protocol': 0,
               'robot_type': 'Tool',
               'slaveID': 0,
               'speed': 0}

tools_data = r.create_tool(tool_data)
```

22. Get encoder offsets

```
from neurapy.robot import Robot
r = Robot()
encoder_offsets = r.get_encoder_offsets()
```

23. Encoder to radian values

```
from neurapy.robot import Robot
r = Robot()
encoder_ticks = r.get_current_load_side_encoder_values()
joint_angles = r.encoder2rad(encoder_ticks)
```

24. Quaternion to roll pitch yaw

```
from neurapy.robot import Robot
r = Robot()
rpy = r.quaternion_to_rpy(0.85,0,0.52,0)
```

25. Roll pitch yaw to quaternion

```
from neurapy.robot import Robot
r = Robot()
quaternion = r.rpy_to_quaternion(0,1.1,0)
```

26. Get reference frame

```
from neurapy.robot import Robot
r = Robot()
frame = r.get_reference_frame("tool_frame")
print(frame)
```

27. Jogging

```
from neurapy.robot import Robot

robot = Robot()
"""
jog_velocity - velocity ranging from [-1,1] for all joints
jog_type - can be either cartesian or joint jogging
turn_on_jog changes the state from internal jogging (from GUI) to
↳ external jogging
"""
robot.turn_on_jog(jog_velocity=[0.2, 0.2, 0.2, 0.2, 0.2, 0.2], jog_type=
↳ 'Joint')

# command to set flag for jogging in external mode.
robot.jog(set_jogging_external_flag = 1)
i = 0

"""
Requires minimum number of cycles in the loop for performing jogging.
Depends upon jogging velocity, override.
"""
while(i < 500):
    """
    command to set flag for jogging in external mode. This command has
    ↳ to be used each time
    external jog command has to be sent
    """
```

(continues on next page)

(continued from previous page)

```

robot.jog(set_jogging_external_flag = 1)
i+=1

"""
Change the state from external jog to internal jog (GUI) and sets all
↪ other
external parameters to false
"""
robot.turn_off_jog()

```

28. Get reference frame with offset

```

from neurapy.robot import Robot
r = Robot()
x_offset = 0.02 # in meters
y_offset = 0.1 # in meters
z_offset = 0.0 # in meters
frame = r.get_reference_frame_with_offset("world",[x_offset,y_offset,z_
↪ offset])
print(frame)

```

29. Get TCP pose

```

from neurapy.robot import Robot
r = Robot()
tcp_pose = r.get_tcp_pose()
print(tcp_pose)

```

30. Read safeio

```

from neurapy.robot import Robot
r = Robot()
safe_io = r.read_safeio(1)
print(safe_io) #True/False

```

31. Servo J

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

dof = 6

otg = Ruckig(dof, 0.001) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]

```

(continues on next page)

(continued from previous page)

```

inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
res = Result.Working

error_code = 0

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    if(error_code == 0 or error_code is None):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

32. Servo J with cycle time

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

dof = 6
cycle_time = 0.008

otg = Ruckig(dof, cycle_time) # DoFs, control cycle
inp = InputParameter(dof)
out = OutputParameter(dof)

inp.current_position = r.get_current_joint_angles()
inp.current_velocity = [0.]*dof
inp.current_acceleration = [0.]*dof

inp.target_position = [0., 0., 0., 0., 0., 0.]

```

(continues on next page)

(continued from previous page)

```

inp.target_velocity = [0.]*dof
inp.target_acceleration = [0.]*dof

inp.max_velocity = [0.5]*dof
inp.max_acceleration = [3]*dof
inp.max_jerk = [10.]*dof
res = Result.Working

error_code = 0

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    if(error_code == 0 or error_code is None):

        res = otg.update(inp, out)

        position = out.new_position
        velocity = out.new_velocity
        acceleration = out.new_acceleration

        error_code = r.servo_j(position, velocity, acceleration, cycle_
↪time)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(cycle_time)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

33. Servo X

```

from neurapy.robot import Robot
import time
from ruckig import InputParameter, OutputParameter, Result, Ruckig
import copy

r = Robot()

#Switch to external servo mode
r.activate_servo_interface('position')

cart_pose_length = 7 #X,Y,Z,qw,qx,qy,qz

otg = Ruckig(cart_pose_length, 0.001) # control cycle
inp = InputParameter(cart_pose_length)
out = OutputParameter(cart_pose_length)

inp.current_position = r.get_tcp_pose()
inp.current_velocity = [0.]*cart_pose_length
inp.current_acceleration = [0.]*cart_pose_length

```

(continues on next page)

(continued from previous page)

```

target = copy.deepcopy(inp.current_position)
target[0] += 0.2 # Move 200mm in positive X direction
inp.target_position = target
inp.target_velocity = [0.]*cart_pose_length
inp.target_acceleration = [0.]*cart_pose_length

inp.max_velocity = [0.5]*cart_pose_length
inp.max_acceleration = [3]*cart_pose_length
inp.max_jerk = [10.]*cart_pose_length
res = Result.Working

servox_proportional_gain = 25

velocity = [0.] * 6 #Since ruckig does not provide rotational velocity,
↳if quaternion is input, we can send 0 rotational feedforward velocity
acceleration = [0.] * 6 #Since ruckig does not provide rotational,
↳acceleration if quaternion is input, we can send 0 rotational,
↳feedforward acceleration

error_code = 0

while res == Result.Working:
    """
    Error code is returned through Servo.
    """
    if(error_code == 0 or error_code is None):

        res = otg.update(inp, out)

        position = out.new_position

        for i in range(0,3): # Updating target translation velocity and,
↳accelerations
            velocity[i] = out.new_velocity[i]
            acceleration[i] = out.new_acceleration[i]

        error_code = r.servo_x(position, velocity, acceleration, servox_
↳proportional_gain)
        scaling_factor = r.get_servo_trajectory_scaling_factor()
        out.pass_to_input(inp)
        time.sleep(0.001)
    else:
        print("Servo in error, error code, ", error_code)
        break
r.deactivate_servo_interface()

r.stop()

```

34. MoveLinearOnline

```

from neurapy.robot import Robot
import time
import copy

r = Robot()

```

(continues on next page)

(continued from previous page)

```

target_1 = 0.3
target_2 = 0.25

#Switch to external servo mode
r.activate_servo_interface('position')
cart_pose_length = 7 # X,Y,Z,qw,qx,qy,qz
target = copy.deepcopy(r.get_tcp_pose())
print(target)

# Move target_1 unit in -X direction
target[0] -= target_1
velocity = [0.15]*6 #Max velocities
acceleration = [2.]*6 #Max accelerations
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion.
time.sleep(5)

target = copy.deepcopy(r.get_tcp_pose())
#Move target_2 units in +Z direction
target[2] += target_2
error_code = r.movelinear_online(target, velocity, acceleration)

#Sleep for 5 sec to complete the motion
time.sleep(5)

#Stop MoveLinearOnline if robot is still in motion
r.stop_movelinear_online()

#Sleep for 1 sec to stop the robot motion
time.sleep(1)

print("Robot stopped")
r.deactivate_servo_interface()
r.stop()

```

35. SpeedJ

```

from neurapy.robot import Robot
from neurapy.commands.motion.servo import Servo
import time
import numpy as np
import copy

r = Robot()
r.activate_servo_interface('position') # Activate servo interface,
↪ before sending trajectories.

dof = 6
joint_angles = np.array(r.get_current_joint_angles())
initial_joint_angle = np.array([0.] * dof)
vel = [0.]*dof
acc = [0.]*dof

while(i < 1000):
    error_code = r.servo_j(initial_joint_angle.tolist(), vel, acc)

```

(continues on next page)

(continued from previous page)

```

    time.sleep(0.001)

target_1 = [0.5, 0., 0.5, 0., 0.5, 0.]
target_2 = [-0.5, 0., -0.5, 0., -0.5, 0.]

change_target = False

i = 0

while(True):
    if(i % 500 == 0):
        change_target = not change_target
    if(change_target):
        velocity = target_1
        acceleration = [0.] * dof

    else:
        velocity = target_2
        acceleration = [0.] * dof

    error_code = r.speed_j(velocity, acceleration)

    time.sleep(0.001)
    i+=1

print("Servo has ended")
r.deactivate_servo_interface() # deactivate servo interface to resume
↪ internal primitive execution
r.stop()

```

36. SpeedX

```

from neurapy.robot import Robot
from neurapy.commands.motion.servo import Servo
import time
import numpy as np
import copy

r = Robot()
r.activate_servo_interface('position') # Activate servo interface
↪ before sending trajectories.

joint_angles = np.array(r.get_current_joint_angles())
initial_joint_angle = np.array([0., 0., 1.57, 0., 1.57, 0.]) #For 6 DoF
↪ Robot
vel = [0.]*6
acc = [0.]*6

while(i < 1000):
    error_code = r.servo_j(initial_joint_angle.tolist(), vel, acc)
    time.sleep(0.001)

print("Starting Servo Motion")

target_1 = [0.0, 0.35, 0.0, 0., 0.0, 0.]
target_2 = [0.0, -0.35, 0.0, 0., 0., 0.]

```

(continues on next page)

(continued from previous page)

```

change_target = False

i = 0

while(True):
    if(i % 350 == 0):
        change_target = not change_target
    if(change_target):
        velocity = target_1
        acceleration = [0.] * 6
    else:
        velocity = target_2
        acceleration = [0.] * 6

    error_code = r.speed_x(velocity, acceleration)

    time.sleep(0.001)
    i+=1

print("Servo has ended")
r.deactivate_servo_interface() # deactivate servo interface to resume
↪ internal primitive execution
r.stop()

```

6.2 Examples for Lara 8

1. Robot Object Initialization

```

from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.platform)
print(r.payload)
print(r.kURL)
print(r.robot_urdf_path)
print(r.current_tool)
print(r.connection)
print(r.version)

```

2. Move joint

```

from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        0.7740276502469322,
        0.13599234975308788,

```

(continues on next page)

(continued from previous page)

```

        1.2223307505101257,
        -0.31901765024691225,
        0.3242476502469122,
        -0.31901765024691225
    ],
    [
        0.30333999999999633,
        -0.5334353004938217,
        1.4419907505101492,
        -0.31901765024691225,
        0.3242476502469122,
        -0.31901765024691225
    ],
    [
        0.059259210493824405,
        0.08133676975309524,
        1.4419907505101492,
        -1.0084715804938613,
        -0.7008323497530976,
        -0.31901765024691225
    ]
]
}
r.move_joint(**joint_property)

```

3. Move linear

```

import time
from neurapy.robot import Robot

r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

# moving the robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            -0.0007546,
            0.2713685,
            1.2664022,
            -0.0007550,
            1.6046191,
            -0.0007501,
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)

linear_property = {
    "speed": 0.9,
    "acceleration": 0.2,

```

(continues on next page)

(continued from previous page)

```

"target_pose": [
    [
        0.521504613338733,
        -0.0005120121635832758,
        0.4429621801014548,
        3.140838623046875,
        -0.0007978816283866763,
        3.1415627002716064
    ],
    [
        0.5216067833501538,
        -0.21429644443731097,
        0.44311804388931564,
        3.140573024749756,
        -0.0008633044781163335,
        3.1414897441864014
    ],
    [
        0.30336607796720333,
        -0.2139397968890111,
        0.44282379715713066,
        3.140761613845825,
        -0.0011759602930396795,
        3.1410810947418213
    ],
    [
        0.3033097863631349,
        0.19367482201823044,
        0.44268805519921206,
        -3.1399381160736084,
        -0.001531908637844026,
        3.141218662261963
    ],
    [
        0.5376223865405412,
        0.19093033055550618,
        0.44330028363322316,
        -3.1380574703216553,
        -0.0019246124429628253,
        3.1399550437927246
    ],
    [
        0.521504613338733,
        -0.0005120121635832758,
        0.4429621801014548,
        3.140838623046875,
        -0.0007978816283866763,
        3.1415627002716064
    ]
],
"current_joint_angles": r.get_current_joint_angles()
}
r.move_linear(**linear_property)
r.stop()

```

4. Move circular

```

import time
from neurapy.robot import Robot

r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        0.3223498, 0.170158, 1.3894271, -0.0039065, 1.5862455, -0.
→ 0.013291
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "target_pose": [
        [
            0.3744609827431085,
            -0.3391784988266481,
            0.23276604279256016,
            3.14119553565979,
            -0.00017731254047248513,
            -0.48800110816955566
        ],
        [
            0.37116786741831503,
            -0.19686307684994242,
            0.23300456855796453,
            3.141423225402832,
            -0.00020668463548645377,
            -0.48725831508636475
        ],
        [
            0.5190337951593321,
            -0.1969996948428492,
            0.23267853691809767,
            3.1414194107055664,
            -0.00017726201622281224,
            -0.48750609159469604
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_circular(**circular_property)
r.stop()

```

5. Move composite

```

import time
from neurapy.robot import Robot

```

(continues on next page)

(continued from previous page)

```

r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [0.9855635563580816, 0.3476807153487805, 1.1687323223466928, -0.
↪ 0007014516639916519, 1.6256822043262882, -0.000683475326633617]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Composite
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [ {
        "linear": {
            "blend_radius": 0.005,

            "targets": [
                [
                    0.30323030824121955,
                    0.4573874594177961,
                    0.44309265860894687,
                    3.140892505645752,
                    -0.0005030535394325852,
                    -2.155384063720703
                ],
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,
                    3.1414873600006104,
                    -0.0014394369209185243,
                    -3.0334575176239014
                ]
            ]
        }
    },
    {
        "circular": {
            "targets": [
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,
                    3.1414873600006104,

```

(continues on next page)

(continued from previous page)

```

-0.0014394369209185243,
-3.0334575176239014
],
[
0.4708356936660009,
0.280180550729363,
0.4431937440044086,
3.1407277584075928,
-0.002278585685417056,
-2.601893424987793
],
[
0.30323030824121955,
0.4573874594177961,
0.44309265860894687,
3.140892505645752,
-0.0005030535394325852,
-2.155384063720703
]
]
}
}
}

r.move_composite(**composite_motion_property)
r.stop()

```

6. Move recorded path

```

from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "file_name": "<path to file loaction>",
    "speed": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
r.move_recorded_path(**recorded_path_property)

```

6.3 Examples for Lara 10

1. Robot Object Initialization

```

from neurapy.robot import Robot

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.platform)
print(r.payload)
print(r.kURL)
print(r.robot_urdf_path)

```

(continues on next page)

(continued from previous page)

```
print(r.current_tool)
print(r.connection)
print(r.version)
```

2. Move joint

```
from neurapy.robot import Robot

r = Robot()
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            0.7740276502469322,
            0.13599234975308788,
            1.2223307505101257,
            -0.31901765024691225,
            0.3242476502469122,
            -0.31901765024691225
        ],
        [
            0.30333999999999633,
            -0.5334353004938217,
            1.4419907505101492,
            -0.31901765024691225,
            0.3242476502469122,
            -0.31901765024691225
        ],
        [
            0.059259210493824405,
            0.08133676975309524,
            1.4419907505101492,
            -1.0084715804938613,
            -0.7008323497530976,
            -0.31901765024691225
        ]
    ]
}
r.move_joint(**joint_property)
```

3. Move linear

```
import time
from neurapy.robot import Robot

r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

# moving the robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            -0.0007546,
```

(continues on next page)

(continued from previous page)

```

        0.2713685,
        1.2664022,
        -0.0007550,
        1.6046191,
        -0.0007501,
    ],
],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)

linear_property = {
    "speed": 0.9,
    "acceleration": 0.2,
    "target_pose": [
        [
            0.521504613338733,
            -0.0005120121635832758,
            0.4429621801014548,
            3.140838623046875,
            -0.0007978816283866763,
            3.1415627002716064
        ],
        [
            0.5216067833501538,
            -0.21429644443731097,
            0.44311804388931564,
            3.140573024749756,
            -0.0008633044781163335,
            3.1414897441864014
        ],
        [
            0.30336607796720333,
            -0.2139397968890111,
            0.44282379715713066,
            3.140761613845825,
            -0.0011759602930396795,
            3.1410810947418213
        ],
        [
            0.3033097863631349,
            0.19367482201823044,
            0.44268805519921206,
            -3.1399381160736084,
            -0.001531908637844026,
            3.141218662261963
        ],
        [
            0.5376223865405412,
            0.19093033055550618,
            0.44330028363322316,
            -3.1380574703216553,
            -0.0019246124429628253,
            3.1399550437927246
        ]
    ],

```

(continues on next page)

(continued from previous page)

```

        [
            0.521504613338733,
            -0.0005120121635832758,
            0.4429621801014548,
            3.140838623046875,
            -0.0007978816283866763,
            3.1415627002716064
        ],
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_linear(**linear_property)
r.stop()

```

4. Move circular

```

import time
from neurapy.robot import Robot

r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        0.3223498, 0.170158, 1.3894271, -0.0039065, 1.5862455, -0.
↪0013291
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)
circular_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "target_pose": [
        [
            0.6003694497970715, 0.14032636478337754, 0.5562804129025449, ↪
↪-3.141592653589793, -0.04762987720513421, -2.9119813881923298
        ],
        [
            0.2915217606232166, 0.5432771189605686, 0.5562804129025449, -
↪3.141592653589793, -0.047629877205134126, -2.063292395583077
        ],
        [
            -0.21026237167017361, 0.5795901137550482, 0.5562804129025449, ↪
↪3.1415926535897927, -0.04762987720513417, -1.2227839090482988
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_circular(**circular_property)
r.stop()

```

5. Move composite

```

import time
from neurapy.robot import Robot

r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

# Move to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [0.9855635563580816, 0.3476807153487805, 1.1687323223466928, -0.
→ 0007014516639916519, 1.6256822043262882, -0.000683475326633617]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Composite
composite_motion_property = {
    "speed": 0.25,
    "acceleration": 0.1,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [ {
        "linear": {
            "blend_radius": 0.005,

            "targets": [
                [
                    0.30323030824121955,
                    0.4573874594177961,
                    0.44309265860894687,
                    3.140892505645752,
                    -0.0005030535394325852,
                    -2.155384063720703
                ],
                [
                    0.5452084494942819,
                    0.058104230123491495,
                    0.44311138849670073,
                    3.1414873600006104,
                    -0.0014394369209185243,
                    -3.0334575176239014
                ]
            ]
        }
    },
    {
        "circular": {
            "targets": [
                [
                    0.5452084494942819,

```

(continues on next page)

(continued from previous page)

```

0.058104230123491495,
0.44311138849670073,
3.1414873600006104,
-0.0014394369209185243,
-3.0334575176239014
],
[
0.4708356936660009,
0.280180550729363,
0.4431937440044086,
3.1407277584075928,
-0.002278585685417056,
-2.601893424987793
],
[
0.30323030824121955,
0.4573874594177961,
0.44309265860894687,
3.140892505645752,
-0.0005030535394325852,
-2.155384063720703
]
]
}
}
}

r.move_composite(**composite_motion_property)
r.stop()

```

6. Move recorded path

```

from neurapy.robot import Robot

r = Robot()
recorded_path_property = {
    "file_name": "<path to file location>",
    "speed": 0.25,
    "current_joint_angles": r.get_current_joint_angles(),
}
r.move_recorded_path(**recorded_path_property)

```

6.4 Examples for Maira 7

1. Move linear

```

from neurapy.robot import Robot
import time

# Initializing the robot and setting it to Automatic mode
r = Robot()
r.switch_to_automatic_mode()

```

(continues on next page)

(continued from previous page)

```

time.sleep(1)

# Moving the robot to Initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            0,0,1.5708001839725434,0,1.5708001839725434,0
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)

#Linear motion
linear_property = {
    "speed": 0.9,
    "acceleration": 0.2,
    "target_pose": [
        [
            -0.14602202042158488,
            -1.2039712892654788,
            0.5542529402524848,
            -3.1250412464141846,
            -0.05224483087658882,
            -1.030046820640564
        ],
        [
            0.32102659790371435,
            -1.2039027313771722,
            0.5543345835272986,
            -3.1251614093780518,
            -0.05228979140520096,
            -1.030221700668335
        ],
        [
            0.3214140596556481,
            -0.8018749844104675,
            0.55438618778188,
            -3.1251306533813477,
            -0.052478402853012085,
            -1.0297577381134033
        ],
        [
            -0.2883707227945903,
            -0.8016352570186388,
            0.5542651616343833,
            -3.125182628631592,
            -0.052266936749219894,
            -1.0304018259048462
        ],
        [
            -0.14602202042158488,
            -1.2039712892654788,
            0.5542529402524848,

```

(continues on next page)

(continued from previous page)

```

        -3.1250412464141846,
        -0.05224483087658882,
        -1.030046820640564
    ],
    ],
    "current_joint_angles":r.get_current_joint_angles()
}
r.move_linear(**linear_property)
r.stop()

```

2. Move circular

```

from neurapy.robot import Robot
import time

# Initializing the robot and setting it to Automatic mode
r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)

#move robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            0,0,1.5708001839725434,0,1.5708001839725434,0
        ]
    ],
    "current_joint_angles":r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Circular
circular_property = {
    "speed": 1.0,
    "acceleration": 0.1,
    "target_pose": [
        [
            0.8827123230328554,
            -0.4975248049417001,
            0.5826623081385038,
            3.135085105895996,
            -0.06406070291996002,
            0.655619740486145
        ],
        [
            -0.03171114438156373,
            -1.0127658173803937,
            0.58266951895531,
            3.1350698471069336,
            -0.064027339220047,
            -0.43322843313217163
        ],
    ],
    [

```

(continues on next page)

(continued from previous page)

```

        -0.7324668416327569,
        -0.7001335665738433,
        0.5826737224363503,
        3.1350419521331787,
        -0.06396733969449997,
        -1.209897518157959
    ],
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_circular(**circular_property)
r.stop()

```

3. Move composite

```

from neurapy.robot import Robot
import time

# Initializing the robot and setting it to Automatic mode
r = Robot()
r.switch_to_automatic_mode()
time.sleep(1)
# Move robot to initial position
joint_property = {
    "speed": 50.0,
    "acceleration": 50.0,
    "target_joint": [
        [
            0, 0, 1.5708001839725434, 0, 1.5708001839725434, 0
        ]
    ],
    "current_joint_angles": r.get_current_joint_angles()
}
r.move_joint(**joint_property)
time.sleep(1)

# Move Composite
composite_motion_property = {
    "speed": 0.432,
    "acceleration": 0.1,
    "current_joint_angles": r.get_current_joint_angles(),
    "commands": [ {
        "linear": {
            "blend_radius": 0.005,

            "targets": [
                [
                    0.8827123230328554,
                    -0.4975248049417001,
                    0.5826623081385038,
                    3.135085105895996,
                    -0.06406070291996002,
                    0.655619740486145
                ],
                [
                    -0.7324668416327569,

```

(continues on next page)

(continued from previous page)

```

-0.7001335665738433,
0.5826737224363503,
3.1350419521331787,
-0.06396733969449997,
-1.209897518157959
    ]
    }
},
{
    "circular": {
        "targets": [
            [
                -0.7324668416327569,
                -0.7001335665738433,
                0.5826737224363503,
                3.1350419521331787,
                -0.06396733969449997,
                -1.209897518157959
            ],
            [
                -0.03171114438156373,
                -1.0127658173803937,
                0.58266951895531,
                3.1350698471069336,
                -0.064027339220047,
                -0.43322843313217163
            ],
            [
                0.8827123230328554,
                -0.4975248049417001,
                0.5826623081385038,
                3.135085105895996,
                -0.06406070291996002,
                0.655619740486145
            ]
        ]
    }
},
]
}
r.move_composite(**composite_motion_property)
r.stop()

```


SAMPLE APPLICATIONS

7.1 Pick and place for Maira

1. Example with points defined in GUI

 Warning:

Add pickup point Pi0 and place point Pl0 in GUI. Also add points Pi1 and Pl1 10 cm above pickup and place point respectively.

```
from neurapy.robot import Robot
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.5)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "Pl1"])
r.set_override(0.5)
r.move_linear(["Pl1", "Pl0"])
r.release()
time.sleep(1)
r.move_linear(["Pl0", "Pl1"])
r.set_override(1)
r.move_linear(["Pl1", "Home"])
```

2. Example using move fuctions with points defined in Neurapy

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi] #define placing point,
↪ here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos,
↪ r.get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos,
↪ r.get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos, pick_pos])
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos, offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos, place_pos])
r.release()
time.sleep(1)
r.move_linear(target_pose = [place_pos, offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

3. Example with points saved using Neurapy

```

from neurapy.robot import Robot
import numpy as np
import time

```

(continues on next page)

(continued from previous page)

```

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi] #define placing point
↪ here

"""
A function to create four points according to the pickup and placing
↪ point.
"""
def save_point(target_end_effector_pose, target, reference_joint_angles =
↪ r.get_point('Home', representation='Joint')):
    for i in range(2):

        if target == "pick":
            name = "Pi" + str(i)
        else:
            name = "Pl" + str(i)

        r.create_point(name = name, target_end_effector_pose = target_
↪ end_effector_pose)
        target_end_effector_pose[2] = target_end_effector_pose[2] + 0.1
    return True

save_point(pick_pos, "pick")
save_point(place_pos, "place")

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.5)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "Pl1"])
r.set_override(0.5)
r.move_linear(["Pl1", "Pl0"])
r.release()
time.sleep(1)
r.move_linear(["Pl0", "Pl1"])
r.set_override(1)
r.move_linear(["Pl1", "Home"])

```

4. Example with controlling gripper action through IOs

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi] #define placing point, ↪
↪here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos, ↪
↪r.get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos,
↪r.get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos,pick_pos])
r.wait(port_name="DI_2", expected_value=1.0, wait_for_signal=True, wait_
↪time=0.0) #waits till digital input 2 is True
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos,offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos,place_pos])
r.release()
time.sleep(1)
r.set_digital_output(1,True) # set digital output 1 to true after release
r.move_linear(target_pose = [place_pos,offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

5. Example using planner

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

pick_pos = [0.36, 0.36, 0.5, np.pi, 0, np.pi]
place_pos = [0.36, -0.36, 0.5, np.pi, 0, np.pi]
offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1
offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')

plan_id_1 = r.plan_move_linear(target_pose =[home,offset_pick_pos,pick_
↪pos],store_id=1)
plan_id_2 = r.plan_move_linear(target_pose =[pick_pos,offset_pick_pos,
↪offset_place_pos,place_pos],store_id=2)
plan_id_3 = r.plan_move_linear(target_pose =[place_pos,offset_place_pos,
↪home],store_id=3)

execute_motion = r.executor([plan_id_1]) #To execute the planned id
r.grasp()
time.sleep(1)
execute_motion = r.executor([plan_id_2])
r.release()
time.sleep(1)
execute_motion = r.executor([plan_id_3])

```

6. Example with pick and place function in 4 different position

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')
pick_pos = [
    [0.36, 0.36, 0.5, np.pi, 0, np.pi],
    [0.45, 0.45, 0.5, np.pi, 0, np.pi],
    [0.36, 0.45, 0.5, np.pi, 0, np.pi],
    [0.45, 0.36, 0.5, np.pi, 0, np.pi]
]

place_pos = [
    [0.36, -0.36, 0.5, np.pi, 0, np.pi],
    [0.45, -0.45, 0.5, np.pi, 0, np.pi],
    [0.36, -0.45, 0.5, np.pi, 0, np.pi],
    [0.45, -0.36, 0.5, np.pi, 0, np.pi]
]

```

(continues on next page)

(continued from previous page)

```

def planner(pick_pos, place_pos):
    offset_pick_pos = copy.deepcopy(pick_pos)
    offset_pick_pos[2] = offset_pick_pos[2] + 0.1
    offset_place_pos = copy.deepcopy(place_pos)
    offset_place_pos[2] = offset_place_pos[2] + 0.1
    plan_id = [0,0,0]
    plan_id[0] = r.plan_move_linear(target_pose = [home, offset_pick_pos,
↪ pick_pos])
    plan_id[1] = r.plan_move_linear(target_pose = [pick_pos, offset_pick_
↪ pos, offset_place_pos, place_pos])
    plan_id[2] = r.plan_move_linear(target_pose = [place_pos, offset_place_
↪ pos, home])
    return plan_id

def execute_pp(plan_id):
    execute_motion = r.executor([plan_id[0]]) #To execute the planned id
    r.grasp()
    time.sleep(1)
    execute_motion = r.executor([plan_id[1]])
    r.release()
    time.sleep(1)
    execute_motion = r.executor([plan_id[2]])
    return True

for i in range(len(pick_pos)):
    plan_id = planner(pick_pos[i], place_pos[i])
    execute_pp(plan_id)

```

7. Example with simultaneous planning and execution

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.36, 0.36, 0.5, np.pi, 0, np.pi],
    [0.45, 0.45, 0.5, np.pi, 0, np.pi],
    [0.36, 0.45, 0.5, np.pi, 0, np.pi],
    [0.45, 0.36, 0.5, np.pi, 0, np.pi]
]

place_pos = [
    [0.36, -0.36, 0.5, np.pi, 0, np.pi],
    [0.45, -0.45, 0.5, np.pi, 0, np.pi],
    [0.36, -0.45, 0.5, np.pi, 0, np.pi],
    [0.45, -0.36, 0.5, np.pi, 0, np.pi]
]

```

(continues on next page)

(continued from previous page)

```

]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()

def planner(pick_pos, place_pos):
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_
↪pos, pick_pos[i]],current_joint_angles= home_joint, store_id=base_
↪number + (3 * i + 1))
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
↪pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
↪joint, store_id=base_number + 1 + (3 * i + 1))
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i],
↪offset_place_pos, home],current_joint_angles= home_joint, store_
↪id=base_number + 2 + (3 * i + 1))

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_
↪id}\033[0m", flush=True)

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if plan_ids:
                plan_id = plan_ids.pop(0)
            else:
                if planning_thread.is_alive():
                    continue
                else:
                    break
            print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↪flush=True)
            execute_motion = r.executor([plan_id[0]]) # To execute the
↪planned id
            print("Executed plan 1", flush=True)
            r.grasp()
            time.sleep(1)
            execute_motion = r.executor([plan_id[1]])
            print("Executed plan 2", flush=True)
            r.release()
            time.sleep(1)
            execute_motion = r.executor([plan_id[2]])
            print("Executed plan 3", flush=True)

            idx = idx + 1

```

(continues on next page)

(continued from previous page)

```

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

# Create and start execution thread
execute_plan()

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

8. Example with reusing IDs for repeated motion.

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.36, 0.36, 0.5, np.pi, 0, np.pi],
    [0.45, 0.45, 0.5, np.pi, 0, np.pi],
    [0.36, 0.45, 0.5, np.pi, 0, np.pi],
    [0.45, 0.36, 0.5, np.pi, 0, np.pi]
]

place_pos = [
    [0.36, -0.36, 0.5, np.pi, 0, np.pi],
    [0.45, -0.45, 0.5, np.pi, 0, np.pi],
    [0.36, -0.45, 0.5, np.pi, 0, np.pi],
    [0.45, -0.36, 0.5, np.pi, 0, np.pi]
]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()
planning_complete = threading.Event()

def planner(pick_pos, place_pos):
    global planning_complete
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

```

(continues on next page)

(continued from previous page)

```

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_
↪pos, pick_pos[i]],current_joint_angles= home_joint, store_id=base_
↪number + (3 * i + 1),reusable = True)
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
↪pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
↪joint, store_id=base_number +1 + (3 * i + 1),reusable = True)
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i],
↪offset_place_pos, home],current_joint_angles= home_joint, store_
↪id=base_number +2 + (3 * i + 1),reusable = True)

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_
↪id}\033[0m", flush=True)
            planning_complete.set()

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if idx<=3:
                if plan_ids:
                    plan_id = plan_ids[idx]
                else:
                    continue
            else:
                break

        print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↪flush=True)
        execute_motion = r.executor([plan_id[0]]) # To execute the_
↪planned id
        print("Executed motion 1", flush=True)
        r.grasp()
        time.sleep(1)
        execute_motion = r.executor([plan_id[1]])
        print("Executed motion 2", flush=True)
        r.release()
        time.sleep(1)
        execute_motion = r.executor([plan_id[2]])
        print("Executed motion 3", flush=True)

        idx = idx +1

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

iter = 5 # number of loops,specify it as required

# Create and start execution thread
for i in range(iter):
    execute_plan()

```

(continues on next page)

(continued from previous page)

```

print(f'\033[92mExecuted the plan for {i+1} iteration.\033[0m')

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

7.2 Pick and place for Lara

1. Example with points defined in GUI

Warning:

Add pickup point *Pi0* and place point *Pi0* in GUI. Also add points *Pi1* and *Pi1* 10 cm above pickup and place point respectively.

```

from neurapy.robot import Robot
import time

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.2)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "P11"])
r.set_override(0.2)
r.move_linear(["P11", "P10"])
r.release()
time.sleep(1)
r.move_linear(["P10", "P11"])
r.set_override(1)
r.move_linear(["P11", "Home"])

```

2. Example using move fuctions with points defined in Neurapy

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

```

(continues on next page)

(continued from previous page)

```

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi] #define placing point ↵
↵here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos, ↵
↵r.get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos,
↵r.get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos,pick_pos])
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos,offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos,place_pos])
r.release()
time.sleep(1)
r.move_linear(target_pose = [place_pos,offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

3. Example with points saved using Neurapy

```

from neurapy.robot import Robot
import numpy as np
import time

r = Robot()
print(r.robot_name)

```

(continues on next page)

(continued from previous page)

```

print(r.dof)
print(r.payload)
r.power_on()

pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi] #define placing point
↪here

"""
A function to create four points according to the pickup and placing
↪point.
"""
def save_point(target_end_effector_pose, target, reference_joint_angles =
↪r.get_point('Home', representation='Joint')):
    for i in range(2):

        if target == "pick":
            name = "Pi" + str(i)
        else:
            name = "Pl" + str(i)
        r.create_point(name = name, target_end_effector_pose = target_
↪end_effector_pose)
        target_end_effector_pose[2] = target_end_effector_pose[2] + 0.1
    return True

save_point(pick_pos, "pick")
save_point(place_pos, "place")

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()
r.set_joint_speed(50)
r.set_joint_acceleration(50)
r.move_joint("Home")
r.move_linear(["Home", "Pi1"])
r.set_override(0.2)
r.move_linear(["Pi1", "Pi0"])
r.grasp()
time.sleep(1)
r.move_linear(["Pi0", "Pi1"])
r.set_override(1)
r.move_linear(["Pi1", "Pl1"])
r.set_override(0.2)
r.move_linear(["Pl1", "Pl0"])
r.release()
time.sleep(1)
r.move_linear(["Pl0", "Pl1"])
r.set_override(1)
r.move_linear(["Pl1", "Home"])

```

4. Example with controlling gripper action through IOs

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

```

(continues on next page)

(continued from previous page)

```

r = Robot()
print(r.robot_name)
print(r.dof)
print(r.payload)
r.power_on()

#Defining points
pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi] #define pickup point here
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi] #define placing point
↪here

offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1

offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1

#calculating Inverse kinematics for move_joint()
joint_angle_offset_pick = r.compute_inverse_kinematics(offset_pick_pos,
↪r.get_current_joint_angles())
joint_angle_offset_place = r.compute_inverse_kinematics(offset_place_pos,
↪r.get_current_joint_angles())

if r.is_robot_in_teach_mode():
    r.switch_to_automatic_mode()

#picking
r.move_joint(joint_angle_offset_pick)
r.move_linear(target_pose = [offset_pick_pos,pick_pos])
r.wait(port_name="DI_2", expected_value=1.0, wait_for_signal=True, wait_
↪time=0.0) #waits till digital input 2 is True
r.grasp()
time.sleep(1)
r.move_linear(target_pose = [pick_pos,offset_pick_pos])

#placing
r.move_joint(joint_angle_offset_place)
r.move_linear(target_pose = [offset_place_pos,place_pos])
r.release()
time.sleep(1)
r.set_digital_output(1,True) # set digital output 1 to true after release
r.move_linear(target_pose = [place_pos,offset_place_pos])

#default position
r.move_joint('Home')

r.power_off()

```

5. Example using planner

```

from neurapy.robot import Robot
import numpy as np
import copy

```

(continues on next page)

(continued from previous page)

```

import time

pick_pos = [0.32, 0.32, 0.2, np.pi, 0, np.pi]
place_pos = [0.32, -0.32, 0.2, np.pi, 0, np.pi]
offset_pick_pos = copy.deepcopy(pick_pos)
offset_pick_pos[2] = offset_pick_pos[2] + 0.1
offset_place_pos = copy.deepcopy(place_pos)
offset_place_pos[2] = offset_place_pos[2] + 0.1
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')

plan_id_1 = r.plan_move_linear(target_pose=[home,offset_pick_pos,pick_
↪pos],store_id=1)
plan_id_2 = r.plan_move_linear(target_pose=[pick_pos,offset_pick_pos,
↪offset_place_pos,place_pos],store_id=2)
plan_id_3 = r.plan_move_linear(target_pose=[place_pos,offset_place_pos,
↪home],store_id=3)

execute_motion = r.executor([plan_id_1]) #To execute the planned id
r.grasp()
time.sleep(1)
execute_motion = r.executor([plan_id_2])
r.release()
time.sleep(1)
execute_motion = r.executor([plan_id_3])

```

6. Example with pick and place function in 4 different position

```

from neurapy.robot import Robot
import numpy as np
import copy
import time

r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation = 'Cartesian')
pick_pos = [
    [0.32, 0.32, 0.2, np.pi, 0, np.pi],
    [0.31, 0.31, 0.2, np.pi, 0, np.pi],
    [0.32, 0.31, 0.2, np.pi, 0, np.pi],
    [0.31, 0.32, 0.2, np.pi, 0, np.pi]
]

place_pos = [
    [0.32, -0.32, 0.2, np.pi, 0, np.pi],
    [0.31, -0.31, 0.2, np.pi, 0, np.pi],
    [0.32, -0.31, 0.2, np.pi, 0, np.pi],
    [0.31, -0.32, 0.2, np.pi, 0, np.pi]
]

def planner(pick_pos,place_pos):
    offset_pick_pos = copy.deepcopy(pick_pos)

```

(continues on next page)

(continued from previous page)

```

        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos)
        offset_place_pos[2] = offset_place_pos[2] + 0.1
        plan_id= [0,0,0]
        plan_id[0] = r.plan_move_linear(target_pose =[home,offset_pick_pos,
↪pick_pos])
        plan_id[1] = r.plan_move_linear(target_pose =[pick_pos,offset_pick_
↪pos,offset_place_pos,place_pos])
        plan_id[2] =r.plan_move_linear(target_pose =[place_pos,offset_place_
↪pos,home])
        return plan_id

def execute_pp(plan_id):
    execute_motion = r.executor([plan_id[0]]) #To execute the planned id
    r.grasp()
    time.sleep(1)
    execute_motion = r.executor([plan_id[1]])
    r.release()
    time.sleep(1)
    execute_motion = r.executor([plan_id[2]])
    return True

for i in range(len(pick_pos)):
    plan_id = planner(pick_pos[i],place_pos[i])
    execute_pp(plan_id)

```

7. Example with simultaneous planning and execution

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.32, 0.32, 0.2, np.pi, 0, np.pi],
    [0.31, 0.31, 0.2, np.pi, 0, np.pi],
    [0.32, 0.31, 0.2, np.pi, 0, np.pi],
    [0.31, 0.32, 0.2, np.pi, 0, np.pi]
]

place_pos = [
    [0.32, -0.32, 0.2, np.pi, 0, np.pi],
    [0.31, -0.31, 0.2, np.pi, 0, np.pi],
    [0.32, -0.31, 0.2, np.pi, 0, np.pi],
    [0.31, -0.32, 0.2, np.pi, 0, np.pi]
]

```

(continues on next page)

(continued from previous page)

```

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()

def planner(pick_pos, place_pos):
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)
        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_
↪pos, pick_pos[i]],current_joint_angles= home_joint, store_id=base_
↪number + (3 * i + 1))
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
↪pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
↪joint, store_id=base_number + 1 + (3 * i + 1))
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i],
↪offset_place_pos, home],current_joint_angles= home_joint, store_
↪id=base_number + 2 + (3 * i + 1))

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_
↪id}\033[0m", flush=True)

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if plan_ids:
                plan_id = plan_ids.pop(0)
            else:
                if planning_thread.is_alive():
                    continue
                else:
                    break
            print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↪flush=True)
            execute_motion = r.executor([plan_id[0]]) # To execute the
↪planned id
            print("Executed plan 1", flush=True)
            r.grasp()
            time.sleep(1)
            execute_motion = r.executor([plan_id[1]])
            print("Executed plan 2", flush=True)
            r.release()
            time.sleep(1)
            execute_motion = r.executor([plan_id[2]])
            print("Executed plan 3", flush=True)

            idx = idx + 1

# Create and start planning thread

```

(continues on next page)

(continued from previous page)

```

planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

# Create and start execution thread
execute_plan()

# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)

```

8. Example with reusing IDs for repeated motion.

```

import threading
from neurapy.robot import Robot
import numpy as np
import copy
import time
import random

# Initialize the robot
r = Robot()
r.switch_to_automatic_mode()
home = r.get_point('Home', representation='Cartesian')
home_joint = r.get_point('Home', representation='Joint')

# Define pick and place positions
pick_pos = [
    [0.32, 0.32, 0.2, np.pi, 0, np.pi],
    [0.31, 0.31, 0.2, np.pi, 0, np.pi],
    [0.32, 0.31, 0.2, np.pi, 0, np.pi],
    [0.31, 0.32, 0.2, np.pi, 0, np.pi]
]

place_pos = [
    [0.32, -0.32, 0.2, np.pi, 0, np.pi],
    [0.31, -0.31, 0.2, np.pi, 0, np.pi],
    [0.32, -0.31, 0.2, np.pi, 0, np.pi],
    [0.31, -0.32, 0.2, np.pi, 0, np.pi]
]

# Shared list to store plan IDs
plan_ids = []
plan_ids_lock = threading.Lock()
planning_complete = threading.Event()

def planner(pick_pos, place_pos):
    global planning_complete
    for i in range(len(pick_pos)):
        offset_pick_pos = copy.deepcopy(pick_pos[i])
        offset_pick_pos[2] = offset_pick_pos[2] + 0.1
        offset_place_pos = copy.deepcopy(place_pos[i])
        offset_place_pos[2] = offset_place_pos[2] + 0.1

        plan_id = [0, 0, 0]
        base_number = random.randint(1000, 9999)

```

(continues on next page)

(continued from previous page)

```

        plan_id[0] = r.plan_move_linear(target_pose=[home, offset_pick_
↪pos, pick_pos[i]],current_joint_angles= home_joint, store_id=base_
↪number + (3 * i + 1),reusable = True)
        plan_id[1] = r.plan_move_linear(target_pose=[pick_pos[i], offset_
↪pick_pos, offset_place_pos, place_pos[i]],current_joint_angles= home_
↪joint, store_id=base_number +1 + (3 * i + 1),reusable = True)
        plan_id[2] = r.plan_move_linear(target_pose=[place_pos[i],
↪offset_place_pos, home],current_joint_angles= home_joint, store_
↪id=base_number +2 + (3 * i + 1),reusable = True)

        with plan_ids_lock:
            plan_ids.append(plan_id)
            print(f"\033[92mPlanned motion {i+1}/{len(pick_pos)}: {plan_
↪id}\033[0m", flush=True)
            planning_complete.set()

def execute_plan():
    idx = 0
    while True:
        with plan_ids_lock:
            if idx<=3:
                if plan_ids:
                    plan_id = plan_ids[idx]
                else:
                    continue
            else:
                break

        print(f"\033[92mExecuting plan {idx+1}: {plan_id}\033[0m",
↪flush=True)
        execute_motion = r.executor([plan_id[0]]) # To execute the
↪planned id
        print("Executed motion 1", flush=True)
        r.grasp()
        time.sleep(1)
        execute_motion = r.executor([plan_id[1]])
        print("Executed motion 2", flush=True)
        r.release()
        time.sleep(1)
        execute_motion = r.executor([plan_id[2]])
        print("Executed motion 3", flush=True)

        idx = idx +1

# Create and start planning thread
planning_thread = threading.Thread(target=planner, args=(pick_pos, place_
↪pos))
planning_thread.start()

iter = 5 # number of loops,specify it as required

# Create and start execution thread
for i in range(iter):
    execute_plan()
    print(f'\033[92mExecuted the plan for {i+1} iteration.\033[0m')

```

(continues on next page)

(continued from previous page)

```
# Wait for thread to complete
planning_thread.join()
print("\033[92mAll motions planned and executed.\033[0m", flush=True)
```

7.3 External Device Applications Examples

1. Creating external device object (using filename)

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
# Using just the filename (assumes file is in ~/data/process/)
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
```

2. Creating external device object (using full path)

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
# Using full path to the configuration file
robotiq_gripper = r.create_external_device(file_path="/home/hrg/data/
↳process/Robotiq2f85.json")
```

3. Creating external device object with ignore_no_connection=True

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object with ignore_no_
↳connection=True
# This will continue execution even if connection fails
welding_source = r.create_external_device(file_path="iROB401.json",
↳ignore_no_connection=True)
```

4. Connect to external device

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
# When connecting the "init" sequence will also be executed
robotiq_gripper.connect()
```

5. Connect to external device without creating ExternalDevice object

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Connect to external device using the robot's connect method
is_connected = r.connect_external_device(file_path="Robotiq2f85.json")
if is_connected:
    print("Successfully connected to external device")
```

(continues on next page)

(continued from previous page)

```

else:
    print("Failed to connect to external device")

```

6. Set ignore_no_connection for an external device

```

from neurapy.robot import Robot
import time
r = Robot()
# Create the ExternalDeviceApp object for the welding device
welding_source = r.create_external_device(file_path="iROB401.json")
# Connect to the device
welding_source.connect()
# Sets the ignore_no_connection flag for the ExternalDeviceApp object
welding_source.set_ignore_no_connection(ignore_no_connection=True)

```

7. Disconnect external device

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
robotiq_gripper.connect()
robotiq_gripper.disconnect()

```

8. Disconnect external device without creating ExternalDevice object

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Disconnect from external device using the robot's disconnect method
success = r.disconnect_external_device(file_path="Robotiq2f85.json")
if success:
    print("Successfully disconnected from external device")
else:
    print("Failed to disconnect from external device")

```

9. Check if the external device is connected

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
robotiq_gripper.connect()
# Check if the external device is connected.
if(robotiq_gripper.is_connected()):
    print("Robotiq2F85 is connected")
else:
    print("Robotiq2F85 is not connected")

```

10. Check if external device is connected without creating ExternalDevice object

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Check if external device is connected using the robot's method

```

(continues on next page)

(continued from previous page)

```

is_connected = r.is_external_device_connected(file_path="Robotiq2f85.json")
↪")
if is_connected:
    print("External device is connected")
else:
    print("External device is not connected")

```

11. Execute a function on an external device

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
robotiq_gripper.connect()
# Execute the "grip" function with parameters
r.external_device_obj.execute_function(robotiq_gripper, function_name=
↪"grip", function_parameters = {"width":65535,"force":255,"speed":255})

```

12. Execute external device function without creating ExternalDevice object

```

#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Execute function on external device using the robot's method
# This method creates the device, connects, executes the function, and
↪returns the result
return_values = r.execute_external_device_function(
    file_path="Robotiq2f85.json",
    function_name="grip",
    function_parameters={"width":65535,"force":255,"speed":255}
)
print(f"Function executed successfully. Return values: {return_values}")

```

13. Read data from an external device

```

# Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Linak Lift
device = r.create_external_device(file_path="LC3_IC_Lift.json")
# Execute the "ReadStatus" function without parameters and save the
↪return values dict
return_data = device.execute_function("ReadStatus")
# Print the received data
print(f"Device '{device.device_name}' returned data:")
for key, value in return_data.items():
    print(f"{key} = {value}")

```

14. Execute safety stop for device

```

from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
# Connect to the device
robotiq_gripper.connect()

```

(continues on next page)

(continued from previous page)

```
# This Function executes the "SafetyStop" function for the device
robotiq_gripper.execute_safety_stop_function()
```

15. Execute safety stop for device without creating ExternalDevice object

```
from neurapy.robot import Robot
r = Robot()
# Execute safety stop on external device using the robot's method
success = r.execute_external_device_safety_stop(
    file_path="Robotiq2f85.json",
)
if success:
    print("Safety stop executed successfully")
else:
    print("Failed to execute safety stop")
```

16. Execute safety stop for all connected device

```
from neurapy.robot import Robot
r = Robot()
# This Function executes the "SafetyStop" function for all connected_
↪ devices
r.external_device_interface.safety_stop_all()
```

17. Check if external device is in error state

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
# Check if external device is in error state
is_in_error = robotiq_gripper.is_in_error()
if is_in_error:
    print("External device is in error state")
else:
    print("External device is not in error state")
```

18. Check if external device is in error state without creating ExternalDevice object

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Check if external device is in error state using the robot's method
is_in_error = r.is_external_device_in_error(file_path="Robotiq2f85.json")
if is_in_error:
    print("External device is in error state")
else:
    print("External device is not in error state")
```

19. Reset error in an external device.

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
```

(continues on next page)

(continued from previous page)

```
robotiq_gripper.connect()
# Execute the function used to reset the error in the external device as
↳ declared in the configuration file
robotiq_gripper.execute_reset_error_function()
```

20. Reset error in external device without creating ExternalDevice object

```
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Reset error in external device using the robot's method
success = r.reset_external_device_error(file_path="Robotiq2f85.json")
if success:
    print("Error reset successfully")
else:
    print("Failed to reset error")
```

21. Execute multiple functions in a row (spot welding example)

```
import time
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the welding device
# ignore_no_connection set true will continue execution even if
↳ connection fails
welding_source = r.create_external_device(file_path="iROB401.json",
↳ ignore_no_connection=True)
# Connect to the device
welding_source.connect()
# Set the welding parameters in the power source
r.external_device_obj.execute_function(welding_source, function_name=
↳ "job_mode", function_parameters = {"job": 1})
time.sleep(1)
# Start welding
r.external_device_obj.execute_function(welding_source, function_name=
↳ "start_weld")
# Execute welding motion
# Stop welding
r.external_device_obj.execute_function(welding_source, function_name=
↳ "stop_weld")
```

22. ModbusRTU Gripper example for external device

```
from neurapy.robot import Robot
import time
r = Robot()
# Create the external device application object for the Robotiq Gripper
robotiq_gripper = r.create_external_device(file_path="Robotiq2f85.json")
# Connect to the device
robotiq_gripper.connect()
if robotiq_gripper.is_connected():
    # gripper open
    r.external_device_obj.execute_function(robotiq_gripper, function_
↳ name="grip", function_parameters = {"width":0,"force":255,"speed":255})
    time.sleep(1)
    # gripper close
```

(continues on next page)

(continued from previous page)

```

    r.external_device_obj.execute_function(robotiq_gripper, function_
    ↪name="grip", function_parameters = {"width":65535,"force":255,"speed
    ↪":255})

```

23. ModbusTCP lift example for external device

```

from neurapy.robot import Robot
import time
r = Robot()
# Create the external device application for the Linear lift external_
↪axis using Linak.
lift_obj = r.create_external_device(file_path="LC3_IC_Lift.json")
# Connect to the device
lift_obj.connect()
# linear lift moves up
r.external_device_obj.execute_function(lift_obj, function_name=
↪"GoToPosition", function_parameters = {"current":251,"speed":"80",
↪"position":5000})
time.sleep(5)
# linear lift moves down
r.external_device_obj.execute_function(lift_obj, function_name=
↪"GoToPosition", function_parameters = {"current":251,"speed":"80",
↪"position":1000})

```

24. EthernetIP welding source example for external device (spot welding)

```

import time
#Initialize the robot object
from neurapy.robot import Robot
r = Robot()
# Create the external device application object for the welding source
# ignore_no_connection set true will continue execution even if_
↪connection fails
welding_source = r.create_external_device(file_path="iROB401.json",_
↪ignore_no_connection=True)
# Connect to the device
welding_source.connect()
# Set the welding parameters in the power source
r.external_device_obj.execute_function(welding_source, function_name=
↪"job_mode", function_parameters = {"job": 1})
time.sleep(1)
# Start welding
r.external_device_obj.execute_function(welding_source, function_name=
↪"start_weld")
# Execute welding motion
# Stop welding
r.external_device_obj.execute_function(welding_source, function_name=
↪"stop_weld")

```


TCP-FLANGE CONNECTION PIN OUTS FOR LARA



6.3.4.1. TCP Flange connection pin-out

External flange connections pin-out (12 pole M12 connector/ IEC 61076-2-101)

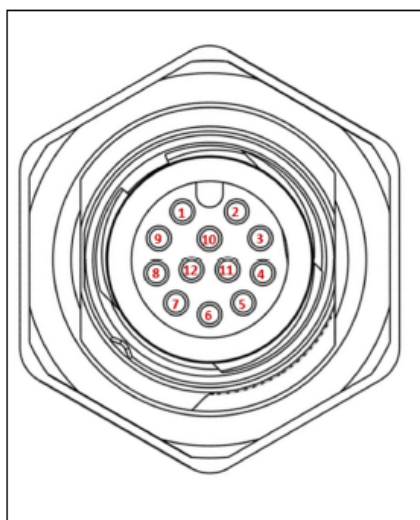


Fig. 25: IO ports, pin assignment of M12 connector (IEC 61076-2-101)

Pin-out of M12 connector for external connection is defined as follows:

Pin	Line	Cable color	Definition
1	Input_0	Brown	Digital input 0
2	Input_1	Blue	Digital input 1
3	Input_2	White	Digital input 2
4	Output_0	Green	Digital output 0
5	Output_1	Powder	Digital output 1
6	Output_2	Yellow	Digital output 2
7	RS485_A (TX+)	Black	RS484+ communication (Modbus RTU)
8	RS485_B (TX-)	Gray	RS484- communication (Modbus RTU)
9	AI 0	Red	Analog input 0
10	AI 1	Purple	Analog input 1
11	24 V	Ash powder (Grey/ Pink)	Power supply (24V)
12	GND	Red / Blue	Common Ground (0V)

Table 12: External wiring for M12 TCP Connector

TCP-FLANGE CONNECTION PIN OUTS FOR MAIRA



External flange connections

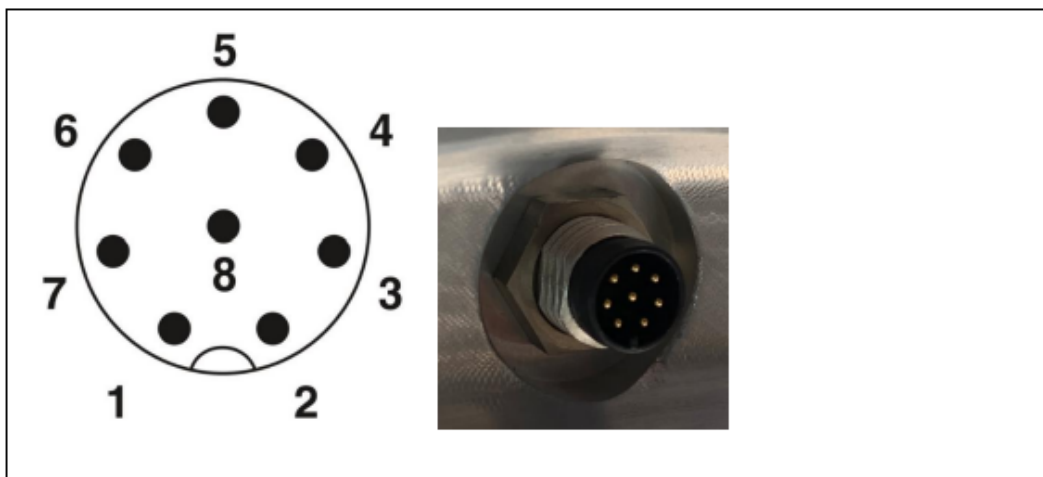


Fig. 29: M8 connectors, pin assignment (SACC-DSI-M8MS-8CON-M8/0,5)

IO external flange connections are defined as follows:

Pin	Line color	Definition	Notes
Port1			
1	White	TX+	
2	Brown	RX-	
3	Green	TX-	
4	Yellow	RX+	
5	Gray	+24V	
6	Pink		Not used
7	Blue		Not used
8	Red	GND	
Port 2			
1	White	DI_0	
2	Brown	DI_1	
3	Green	DO_0	
4	Yellow	DO_1	
5	Gray	+24V	
6	Pink	AI_0	
7	Blue	AI_1	
8	Red	GND	
Port 3			
1	White	RS485_A	
2	Brown	RS485_B	
3	Green		Not used
4	Yellow		Not used
5	Gray	+24V	
6	Pink		Not used
7	Blue		Not used
8	Red	GND	

PYTHON MODULE INDEX

n

`neurapy.robot`, [40](#)

A

activate_ethercat_interface() (neurapy.robot.Robot method), 40
 activate_ethernetIP_interface() (neurapy.robot.Robot method), 40
 activate_modbus_interface() (neurapy.robot.Robot method), 41
 activate_opcua_interface() (neurapy.robot.Robot method), 41
 activate_profinet_interface() (neurapy.robot.Robot method), 41
 activate_ros_interface() (neurapy.robot.Robot method), 42
 activate_servo_interface() (neurapy.robot.Robot method), 42

C

check_circular_feasibility() (neurapy.robot.Robot method), 42
 check_composite_feasibility() (neurapy.robot.Robot method), 47
 check_joint_feasibility() (neurapy.robot.Robot method), 53
 check_joint_relative_feasibility() (neurapy.robot.Robot method), 54
 check_linear_feasibility() (neurapy.robot.Robot method), 56
 check_linear_relative_feasibility() (neurapy.robot.Robot method), 58
 check_move_spline_feasibility() (neurapy.robot.Robot method), 59
 check_recorded_path_feasibility() (neurapy.robot.Robot method), 60
 compute_forward_kinematics() (neurapy.robot.Robot method), 61
 compute_inverse_kinematics() (neurapy.robot.Robot method), 61
 connect_external_device() (neurapy.robot.Robot method), 62
 convert_euler_to_quaternion_pose() (neurapy.robot.Robot method), 63
 convert_quaternion_to_euler_pose() (neurapy.robot.Robot method), 63
 copy_pose() (neurapy.robot.Robot method), 63
 create_external_device() (neurapy.robot.Robot method), 64

create_external_source_app() (neurapy.robot.Robot method), 64
 create_point() (neurapy.robot.Robot method), 65
 create_reference_frame() (neurapy.robot.Robot method), 66
 create_tool() (neurapy.robot.Robot method), 66

D

deactivate_ethercat_interface() (neurapy.robot.Robot method), 69
 deactivate_ethernetIP_interface() (neurapy.robot.Robot method), 69
 deactivate_modbus_interface() (neurapy.robot.Robot method), 69
 deactivate_opcua_interface() (neurapy.robot.Robot method), 69
 deactivate_profinet_interface() (neurapy.robot.Robot method), 70
 deactivate_ros_interface() (neurapy.robot.Robot method), 70
 deactivate_servo_interface() (neurapy.robot.Robot method), 70
 delete_point() (neurapy.robot.Robot method), 70
 delete_points_in_frame() (neurapy.robot.Robot method), 71
 delete_pre_planned_motion() (neurapy.robot.Robot method), 71
 delete_reference_frame() (neurapy.robot.Robot method), 71
 disable_collision_detection() (neurapy.robot.Robot method), 72
 disable_friction_mode() (neurapy.robot.Robot method), 72
 disable_reflex() (neurapy.robot.Robot method), 72
 disconnect_external_device() (neurapy.robot.Robot method), 72

E

enable_collision_detection() (neurapy.robot.Robot method), 73
 enable_friction_mode() (neurapy.robot.Robot method), 73
 enable_reflex() (neurapy.robot.Robot method), 74
 encoder2rad() (neurapy.robot.Robot method), 74
 execute_closest_orthogonal_tcp_align() (neurapy.robot.Robot method), 74

<code>execute_exact_tcp_align()</code> (<i>neurapy.robot.Robot method</i>), 75	<code>get_digital_output()</code> (<i>neurapy.robot.Robot method</i>), 84
<code>execute_external_device_function()</code> (<i>neurapy.robot.Robot method</i>), 75	<code>get_doc()</code> (<i>neurapy.robot.Robot method</i>), 84
<code>execute_external_device_safety_stop()</code> (<i>neurapy.robot.Robot method</i>), 76	<code>get_encoder_offsets()</code> (<i>neurapy.robot.Robot method</i>), 85
<code>execute_pre_plan_motion()</code> (<i>neurapy.robot.Robot method</i>), 76	<code>get_errors()</code> (<i>neurapy.robot.Robot method</i>), 85
<code>execute_pre_plan_motion_from_json()</code> (<i>neurapy.robot.Robot method</i>), 77	<code>get_ext_digital_input()</code> (<i>neurapy.robot.Robot method</i>), 85
<code>execute_program()</code> (<i>neurapy.robot.Robot method</i>), 77	<code>get_ext_digital_output()</code> (<i>neurapy.robot.Robot method</i>), 85
<code>execute_z_axis_tcp_align()</code> (<i>neurapy.robot.Robot method</i>), 78	<code>get_flange_pose()</code> (<i>neurapy.robot.Robot method</i>), 86
<code>executor()</code> (<i>neurapy.robot.Robot method</i>), 78	<code>get_flange_pose_quaternion()</code> (<i>neurapy.robot.Robot method</i>), 86
G	<code>get_flange_pose_quaternion_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 86
<code>get_analog_input()</code> (<i>neurapy.robot.Robot method</i>), 79	<code>get_flange_pose_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 86
<code>get_analog_output()</code> (<i>neurapy.robot.Robot method</i>), 79	<code>get_io_configuration()</code> (<i>neurapy.robot.Robot method</i>), 87
<code>get_commanded_joint_angles()</code> (<i>neurapy.robot.Robot method</i>), 79	<code>get_io_status()</code> (<i>neurapy.robot.Robot method</i>), 87
<code>get_commanded_joint_angles_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 80	<code>get_joint_acceleration()</code> (<i>neurapy.robot.Robot method</i>), 87
<code>get_current_joint_angles()</code> (<i>neurapy.robot.Robot method</i>), 80	<code>get_joint_speed()</code> (<i>neurapy.robot.Robot method</i>), 88
<code>get_current_joint_angles_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 80	<code>get_linear_acceleration()</code> (<i>neurapy.robot.Robot method</i>), 88
<code>get_current_joint_torques()</code> (<i>neurapy.robot.Robot method</i>), 80	<code>get_linear_speed()</code> (<i>neurapy.robot.Robot method</i>), 88
<code>get_current_joint_torques_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 81	<code>get_load_side_encoder_values_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 88
<code>get_current_joint_velocities()</code> (<i>neurapy.robot.Robot method</i>), 81	<code>get_motor_side_encoder_values_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 89
<code>get_current_joint_velocities_with_timestamp()</code> (<i>neurapy.robot.Robot method</i>), 81	<code>get_override()</code> (<i>neurapy.robot.Robot method</i>), 89
<code>get_current_load_side_encoder_values()</code> (<i>neurapy.robot.Robot method</i>), 81	<code>get_point()</code> (<i>neurapy.robot.Robot method</i>), 89
<code>get_current_motor_side_encoder_values()</code> (<i>neurapy.robot.Robot method</i>), 82	<code>get_point_names()</code> (<i>neurapy.robot.Robot method</i>), 90
<code>get_current_tool_cogs()</code> (<i>neurapy.robot.Robot method</i>), 82	<code>get_power_status()</code> (<i>neurapy.robot.Robot method</i>), 90
<code>get_current_tool_inertias()</code> (<i>neurapy.robot.Robot method</i>), 82	<code>get_program_names()</code> (<i>neurapy.robot.Robot method</i>), 90
<code>get_current_tool_mass()</code> (<i>neurapy.robot.Robot method</i>), 82	<code>get_reference_frame()</code> (<i>neurapy.robot.Robot method</i>), 90
<code>get_current_tool_properties()</code> (<i>neurapy.robot.Robot method</i>), 83	<code>get_reference_frame_with_offset()</code> (<i>neurapy.robot.Robot method</i>), 91
<code>get_current_tool_rpy_offsets()</code> (<i>neurapy.robot.Robot method</i>), 83	<code>get_register_bytes()</code> (<i>neurapy.robot.Robot method</i>), 91
<code>get_current_tool_translation_offsets()</code> (<i>neurapy.robot.Robot method</i>), 83	<code>get_register_values()</code> (<i>neurapy.robot.Robot method</i>), 92
<code>get_diagnostics()</code> (<i>neurapy.robot.Robot method</i>), 83	<code>get_safety()</code> (<i>neurapy.robot.Robot method</i>), 92
<code>get_digital_input()</code> (<i>neurapy.robot.Robot method</i>), 84	<code>get_selected_fts_data()</code> (<i>neurapy.robot.Robot method</i>), 92
	<code>get_selected_tool_data()</code> (<i>neurapy.robot.Robot method</i>), 93
	<code>get_selected_tool_name()</code> (<i>neurapy.robot.Robot method</i>), 93
	<code>get_servo_trajectory_scaling_factor()</code> (<i>neurapy.robot.Robot method</i>), 93

- [get_tcp_force\(\)](#) (*neurapy.robot.Robot method*), 94
[get_tcp_pose\(\)](#) (*neurapy.robot.Robot method*), 95
[get_tcp_pose_quaternion\(\)](#) (*neurapy.robot.Robot method*), 95
[get_tcp_pose_quaternion_with_timestamp\(\)](#) (*neurapy.robot.Robot method*), 95
[get_tcp_pose_with_timestamp\(\)](#) (*neurapy.robot.Robot method*), 95
[get_tcp_speed\(\)](#) (*neurapy.robot.Robot method*), 96
[get_tcp_torque\(\)](#) (*neurapy.robot.Robot method*), 96
[get_tcp_wrench\(\)](#) (*neurapy.robot.Robot method*), 96
[get_tool_analog_input\(\)](#) (*neurapy.robot.Robot method*), 96
[get_tool_digital_input\(\)](#) (*neurapy.robot.Robot method*), 97
[get_tool_digital_output\(\)](#) (*neurapy.robot.Robot method*), 97
[get_tools\(\)](#) (*neurapy.robot.Robot method*), 97
[get_warnings\(\)](#) (*neurapy.robot.Robot method*), 98
[grasp\(\)](#) (*neurapy.robot.Robot method*), 98
[gripper\(\)](#) (*neurapy.robot.Robot method*), 98
- ## I
- [init_program\(\)](#) (*neurapy.robot.Robot method*), 98
[io\(\)](#) (*neurapy.robot.Robot method*), 99
[is_collision_enabled\(\)](#) (*neurapy.robot.Robot method*), 99
[is_external_device_connected\(\)](#) (*neurapy.robot.Robot method*), 99
[is_external_device_in_error\(\)](#) (*neurapy.robot.Robot method*), 100
[is_free_drive_mode_enabled\(\)](#) (*neurapy.robot.Robot method*), 100
[is_reflex_enabled\(\)](#) (*neurapy.robot.Robot method*), 100
[is_robot_in_automatic_mode\(\)](#) (*neurapy.robot.Robot method*), 101
[is_robot_in_collision\(\)](#) (*neurapy.robot.Robot method*), 101
[is_robot_in_semi_automatic_mode\(\)](#) (*neurapy.robot.Robot method*), 101
[is_robot_in_simulation\(\)](#) (*neurapy.robot.Robot method*), 102
[is_robot_in_teach_mode\(\)](#) (*neurapy.robot.Robot method*), 102
[is_robot_powered_on\(\)](#) (*neurapy.robot.Robot method*), 102
- ## J
- [jog\(\)](#) (*neurapy.robot.Robot method*), 102
- ## L
- [list_methods\(\)](#) (*neurapy.robot.Robot method*), 103
[list_recorded_path\(\)](#) (*neurapy.robot.Robot method*), 103
[load_pre_planned_motion_from_json\(\)](#) (*neurapy.robot.Robot method*), 104
- [lock_cartesian_axes\(\)](#) (*neurapy.robot.Robot method*), 104
[lock_joint_axes\(\)](#) (*neurapy.robot.Robot method*), 105
- ## M
- [module](#)
[neurapy.robot](#), 40
[motion_status\(\)](#) (*neurapy.robot.Robot method*), 105
[move_circular\(\)](#) (*neurapy.robot.Robot method*), 105
[move_composite\(\)](#) (*neurapy.robot.Robot method*), 110
[move_joint\(\)](#) (*neurapy.robot.Robot method*), 117
[move_joint_relative\(\)](#) (*neurapy.robot.Robot method*), 120
[move_linear\(\)](#) (*neurapy.robot.Robot method*), 124
[move_linear_relative\(\)](#) (*neurapy.robot.Robot method*), 130
[move_pose\(\)](#) (*neurapy.robot.Robot method*), 135
[move_recorded_path\(\)](#) (*neurapy.robot.Robot method*), 135
[move_spline\(\)](#) (*neurapy.robot.Robot method*), 136
[movelinear_online\(\)](#) (*neurapy.robot.Robot method*), 138
- ## N
- [neurapy.robot](#)
[module](#), 40
[notify_error\(\)](#) (*neurapy.robot.Robot method*), 139
[notify_info\(\)](#) (*neurapy.robot.Robot method*), 139
[notify_warning\(\)](#) (*neurapy.robot.Robot method*), 139
- ## P
- [pause\(\)](#) (*neurapy.robot.Robot method*), 140
[plan_move_circular\(\)](#) (*neurapy.robot.Robot method*), 140
[plan_move_composite\(\)](#) (*neurapy.robot.Robot method*), 144
[plan_move_joint\(\)](#) (*neurapy.robot.Robot method*), 151
[plan_move_linear\(\)](#) (*neurapy.robot.Robot method*), 154
[plan_move_recorded_path\(\)](#) (*neurapy.robot.Robot method*), 160
[plan_move_spline\(\)](#) (*neurapy.robot.Robot method*), 161
[plan_realtime_wait\(\)](#) (*neurapy.robot.Robot method*), 162
[power_off\(\)](#) (*neurapy.robot.Robot method*), 162
[power_on\(\)](#) (*neurapy.robot.Robot method*), 163
[pre_plan_move_circular\(\)](#) (*neurapy.robot.Robot method*), 163
[pre_plan_move_composite\(\)](#) (*neurapy.robot.Robot method*), 167
[pre_plan_move_joint\(\)](#) (*neurapy.robot.Robot method*), 174

- [pre_plan_move_joint_relative\(\)](#) (*neurapy.robot.Robot method*), 177
[pre_plan_move_linear\(\)](#) (*neurapy.robot.Robot method*), 182
[pre_plan_move_linear_relative\(\)](#) (*neurapy.robot.Robot method*), 188
[pre_plan_move_recorded_path\(\)](#) (*neurapy.robot.Robot method*), 192
[pre_plan_move_spline\(\)](#) (*neurapy.robot.Robot method*), 193
[pre_plan_realtime_wait\(\)](#) (*neurapy.robot.Robot method*), 194
[program_status\(\)](#) (*neurapy.robot.Robot method*), 194
- ## Q
- [quaternion_to_rpy\(\)](#) (*neurapy.robot.Robot method*), 195
- ## R
- [rad2encoder\(\)](#) (*neurapy.robot.Robot method*), 195
[read_general_purpose_registers\(\)](#) (*neurapy.robot.Robot method*), 195
[read_safeio\(\)](#) (*neurapy.robot.Robot method*), 195
[realtime_wait\(\)](#) (*neurapy.robot.Robot method*), 196
[release\(\)](#) (*neurapy.robot.Robot method*), 196
[remove_load\(\)](#) (*neurapy.robot.Robot method*), 196
[reparent_pose\(\)](#) (*neurapy.robot.Robot method*), 196
[reset_axes_lock\(\)](#) (*neurapy.robot.Robot method*), 197
[reset_collision\(\)](#) (*neurapy.robot.Robot method*), 197
[reset_control\(\)](#) (*neurapy.robot.Robot method*), 198
[reset_errors\(\)](#) (*neurapy.robot.Robot method*), 198
[reset_external_device_actions\(\)](#) (*neurapy.robot.Robot method*), 198
[reset_external_device_error\(\)](#) (*neurapy.robot.Robot method*), 198
[reset_led_mode\(\)](#) (*neurapy.robot.Robot method*), 199
[reset_warnings\(\)](#) (*neurapy.robot.Robot method*), 199
[Robot](#) (*class in neurapy.robot*), 40
[rpy_to_quaternion\(\)](#) (*neurapy.robot.Robot method*), 199
- ## S
- [save_pre_planned_motion_to_json\(\)](#) (*neurapy.robot.Robot method*), 200
[servo_j\(\)](#) (*neurapy.robot.Robot method*), 200
[servo_x\(\)](#) (*neurapy.robot.Robot method*), 201
[set_analog_output\(\)](#) (*neurapy.robot.Robot method*), 203
[set_current_position_as_zero\(\)](#) (*neurapy.robot.Robot method*), 203
[set_dblout_register\(\)](#) (*neurapy.robot.Robot method*), 204
[set_digital_output\(\)](#) (*neurapy.robot.Robot method*), 204
[set_encoder_offsets\(\)](#) (*neurapy.robot.Robot method*), 205
[set_ext_digital_output\(\)](#) (*neurapy.robot.Robot method*), 205
[set_external_device_actions\(\)](#) (*neurapy.robot.Robot method*), 206
[set_intout_register\(\)](#) (*neurapy.robot.Robot method*), 207
[set_joint_acceleration\(\)](#) (*neurapy.robot.Robot method*), 207
[set_joint_speed\(\)](#) (*neurapy.robot.Robot method*), 208
[set_led_mode\(\)](#) (*neurapy.robot.Robot method*), 208
[set_linear_acceleration\(\)](#) (*neurapy.robot.Robot method*), 209
[set_linear_speed\(\)](#) (*neurapy.robot.Robot method*), 209
[set_load\(\)](#) (*neurapy.robot.Robot method*), 210
[set_opcua_msg\(\)](#) (*neurapy.robot.Robot method*), 210
[set_override\(\)](#) (*neurapy.robot.Robot method*), 210
[set_safety\(\)](#) (*neurapy.robot.Robot method*), 211
[set_tool\(\)](#) (*neurapy.robot.Robot method*), 211
[set_tool_digital_output\(\)](#) (*neurapy.robot.Robot method*), 211
[set_tool_digital_outputs\(\)](#) (*neurapy.robot.Robot method*), 212
[speed_j\(\)](#) (*neurapy.robot.Robot method*), 212
[speed_x\(\)](#) (*neurapy.robot.Robot method*), 213
[stop\(\)](#) (*neurapy.robot.Robot method*), 214
[stop_movelinear_online\(\)](#) (*neurapy.robot.Robot method*), 214
[switch_to_automatic_mode\(\)](#) (*neurapy.robot.Robot method*), 215
[switch_to_real\(\)](#) (*neurapy.robot.Robot method*), 215
[switch_to_semi_automatic_mode\(\)](#) (*neurapy.robot.Robot method*), 215
[switch_to_simulation\(\)](#) (*neurapy.robot.Robot method*), 216
[switch_to_teach_mode\(\)](#) (*neurapy.robot.Robot method*), 216
- ## T
- [tare_fts\(\)](#) (*neurapy.robot.Robot method*), 216
[terminate_current_motion\(\)](#) (*neurapy.robot.Robot method*), 216
[turn_off_free_drive_mode\(\)](#) (*neurapy.robot.Robot method*), 216
[turn_off_jog\(\)](#) (*neurapy.robot.Robot method*), 217
[turn_on_free_drive_mode\(\)](#) (*neurapy.robot.Robot method*), 217
[turn_on_jog\(\)](#) (*neurapy.robot.Robot method*), 217
- ## U
- [unpause\(\)](#) (*neurapy.robot.Robot method*), 218

`update_current_tool_parameters()` (*neurapy.robot.Robot method*), 218
`update_reference_frame()` (*neurapy.robot.Robot method*), 219
`update_tool_parameters()` (*neurapy.robot.Robot method*), 220

W

`wait()` (*neurapy.robot.Robot method*), 222
`wait_for_analog_input()` (*neurapy.robot.Robot method*), 222
`wait_for_digital_input()` (*neurapy.robot.Robot method*), 222
`wait_for_digital_input_timer_off_delay()` (*neurapy.robot.Robot method*), 223
`wait_for_digital_input_timer_on_delay()` (*neurapy.robot.Robot method*), 223
`wait_for_tool_analog_input()` (*neurapy.robot.Robot method*), 224
`wait_for_tool_digital_input()` (*neurapy.robot.Robot method*), 224
`wait_for_tool_digital_input_timer_off_delay()` (*neurapy.robot.Robot method*), 224
`wait_for_tool_digital_input_timer_on_delay()` (*neurapy.robot.Robot method*), 225
`wait_motion_finished()` (*neurapy.robot.Robot method*), 225

COPYRIGHT, CHANGES and LIABILITY

This document and any information contained herein, including pictures and drawings, is the sole property and confidential information of NEURA Robotics GmbH. No intellectual property rights or rights of use are granted by the delivery of this document or the disclosure of its content.

This document and its content shall not be used for any purpose other than that for which it is supplied. In particular, this document and any information contained therein shall not be reproduced, distributed, made publicly available and/or otherwise be disclosed to a third party without the express written consent of NEURA Robotics GmbH.

NEURA Robotics GmbH has made considerable efforts to ensure the content of this document is correct at the date of publication but does not give warranties regarding the content.

The statements in this document are based on the mentioned assumptions and are expressed in good faith. NEURA Robotics GmbH excludes liability, howsoever arising, for any inaccuracies in this document.

We reserve the right to make changes to our products and changes to the contents of this document at any time without prior notice.

The statements made herein do not constitute an offer. For orders, the respective agreed provisions are decisive.

© 2024 NEURA Robotics GmbH. All rights reserved.

